

Abstract

Virtual orbital cost is a fundamental bottleneck in post-Hartree–Fock quantum chemistry and near-term quantum computing. Computing correlation energy via MP2, coupled cluster, or variational quantum algorithms requires expensive summations over all virtual orbitals, severely limiting the molecular system size and circuit depth on NISQ devices. This thesis implements the Optimized Virtual Orbital Space (OVOS) method of Adamowicz & Bartlett (1987). The approach minimises a second-order Hylleraas functional via unrestricted Hartree–Fock reference states and Newton–Raphson orbital rotations. Validation across five representative molecules (H_2O , CO , HF , NH_3 , Li_2) in both 6–31G and cc–pVDZ basis sets demonstrates that 90% of full-space MP2 correlation energy is recovered using 50–68% of virtual orbitals, except Li_2 . Li_2 shows greater consistency with the original Adamowicz & Bartlett reference calculations at 28% of virtual orbitals. However, convergence becomes challenging as the active space approaches full dimension. OVOS optimised orbitals provide superior trial states for variational quantum eigensolvers compared to canonical Hartree–Fock, reducing circuit depth while maintaining energy accuracy. These findings demonstrate practical value for NISQ-era quantum chemistry.

Keywords: Optimized virtual orbital space, OVOS, MP2, orbital optimisation, virtual orbital reduction, quantum computing, PySCF, Newton-Raphson.

Acknowledgements

I would like to express my deepest gratitude to my main supervisor, Assistant Professor Phillip W. K. Jensen, for his invaluable guidance, support, and encouragement throughout this project. His expertise and insights have been instrumental in shaping the direction of my research and helping me navigate challenges along the way. I am also grateful to my co-supervisor, Professor Stephan P. A. Sauer, for his constructive feedback and support. I would like to thank my colleagues and friends in the Sauer Group at the University of Copenhagen for their camaraderie and insightful discussions that enriched my research experience during group meetings and informal office discussions. I also want to acknowledge the opportunity to join the HQC² meetings, which provided me with a broader perspective on the field and allowed me to connect with other researchers. Finally, I want to thank my partner, family, and friends for their unwavering support and encouragement throughout this journey. This thesis would not have been possible without the contributions of all these individuals, and I am deeply grateful for their support.

This thesis builds on a preliminary PREP project on the Correlated Optimized Virtual Orbitals (COVO) method [1]. Wherever concepts or derivations from that earlier work are reused, they are cited accordingly. The preliminary PREP project was supervised by Professor Stephan P. A. Sauer and Assistant Professor Phillip W. K. Jensen.

List of Abbreviations

HF	Hartree-Fock
RHF	Restricted Hartree-Fock
UHF	Unrestricted Hartree-Fock
MP2	Second-Order Møller-Plesset Perturbation Theory
CI	Configuration Interaction
FCI	Full Configuration Interaction
CC	Coupled Cluster
CCSD	Coupled Cluster Singles and Doubles
OVOS	Optimized Virtual Orbital Space
NO	Natural Orbitals
FNO	Frozen Natural Orbitals
VQE	Variational Quantum Eigensolver
NISQ	Noisy Intermediate-Scale Quantum
QC	Quantum Computing
AO	Atomic Orbital
MO	Molecular Orbital
SCF	Self-Consistent Field
LCAO	Linear Combination of Atomic Orbitals
ERI	Electron Repulsion Integral
GTO	Gaussian-Type Orbital
STO	Slater-Type Orbital
NR	Newton-Raphson
RLE	Reduced Linear Equation
PySCF	Python-based Simulations of Chemistry Framework
UCPH	University of Copenhagen

Contents

Abstract	i
Acknowledgements	ii
List of Abbreviations	iii
1 Introduction	1
1.1 Motivation	1
1.2 The Virtual Orbital Space Problem	2
1.3 Quantum Computing Context	2
1.4 Objectives	3
1.5 Thesis Outline	3
1.6 Notation and Integral Conventions	4
2 Theory	5
2.1 The Electronic Structure Problem	5
2.2 Wavefunction Methods	8
2.3 Background for understanding OVOS	11
2.4 Relevance to Quantum Computing	25
3 OVOS Implementation	29
3.1 Algorithm	29
3.2 Initialisation Strategies	38
4 Computational Details	41
4.1 OVOS Calculation Settings	42
4.2 VQE Calculation Settings	44
5 Results	46
5.1 OVOS	46
5.2 VQE	57
5.3 ooVQE	60
6 Conclusion and Outlook	64
6.1 Conclusion	64
6.2 Outlook	66
Bibliography	67
A Implementation Validation	71
B Additional Data and Figures	72
C Implementation Details and Source Code	84

Chapter 1

Introduction

This chapter introduces the motivation, context, and objectives of this thesis, which focuses on the optimisation of virtual orbital spaces for post-Hartree-Fock methods and their implications for quantum computing. We begin by outlining the computational challenges in quantum chemistry that motivate the need for orbital optimisation. We then review existing strategies for virtual orbital reduction, leading to the OVOS method, which serves as the focus of this work. Finally, we state the specific research objectives and outline the thesis structure.

1.1 Motivation

Chemistry fundamentally arises from the quantum mechanical interactions of electrons and nuclei [2]. The exact solution of the electronic Schrödinger equation for a molecule with N electrons scales as $\mathcal{O}(N!)$ in the size of the configurational space, which rapidly renders exact calculations (such as Full Configuration Interaction) infeasible for any but the smallest systems [3]. To achieve chemical accuracy (approximately 1 kcal/mol), it is essential to account for electron correlation effects beyond the mean-field Hartree-Fock (HF) approximation. Post-HF methods such as second-order Møller-Plesset perturbation theory (MP2), coupled cluster with single and double excitations (CCSD), and CCSD with perturbative triples [CCSD(T)] are widely used to capture these correlation effects [4]. However, the computational cost of these methods scales steeply with system size, primarily due to the virtual orbital space. The number of virtual orbitals grows with the size of the basis set, and the computational cost of post-HF methods scales as $\mathcal{O}(O^2V^2)$ for MP2 (AO to MO basis), $\mathcal{O}(V^4O^2)$ for CCSD, and $\mathcal{O}(O^3V^4)$ for CCSD(T), where O is the number of occupied and V is the number of virtual orbitals, notation from [5]. As occupied orbitals are typically fewer than virtual orbitals, the virtual space dominates the computational cost.

This challenge is not only relevant for classical computational chemistry but also has direct implications for quantum computing. Quantum algorithms for chemistry, such as the Variational Quantum Eigensolver (VQE) [6], require a mapping of the electronic structure problem onto qubits [6]. The number of qubits required scales with the number of orbitals. Although current NISQ devices may have over 100 physical qubits (e.g., IBM's systems with 130+ qubits), device noise severely limits the number of qubits that can be reliably used. Reducing the number of orbitals (the virtual orbital space) is therefore crucial to minimise qubit requirements and mitigate the effects of noise, enabling chemistry simulations feasible in the near term [7]. This motivates a closer examination of the virtual orbital space itself, which is the bottleneck for all post-HF methods and is the subject of the following section.

1.2 The Virtual Orbital Space Problem

The virtual orbital space, which consists of the unoccupied molecular orbitals, plays a critical role in post-Hartree-Fock methods as it defines the space of excited configurations that contribute to electron correlation. The number of virtual orbitals increases with the size of the basis set, and their contributions to the correlation energy are not uniform. This section surveys various strategies developed to reduce or optimise the virtual orbital space, ultimately motivating the focus of this thesis on the Optimised Virtual Orbital Space (OVOS) method, originally proposed by Adamowicz and Bartlett in 1987 [8].

The standard approach to defining virtual orbitals is to use the canonical Hartree-Fock (HF) virtual orbitals, which are obtained by diagonalising the Fock matrix [5]. These orbitals are optimal for describing the single-determinant HF state. However, when we extend to multi-determinantal post-HF methods such as configuration interaction (CI) or coupled cluster (CC), the HF orbitals are no longer optimal for capturing electron correlation. This motivates the need to optimise the orbital space for correlated wavefunctions.

Natural orbitals, introduced by Löwdin in 1955 [9], are obtained by diagonalising the one-particle reduced density matrix (1-RDM) of an MP2 wavefunction. They are ordered by their occupation numbers, which can be used to truncate the virtual space. However, this approach requires an MP2 calculation to obtain the 1-RDM, creating a circular dependency.

Frozen natural orbitals (FNO) are a practical solution to this problem. They are computed from a low-cost correlated calculation, such as MP2, and the virtual orbitals are truncated based on an occupation threshold [10]. This method is widely used in practice and can significantly reduce the computational cost while retaining accuracy.

The OVOS method takes a different approach by directly optimising a subset of the virtual orbitals to maximise the second-order correlation energy [8]. Unlike natural orbitals and FNO, OVOS does not necessarily require an initial MP2 calculation and can be systematically applied to any desired number of virtual orbitals. The OVOS method is the focus of this thesis, and we will implement and benchmark it in the context of unrestricted Hartree-Fock (UHF) calculations using the PySCF software package [11].

1.3 Quantum Computing Context

Quantum computing has emerged as a promising paradigm for simulating quantum systems, including molecular electronic structure. Algorithms such as VQE can efficiently find ground state energies by preparing a parameterised quantum state and optimising the parameters using a classical optimiser [6]. However, current NISQ devices are severely limited in the number of qubits and the depth of quantum circuits they can reliably execute, primarily due to noise [7]. This makes the connection between orbital-space reduction and quantum resource requirements directly relevant to this thesis.

In quantum chemistry, the electronic structure problem is typically mapped onto qubits using transformations such as the Jordan-Wigner [12], which requires $Q = 2N_{\text{orb}}$ qubits for N_{orb} spatial orbitals. Given the limited number of qubits and the need for shallow circuits, reducing the number of virtual orbitals directly translates to a reduction in qubit requirements and circuit depth [7]. The OVOS method, by optimising a subset of the virtual orbitals, can significantly reduce the number of qubits needed for a given level of accuracy, $Q = 2N'_{\text{orb}}$, where $N'_{\text{orb}} < N_{\text{orb}}$ is the number of orbitals retained after OVOS optimisation [8].

1.4 Objectives

The thesis will seek to achieve the following specific objectives:

1. Implement and validate the OVOS algorithm of Adamowicz & Bartlett [8] using the PySCF software package, for a representative set of molecules and basis sets.
2. Characterise the energy recovery as a function of the active virtual orbital count for all test systems, quantifying the fraction of correlation energy recovered by the OVOS optimised orbitals.
3. Investigate whether OVOS optimised molecular orbitals satisfy the Brueckner orbital condition at the MP2 level (vanishing T1 amplitudes), and assess the theoretical connection between OVOS and orbital-optimised MP2 [13].
4. Quantify the potential benefit of OVOS optimised orbitals as a reference trial state for VQE type quantum computing algorithms: assess the expected improvement in reference-state overlap and compare initialisation strategies (default UHF, OVOS, UMP2 natural orbitals).

1.5 Thesis Outline

The structure of this thesis follows. Chapter 2 reviews the theoretical background necessary to understand the OVOS method, including the relevance to quantum computing. Chapter 3 describes the algorithmic implementation of the OVOS method, detailing code snippets of the full implementation. Chapter 4 details the computational setup, including the choice of test molecules, basis sets, and convergence criteria. Chapter 5 presents the results of applying OVOS to a set of test molecules, analysing the energy recovery as a function of active virtual orbital count and using the OVOS orbitals for quantum computing applications, particularly in the context of VQE algorithms. Finally, Chapter 6 summarises the key findings of the thesis, discusses the implications for quantum chemistry and quantum computing, and outlines directions for future research.

1.6 Notation and Integral Conventions

We establish the following specific notation and conventions for OVOS, maintaining consistency with the original Adamowicz & Bartlett paper [8].

Index Ordering and Orbital Partitioning

Orbital indices follows the standard convention:

- $i, j, k, l, \dots$ for occupied orbitals
- $a, b, c, d, \dots$ for unoccupied orbitals
- $p, q, r, s, \dots$ for unrestricted orbitals

In OVOS, the virtual orbitals are further partitioned into active ($a, b, \dots$) and inactive ($e, f, \dots$) subsets, with the active space containing N' orbitals optimised for correlation recovery.

Integral Notation: Antisymmetrised vs Non-antisymmetrised

For integral notation, we distinguish between the antisymmetrised physicists' notation $\langle ab||ij \rangle$ and the non-antisymmetrised chemists' notation $\langle ab|ij \rangle$. The antisymmetrised integral is defined as:

$$\langle ab||ij \rangle = \langle ab|ij \rangle - \langle ab|ji \rangle$$

where $\langle ab|ij \rangle$ is the standard two-electron integral in the MO basis. The physicists' notation is used throughout the OVOS derivations for consistency with the original literature and to simplify the expressions for the gradient and Hessian.

Integral Notation: Chemists' vs. Physicists' Conventions

The notation for two-electron integrals can differ between chemists and physicists, leading to potential confusion when comparing formulas from the literature with the output of quantum chemistry software. In chemists' notation, the integral is written as $(pq|rs)$, while in physicists' notation, it is written as $\langle pq|rs \rangle = (pr|qs)$ [5]. The antisymmetrised integral, which is relevant for post-HF methods, is defined as:

$$\langle pq||rs \rangle = \langle pq|rs \rangle - \langle pq|sr \rangle = (pr|qs) - (ps|qr)$$

The OVOS formulas in this thesis use the antisymmetrised physicists' notation, while PySCF returns integrals in the chemists' notation[11]. The conversion between these notations is explicitly implemented in the code as: $\langle pq||rs \rangle = (pr|qs) - (ps|qr)$. This establishes a clear and consistent framework for interpreting the integrals used in the OVOS method and ensures that the theoretical formulas align with the computational implementation.

Chapter 2

Theory

This chapter is meant to provide the theoretical background necessary to understand the OVOS method and its implementation, and how we extend it to Quantum computing. The content is structured to build from fundamental concepts in electronic structure theory to the specific mathematical formulations used in OVOS. The chapter is structured in four sections: (2.1) the electronic structure problem and the importance of electron correlation, (2.2) the wavefunction methods that are the basis for OVOS, (2.3) the mathematical framework for OVOS, and (2.4) the relevant aspects of OVOS on quantum computers.

2.1 The Electronic Structure Problem

The electronic structure problem is the task of solving the non-relativistic, time-independent Schrödinger equation for a system of N electrons moving in the electrostatic potential of M fixed nuclei, reflecting the Born-Oppenheimer approximation.¹ Typically, the electronic system is expressed as the Schrödinger equation [5]:

$$\hat{H}_{el}\Psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) = E\Psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) \quad (2.1)$$

where $\hat{H}_{el}$ is the electronic Hamiltonian, Ψ is the many-electron wavefunction, and E is the electronic energy. The electronic Hamiltonian in Equation 2.1 can be written as [5]:

$$\hat{H}_{el} = \sum_{i=1}^N \left(-\frac{1}{2} \nabla_i^2 - \sum_{A=1}^M \frac{Z_A}{|\mathbf{r}_i - \mathbf{R}_A|} \right) + \sum_{i < j} \frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} \quad (2.2)$$

where the first term represents the kinetic energy of the electrons, the second term is the nuclear attraction, and the third term is the electron-electron repulsion.

The electron-electron repulsion term couples all electrons together, making the exact solution of the Schrödinger equation computationally hard with increasing system size. While we can solve the Schrödinger equation exactly by Full Configuration Interaction (FCI), this approach is computationally infeasible for large systems, due to the exponential growth of the configuration space. This motivates the development of approximate methods that can capture the essential physics of electron correlation while remaining computationally feasible [2, 14].

2.1.1 Molecular Orbital Integrals

At the core of electronic structure methods are the one- and two-electron integrals evaluated in the molecular orbital basis. These integrals form the building blocks of

¹The Born-Oppenheimer approximation assumes that the nuclei are stationary due to their much larger mass compared to electrons, allowing the electronic and nuclear motions to be separated.

the Roothaan-Hall equations and the correlation energy expressions used in post-HF methods [5]. This section defines these integrals and establishes the notation conventions applied consistently throughout this thesis, resolving an important ambiguity that arises when comparing PySCF output with the formulas of Adamowicz & Bartlett [8].

One-Electron Integrals

One-electron integrals reflect the kinetic energy of electrons and their attraction to the nuclei [5]. In an atomic orbital (AO) basis, the overlap integral is defined as: $S_{\mu\nu} = \langle \mu | \nu \rangle = \int \phi_\mu^*(\mathbf{r}) \phi_\nu(\mathbf{r}) d\mathbf{r}$, the kinetic energy integral is: $T_{\mu\nu} = \langle \mu | -\frac{1}{2} \nabla^2 | \nu \rangle$, and the nuclear attraction integral is: $V_{\mu\nu} = \langle \mu | -\sum_A \frac{Z_A}{|\mathbf{r} - \mathbf{R}_A|} | \nu \rangle$ [5]. The core Hamiltonian in the AO basis is then given by: $h_{\mu\nu} = T_{\mu\nu} + V_{\mu\nu}$, see Eq. 2.2. To transform these integrals to the molecular orbital (MO) basis, we use the MO coefficients from the HF calculation: $h_{pq} = \sum_{\mu\nu} C_{\mu p}^* h_{\mu\nu} C_{\nu q}$, where $C_{\mu p}, C_{\nu q}$ are elements of the coefficient matrix C , that transforms from the AO to MO basis. This transformation is essential for evaluating the Fock matrix and the correlation energy in the MO basis. The one-electron integrals are straightforward to compute [11] and are typically stored in a dense matrix format.

Two-Electron Integrals

Two-electron integrals, also known as electron repulsion integrals (ERIs) are defined as: $(pq|rs) = \iint \phi_p^*(1) \phi_q(1) \frac{1}{r_{12}} \phi_r^*(2) \phi_s(2) d1 d2$ in chemists' notation, which is the convention used by PySCF [11]. These integrals represent the Coulomb repulsion between pairs of electrons and are central to the construction of the Fock matrix and the evaluation of correlation energies [5]. The ERIs exhibit an 8-fold permutation symmetry for real orbitals, which can be exploited to reduce computational cost. This allows for the storage of only unique integrals, for indices $p \geq q$ and $r \geq s$, and $(pq|rs) = (qp|rs) = (pq|sr) = (qp|sr) = (rs|pq) = (sr|pq) = (rs|qp) = (sr|qp)$. The Fock matrix in the MO basis can be expressed in terms of these integrals as: $F_{pq} = h_{pq} + \sum_i [2(p_i|q_i) - (pq|ii)]$ for RHF, where the sum runs over occupied orbitals. The two-electron integrals are typically stored in a 4-dimensional array, and their evaluation is the computational bottleneck in many electronic structure methods [14]. For UHF calculations, the Fock matrix has separate contributions from α and β electrons, leading to a larger number of integrals to be evaluated and stored.

2.1.2 Orbital Space

A quantum state in electronic structure theory is described by a wavefunction that depends on the coordinates of all electrons, and can be represented by Slater determinants, which are products of spatial orbitals and spin functions,

$$\chi_p(\mathbf{r}, m_s) = \phi_p(\mathbf{r}) \sigma(m_s) \quad (2.3)$$

where $\phi_p(\mathbf{r}) = \sum_\mu C_{\mu p} \phi_\mu(\mathbf{r})$ is the spatial orbital expanded in an atomic basis [5]. The spatial part of the spin-orbitals comes from basis sets, which are typically composed of Gaussian-type orbitals (GTOs) or Slater-type orbitals (STOs) [14]. The choice of

basis set can significantly affect the accuracy and computational cost of electronic structure calculations. Commonly used basis sets include minimal basis sets like STO-3G, split-valence basis sets like 6-31G [15–17], and correlation-consistent basis sets like cc-pVDZ [18]. The size and quality of the basis set determine the number of spin-orbitals available for constructing the wavefunction, which in turn affect the description of electron correlation and the convergence of post-HF methods [5, 14].

2.1.3 Spin-Orbital Formalism

The spin-orbital formalism provides a framework for unrestricted calculations by treating each spatial orbital as giving rise to a pair of spin-orbitals, one for each spin projection (α and β) [5]. A spin-orbital is defined as $\chi_{p,\sigma}(\mathbf{r}, m_s) = \phi_p(\mathbf{r})\sigma(m_s)$, where $\phi_p(\mathbf{r})$ is the spatial part of the orbital and $\sigma(m_s)$ is the spin function, which can be either α (spin-up) or β (spin-down). The total number of spin-orbitals is therefore $n_{spin} = 2 \times n_{spatial}$. In this implementation, an interleaved ordering convention is used for the spin-orbitals, even indices correspond to α spin-orbitals, while odd indices correspond to β spin-orbitals, as illustrated in Table 2.1. The total number of spatial orbitals is $n_{spatial} = n_{spin}/2$, and the mapping is as follows, index 0 corresponds to spatial orbital 0 with α spin, index 1 corresponds to spatial orbital 0 with β spin, index 2 corresponds to spatial orbital 1 with α spin, etc.

This interleaved ordering allows for a unified treatment of α and β electrons in the construction of the Fock matrix and the evaluation of integrals. The Fock matrix in the spin-orbital basis is block-diagonal, with separate blocks for α and β spins.

$F_{spin} = \begin{pmatrix} F_\alpha & 0 \\ 0 & F_\beta \end{pmatrix}$ where F_α and F_β are the Fock matrices for the α and β spin-orbitals, respectively [5].

The two-electron integrals in the spin-orbital basis are also constructed according to this interleaved ordering, with specific blocks corresponding to different spin combinations (e.g., $\alpha\alpha$, $\beta\beta$, $\alpha\beta$) [5]. After the construction of the Fock matrix and the integrals in the spin-orbital basis, the spin-orbitals are re-sorted by their orbital energies to form the canonical ordering used in all subsequent summations and calculations [5]. This ensures that the occupied and virtual orbitals are correctly identified according to their energies, which is essential for the accurate evaluation of correlation energies and amplitudes in post-HF methods [14].

Spatial Orbital	α Index	β Index	Interleaved Sequence
0	0	1	$\chi_{0,\alpha}, \chi_{0,\beta}$
1	2	3	$\chi_{1,\alpha}, \chi_{1,\beta}$
2	4	5	$\chi_{2,\alpha}, \chi_{2,\beta}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 1$	$2n - 2$	$2n - 1$	$\chi_{n-1,\alpha}, \chi_{n-1,\beta}$

Table 2.1: Interleaved α/β spin-orbital ordering. Even spin-orbital indices correspond to α (spin-up) orbitals, while odd indices correspond to β (spin-down) orbitals. The total number of spin-orbitals is $n_{spin} = 2n_{spatial}$.

2.2 Wavefunction Methods

This section introduces the wavefunction methods that form the basis for the OVOS method developed in this thesis. We start with Hartree-Fock theory, which provides the mean-field reference state, and then discuss post-HF methods that capture electron correlation. The mathematical framework of these methods is essential for understanding the derivation and implementation of OVOS, which builds upon the MP2 formalism to optimise the virtual orbital space for improved correlation energy recovery.

2.2.1 Hartree-Fock Theory

Hartree-Fock (HF) theory approximates the N-electron wavefunction as a single Slater determinant of N orthonormal spin-orbitals, treating electron-electron repulsion in a mean-field sense [5]. Despite its simplicity, HF underpins virtually all post-HF correlated methods.

Slater determinants are described by the Pauli exclusion principle, ensuring that no two electrons can occupy the same quantum state. This antisymmetry is crucial for accurately describing fermionic systems. The variational principle leads to the Fock equations, which can be expressed in matrix form as the Roothaan-Hall equations[5]: $FC = SC\epsilon$, where F is the Fock matrix, S is the overlap matrix, C is the coefficient matrix for the molecular orbitals, and ϵ is the diagonal matrix of orbital energies. The Fock matrix incorporates the Coulomb (J) and exchange (K) operators, which represent the average electron-electron repulsion and the antisymmetry of the wavefunction, respectively. It is this equation that is solved iteratively in the self-consistent field (SCF) procedure. The Fock matrix depends on the molecular orbital coefficients (the solution itself), creating a circular dependency, the coefficients are needed to construct the Fock matrix, but solving the Fock matrix produces the coefficients. This self-consistent iteration continues until convergence, meaning the coefficients no longer change between iterations, hence the name "self-consistent field." (SCF). The variational principle ensures that the HF energy is an upper bound to the exact ground state energy, $E_{HF} = \langle \Phi_{HF} | \hat{H} | \Phi_{HF} \rangle \geq E_{exact}$ [2]. This allows us to define that the correlation energy is negative: $E_{corr} = E_{exact} - E_{HF} < 0$. The resulting orbital energies ϵ_p provide insight into the electronic structure, with occupied orbitals typically having negative energies and virtual orbitals having positive energies, as per Koopmans' theorem [5].

Restricted and Unrestricted Formalism

Hartree-Fock theory varies in its treatment of spin, leading to two different formalisms, Restricted Hartree-Fock (RHF) and Unrestricted Hartree-Fock (UHF). For spin-orbital formalism see Section 2.1.3 above.

In RHF, the same spatial orbital ϕ_p is used for both α and β electrons, expressed as the constraint $\phi_\alpha = \phi_\beta$. Consequently, each spatial orbital is doubly occupied by electrons of opposite spin, forming a single set of molecular orbitals. This constraint makes RHF suitable for closed-shell systems and ensures that the wavefunction is an eigenfunction of the total spin operator $\hat{S}^2$, keeping the total spin of the system

fixed [5]. However, RHF fails to describe open-shell systems and bond dissociation correctly, as the spin-symmetry constraint cannot be satisfied in these cases [14].

In contrast, UHF allows different spatial orbitals for α and β spins ($\phi_\alpha \neq \phi_\beta$), making it applicable to open-shell systems [5]. This flexibility introduces spin contamination, where $\langle \hat{S}^2 \rangle$ deviates from the ideal value for a pure spin state [14]. In bond breaking and open-shell scenarios, UHF provides qualitatively correct descriptions where RHF fails [5]. For this reason, UHF is used throughout this thesis for generality, as it handles both closed-shell and open-shell systems effectively. While UHF incurs higher computational cost due to separate α and β orbital sets, incurring an additional degree of freedom, the scaling with the number of basis functions remains similar to RHF [5].

Limitations of Hartree-Fock and the Need for Correlation

Hartree-Fock theory provides a mean-field approximation to the electronic structure problem [2]. It captures the average effect of electron-electron repulsion but neglects the instantaneous correlation between electrons, which is essential for an accurate description of molecular properties and reaction energetics [5]. The failure of HF to account for electron correlation leads to significant errors in predicted energies, particularly for systems with strong correlation effects, such as bond dissociation and transition states [14]. This deficiency motivates the development of post-HF methods that explicitly include electron correlation, such as Møller-Plesset perturbation theory (MP2) [5]. These methods build upon the HF reference state to recover correlation energy and provide more accurate descriptions of molecular systems.

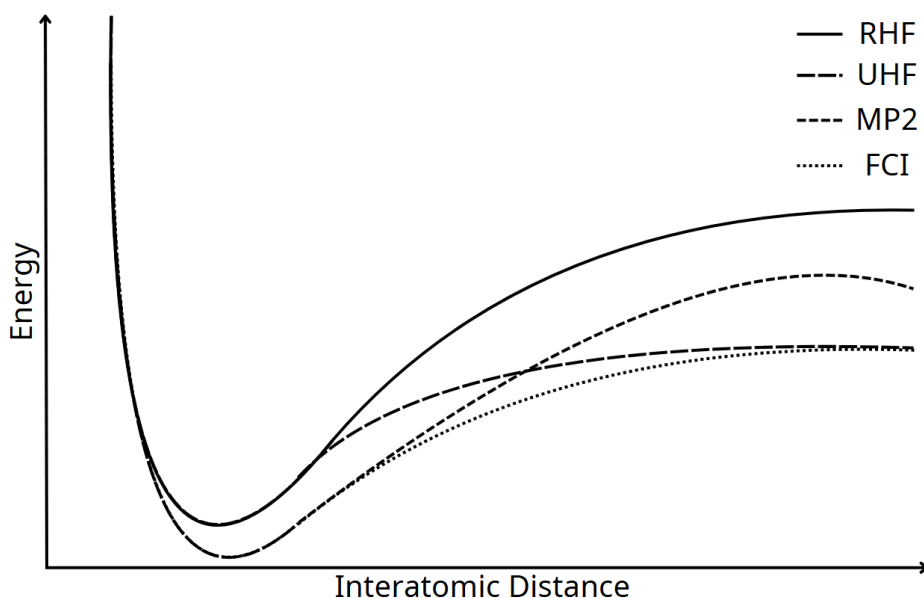


Figure 2.1: Potential energy curves for the Hydrogen atom illustrating the failure of HF near dissociation and the importance of electron correlation. The HF curve deviates significantly from the exact FCI curve as the bond is stretched, while MP2 captures much of the correlation energy and follows the FCI curve more closely around the equilibrium bond length. (Adapted from Helgaker et al. [5] Figs. 5.20 and 5.12.)

2.2.2 Møller-Plesset Perturbation Theory

Møller-Plesset perturbation theory (MPPT) treats electron correlation as a perturbation on top of the mean-field Fock operator. Truncated at second order (MP2), it is the simplest post-HF method that captures dynamical correlation at $\mathcal{O}(O^2V^2)$ computational cost, and furnishes the energy functional that OVOS seeks to optimise.

MP2 Correlation Energy

The MP2 correlation energy is derived by partitioning the electronic Hamiltonian as $\hat{H} = \hat{f} + \lambda\hat{V}$, where $\hat{f}$ is the Fock operator, which is diagonal in the canonical molecular orbital basis, and $\hat{V}$ is the fluctuation potential that accounts for electron correlation [5]. The zeroth-order solution is the Hartree-Fock determinant $|\Phi_0\rangle$. Brillouin's theorem states that singly-excited determinants have zero coupling to $|\Phi_0\rangle$, consequently, first-order perturbation theory contributes no singles terms [5, 14]. The first-order correction to the wavefunction is therefore a linear combination of doubly-excited determinants only. The associated MP1 amplitudes for these double excitations are:

$$t_{ij}^{ab} = -\frac{\langle ab||ij\rangle}{\varepsilon_a + \varepsilon_b - \varepsilon_i - \varepsilon_j} \quad (2.4)$$

where $\langle ab||ij\rangle$ is the antisymmetrised two-electron integral in physicists' notation, see Section 1.6 for notation details, and the denominator $\varepsilon_a + \varepsilon_b - \varepsilon_i - \varepsilon_j$ is the energy cost of exciting occupied orbitals i, j to virtual orbitals a, b . [5, 14].

The MP2 correlation energy over all occupied and virtual orbitals is:

$$E_{corr}^{(2)} = \frac{1}{4} \sum_{i,j,a,b} t_{ij}^{ab} \langle ij||ab\rangle \quad (2.5a)$$

$$= \sum_{i>j, a>b} t_{ij}^{ab} \langle ij||ab\rangle \quad (2.5b)$$

where i, j denotes occupied orbitals and virtual orbitals denoted by a, b [5]. In practice, the summation can be restricted to $i > j$ and $a > b$ avoiding double counting, we drop the $1/4$ factor from Equation 2.5a, leading to 2.5b. It is critical to make clear that both of these expressions sum over the entire orbital space the full, occupied and virtual, MP2 energy. In Section 2.3.2, we will build Equation 2.5b with a truncated virtual orbital space, the MP2 energy used in OVOS, reducing computational cost while retaining most correlation energy. In the code implementation in Chapter 3, we use the restricted summation convention of Equation 2.5b to avoid double counting and ensure consistency with the amplitude definitions.

Lower-Order Amplitudes

In MP2, the MP1 amplitudes correspond to double electron excitations (T2 amplitudes), defined in Equation 2.4. The single excitations (S), the T1 amplitudes, do not appear in the MP2 energy expression due to Brillouin's theorem, which states that the matrix elements of the Hamiltonian between the Hartree-Fock determinant and singly-excited determinants vanish [5]. The presence of non-zero T1 amplitudes is an indicator that the reference determinant is not optimal, it suggests that the orbitals can be rotated to lower the energy [13, 19].

Including the T2 excitations, T1 and T2 excitations, in addition to the HF state, are commonly referred to as Single-Double (SD) space in regards to methods, e.g. Coupled Cluster SD (CCSD) [5]. Here, the stationarity condition $\nabla_R E(R) = 0$ defines optimal orbitals, which are formalized in quantum chemistry as Brueckner orbitals (named after Brueckner’s self-consistency principle) where the T1 amplitudes vanish [20]. This connection between orbital optimisation and the nature of the excitation amplitudes is a key motivation for developing methods like OVOS that seek to optimise the virtual orbital space to minimise correlation energy, effectively minimising the T1 amplitudes [8].

MP2 Limitations and Applicability

MP2 is size-consistent [5], meaning that the correlation energy of non-interacting subsystems is additive, see Equation 2.5b, which is a crucial property for accurately describing large systems and reaction energies [14]. However, MP2 is a single-reference method and can fail in cases where the reference determinant is not a good approximation to the true wavefunction, such as in bond dissociation or transition states [5, 7]. In such cases, higher-order methods or multi-reference approaches may be necessary to capture the correct physics [3].

Computational Scaling Motivation

The $\mathcal{O}(O^2V^2)$ computational scaling of MP2 presents a significant barrier for large systems and higher-order methods [7, 14]. The Optimized Virtual Orbital Space (OVOS) method, detailed in Chapter 2, applies a strategic solution, by restricting the correlation energy calculation to a small, actively-correlated subset of virtual orbitals, the effective scaling can be reduced to $\mathcal{O}(N_{occ}^2 N_{virt}'^2)$, where $N_{virt}' \ll N_{virt}$ [8]. This active space reduction is particularly powerful when combined with higher-order methods like CCSD(T), which scale as $\mathcal{O}(O^3V^4)$ or worse in the full space. For which V reflects the size of the virtual space, and reducing V to N_{virt}' can make previously intractable calculations feasible while retaining much of the correlation energy. This scaling bottleneck directly drives the need for OVOS, especially when aiming for high-level correlation treatments.

2.3 Background for understanding OVOS

This section provides the theoretical background necessary for understanding the Optimized Virtual Orbital Space (OVOS) method developed in this thesis. We will cover the mathematical framework of orbital rotations, the convergence criteria that defines optimal orbitals, and the optimisation strategies employed to find these orbitals. This foundation is critical for grasping how OVOS optimises the virtual orbital space to maximise correlation energy while maintaining computational efficiency. The concepts introduced here will be directly applied in the implementation of OVOS in Chapter 3.

2.3.1 Orbital Rotations and Stationarity

Orbital optimisation in post-HF methods relies on rotating the molecular orbitals to minimise a target energy functional, such as the MP2 correlation energy in the case of OVOS. Here we set up the rotation formalism and the stationarity condition needed to derive the OVOS equations.

Partitioning

Orbitals in electronic structure theory are often partitioned into different subspaces to facilitate the formulation of post-HF methods. The OVOS method partitions the full spin-orbital basis into three non-overlapping subsets, occupied, active unoccupied (active virtual), and inactive unoccupied (inactive virtual), see Figure 2.2(1). The occupied orbitals, denoted by indices i, j, k, l , correspond to the spin-orbitals that are occupied in the HF reference state, and they are indexed from 0 to $n_{elec} - 1$, where n_{elec} is the number of electrons. The active virtual orbitals, denoted by indices a, b, c, d , are the subset of unoccupied orbitals that are optimised in the OVOS method, and they are indexed from n_{elec} to $n_{elec} + N'_{virt} - 1$, where N'_{virt} is the number of active virtual orbitals and is a key parameter in the OVOS method. The inactive virtual orbitals, denoted by indices e, f, g, h , are the remaining unoccupied orbitals that are not explicitly rotated in the OVOS method. They define the complement space and are indexed from $n_{elec} + N'_{virt}$ to $n_{spin} - 1$, where n_{spin} is the total number of spin-orbitals. The index notation is summarised in Table 2.2. The OVOS rotation operates only in the block that couples the active virtuals with the inactive virtuals while the occupied orbitals remain fixed, see Figure 2.2(2). The full set of virtual orbitals can be denoted as $a..e$, which includes all orbitals from n_{elec} to $n_{spin} - 1$ for a full MP2 calculation, see Figure 2.2(3).

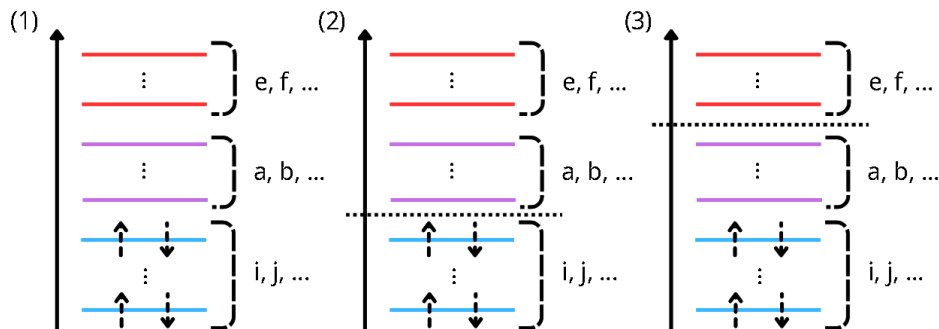


Figure 2.2: Partitioning of spin-orbital space for OVOS into occupied (blue), active virtual (purple), and inactive virtual subspaces. (1) No partitioning. (2) Occupied and unoccupied orbitals. (3) Active and inactive subsets.

By this partitioning scheme, the OVOS method focuses on optimising the active virtual subspace to minimise the correlation energy, while the inactive virtuals provide a fixed reference that defines the complement space [8]. This truncation reduces the computational cost of MP2 from $\mathcal{O}(O^2V^2)$ to $\mathcal{O}(O^2N'^2)$, where N' is the number of active virtuals, which is typically much smaller than the total number of virtuals $N' < V$. The choice of N' is a critical aspect of the OVOS method, as it involves a trade-off between accuracy and computational cost.

Label	Indices	Orbital Space	Range	Count
Occupied	i, j, k, l	occ.	$[0, n_{elec})$	n_{elec}
Active Virtual	a, b, c, d	act. virt.	$[n_{elec}, n_{elec} + N'_{virt})$	N'_{virt}
Inactive Virtual	e, f, g, h	inact. virt.	$[n_{elec} + N'_{virt}, n_{spin})$	$N_{virt} - N'_{virt}$
All Virtual	$a..h$	full virt.	$[n_{elec}, n_{spin})$	N_{virt}

Table 2.2: Orbital subspace notation used throughout this thesis.

2.3.2 The Second-Order Hylleraas Functional

OVOS minimises the second-order Hylleraas functional J_2 with respect to the choice of active virtual orbitals [8]. This section seeks to derive J_2 from the second-order Rayleigh-Schrödinger perturbation expression and to establish its connection to the MP2 correlation energy.

From Perturbation Theory to the Hylleraas Principle

We arrive at the Hylleraas functional by starting from the Rayleigh-Schrödinger (RS) perturbation theory expression for the second-order energy correction [5], starting by an expansion of the total energy:

$$E = E^{(0)} + E^{(1)} + E^{(2)} + \dots \quad (2.6)$$

where the zeroth-order energy $E^{(0)}$ is the energy of the zero-order Hamiltonian H_0 (The Fock Operator), and the first-order energy $E^{(1)}$ is the expectation value of the correlation perturbation operator $V = H - H_0$ with respect to the zero-order wavefunction $\Psi^{(0)}$,

$$E^{(1)} = \langle \Psi^{(0)} | V | \Psi^{(0)} \rangle \quad (2.7)$$

The sum of zero-order and first-order energies, $E^{(0)} + E^{(1)}$, is the Hartree-Fock energy.

The second-order energy correction $E^{(2)}$ is the MP2 correlation energy, expressed in terms of the correlation perturbation operator $V = H - H_0$ and the first-order wavefunction $\Psi^{(1)}$ as:

$$E^{(2)} = \langle \Psi^{(0)} | V | \Psi^{(1)} \rangle \quad (2.8)$$

where $\Psi^{(0)}$ is the Hartree-Fock reference wavefunction. Using intermediate normalization, $\langle \Psi^{(0)} | \Psi^{(1)} \rangle = 0$, and the first-order Schrödinger equation:

$$H_0 | \Psi^{(1)} \rangle + V | \Psi^{(0)} \rangle = E^{(0)} | \Psi^{(1)} \rangle + E^{(1)} | \Psi^{(0)} \rangle \quad (2.9)$$

we can rearrange to express:

$$(H_0 - E^{(0)}) | \Psi^{(1)} \rangle = -(V - E^{(1)}) | \Psi^{(0)} \rangle \quad (2.10)$$

take the inner product with $\langle \Psi^{(1)} |$:

$$\langle \Psi^{(1)} | H_0 - E^{(0)} | \Psi^{(1)} \rangle = -\langle \Psi^{(1)} | V - E^{(1)} | \Psi^{(0)} \rangle \quad (2.11)$$

using, $\langle \Psi^{(1)} | E^{(1)} | \Psi^{(0)} \rangle = E^{(1)} \langle \Psi^{(1)} | \Psi^{(0)} \rangle = 0$, we can rewrite the second-order energy correction as:

$$E^{(2)} \leq \langle \Psi^{(1)} | H_0 - E^{(0)} | \Psi^{(1)} \rangle + 2 \langle \Psi^{(1)} | V - E^{(1)} | \Psi^{(0)} \rangle \quad (2.12)$$

The second-order Hylleraas functional J_2 is defined as the upper bound to the second-order energy correction, such that $E^{(2)} \leq J_2$ for any choice of $\Psi^{(1)}$. In other words, minimising J_2 with respect to $\Psi^{(1)}$ recovers the MP2 correlation energy, $E^{(2)} = \min_{\Psi^{(1)}} J_2$. Minimising by varying $\Psi^{(1)}$ leads to the stationary condition that $\Psi^{(1)}$ must satisfy the first-order Schrödinger equation,

$$\frac{\partial J_2}{\partial \Psi^{(1)}} = 2(H_0 - E^{(0)})|\Psi^{(1)}\rangle + 2(V - E^{(1)})|\Psi^{(0)}\rangle = 0 \quad (2.13)$$

making the ansatz that $\Psi^{(1)}$ is a linear combination of doubly-excited determinants, $\Psi^{(1)} = \sum_{i>j, a>b} t_{ij}^{ab} |\Phi_{ij}^{ab}\rangle$, and $\Psi^{(0)}$ is the HF reference determinant. We can write out the MP2 amplitudes t_{ij}^{ab} as:

$$t_{ij}^{ab} = \frac{\langle ij || ab \rangle}{\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b} \quad (2.14)$$

where $\langle ij || ab \rangle$ are the antisymmetrised two-electron integrals in physicists' notation, and ϵ_p are the orbital energies.

Pair Decomposition and Explicit Form of J_2

To apply the Hylleraas functional to orbital optimisation, we decompose J_2 into unique pairs of occupied orbital indices $i > j$, each pair involves orthogonal configurations, $\langle \Phi_{ij}^{ab} | \Phi_{kl}^{cd} \rangle = 0$ for $(i, j) \neq (k, l)$, which translate to the functional being additively separable, $J_2 = \sum_{i>j} J_{ij}^{(2)}$, each pair contributes:

$$J_{ij}^{(2)} = \langle \Psi_{ij}^{(1)} | H_0 - E^{(0)} | \Psi_{ij}^{(1)} \rangle + 2 \langle \Psi_{ij}^{(1)} | V - E^{(1)} | \Psi^{(0)} \rangle \quad (2.15)$$

where we moved the sum over $i > j$ inside the definition of $\Psi_{ij}^{(1)}$ out to allow writing the decomposed form. Furthermore, we can decompose $J_{ij}^{(2)}$ into two contributions, the expectation value of the zero-order Hamiltonian (first term) and the coupling to the reference state via the perturbation operator (second term).

The Zero-Order Term

The "diagonal" term takes the form when we evaluate the expectation value:

$$\langle \Psi_{ij}^{(1)} | H_0 - E^{(0)} | \Psi_{ij}^{(1)} \rangle = \sum_{a>b, c>d} t_{ij}^{ab} t_{ij}^{cd} \langle \Phi_{ij}^{ab} | (H_0 - E^{(0)}) | \Phi_{ij}^{cd} \rangle \quad (2.16)$$

To evaluate, we recall that $H_0 = \sum_i f_{ii}$, where f_{pq} is the Fock matrix element. The first-order energy is $E^{(0)} = \sum_i \epsilon_i$, which represents the sum of occupied orbital energies. The matrix element of H_0 between the doubly-excited determinants can be evaluated to:

$$\langle \Phi_{ij}^{ab} | H_0 | \Phi_{ij}^{cd} \rangle = f_{ac} \delta_{bd} - f_{ad} \delta_{bc} + f_{bd} \delta_{ac} - f_{bc} \delta_{ad} \quad (2.17)$$

The four term Fock structure arises from the antisymmetrised electron exchange and orbital index rearrangements, $\langle \Phi_{ij}^{ab} | f_{pq} | \Phi_{ij}^{cd} \rangle$ is non-zero only when the orbital indices match, which is selected by the δ structure.

The matrix element of $E^{(0)}$ is:

$$\langle \Phi_{ij}^{ab} | E^{(0)} | \Phi_{ij}^{cd} \rangle = (\epsilon_i + \epsilon_j)(\delta_{ac}\delta_{bd} - \delta_{ad}\delta_{bc}) \quad (2.18)$$

Combined, the "diagonal" term becomes:

$$\langle \Psi_{ij}^{(1)} | H_0 - E^{(0)} | \Psi_{ij}^{(1)} \rangle = \sum_{a>b, c>d} t_{ij}^{ab} t_{ij}^{cd} [(f_{ac}\delta_{bd} - f_{ad}\delta_{bc} + f_{bd}\delta_{ac} - f_{bc}\delta_{ad}) \quad (2.19)$$

$$- (\epsilon_i + \epsilon_j)(\delta_{ac}\delta_{bd} - \delta_{ad}\delta_{bc})] \quad (2.20)$$

Evaluation of the Perturbative Coupling

The "off-diagonal" term, consisting of the perturbation operator V and the first-order energy $E^{(1)}$, assesses the coupling between the reference determinant and the doubly-excited determinants. Evaluating the matrix element gives for each:

$$\langle \Psi_{ij}^{(1)} | V | \Psi^{(0)} \rangle = \sum_{a>b} t_{ij}^{ab} \langle ij || ab \rangle \quad (2.21)$$

and,

$$\langle \Psi_{ij}^{(1)} | E^{(1)} | \Psi^{(0)} \rangle = E^{(1)} \langle \Psi_{ij}^{(1)} | \Psi^{(0)} \rangle = 0 \quad (2.22)$$

therefore the "off-diagonal" term reduces to:

$$2 \langle \Psi_{ij}^{(1)} | V - E^{(1)} | \Psi^{(0)} \rangle = 2 \sum_{a>b} t_{ij}^{ab} \langle ij || ab \rangle \quad (2.23)$$

The final expression for $J_{ij}^{(2)}$ is then:

$$J_{ij}^{(2)} = \sum_{a>b, c>d} t_{ij}^{ab} t_{ij}^{cd} (f_{ac}\delta_{bd} - f_{ad}\delta_{bc} + f_{bd}\delta_{ac} - f_{bc}\delta_{ad} - (\epsilon_i + \epsilon_j)(\delta_{ac}\delta_{bd} - \delta_{ad}\delta_{bc})) + 2 \sum_{a>b} t_{ij}^{ab} \langle ij || ab \rangle \quad (2.24)$$

As such we derived the explicit form of the second-order Hylleraas functional J_2 in terms of the MP2 amplitudes t_{ij}^{ab} , the Fock matrix elements f_{pq} , and the antisymmetrised two-electron integrals $\langle ij || ab \rangle$. This functional serves as the target for orbital optimisation in OVOS, where the active virtual orbitals are rotated to minimise J_2 .

2.3.3 Orbital Rotation Parameterisation

Optimising orbitals in OVOS is achieved by applying a unitary transformation, rotating the virtual orbitals to maximise the correlation energy captured by the active space [8]. This section introduces the mathematical formalism of orbital rotations, parameterised by an antisymmetric matrix $\mathbf{R}$, and how the unitary transformation $\mathbf{U} = e^{\mathbf{R}}$ is constructed to mix only the active and inactive virtual orbitals.

Orbital Rotation

Orbital rotations can be represented mathematically as unitary transformations of the molecular orbital coefficients [5]. A unitary transformation can be expressed as $U = e^{\mathbf{R}}$, where $\mathbf{R}$ is an antisymmetric matrix that parameterises the rotation. The new orbital coefficients after rotation are given by $C_{new} = C_{old}U$. The effect of this rotation

on the one-electron integrals is given by $h_{new} = U^\dagger h_{old} U$, and for the two-electron integrals, the transformation is more complex due to their four-index nature, but it can be expressed as $g_{new} = U^\dagger g_{old} U$ in a suitable tensor contraction form. The Fock matrix also transforms under the same unitary rotation: $F_{new} = U^\dagger F_{old} U$. The use of unitary transformations ensures that the orthonormality of the orbitals is preserved during the optimisation process [5].

Active and Inactive Virtual Subspaces

The antisymmetric matrix $\mathbf{R}$ is defined such that only the block corresponding to active-inactive virtual orbital mixing is non-zero. Specifically, R_{ae} is non-zero, while all other blocks (e.g., occupied-occupied R_{ij} , active-active R_{ab} , inactive-inactive R_{ef}) are zero.

We can restrict the orbital rotations to only mix active and inactive virtual orbitals because active-active rotations do not change the value of J_2 , as J_2 depends only on the choice of optimised orbitals and not on their specific linear combinations of basis states. In other words, any rotation within the active space leaves J_2 invariant:

$$J_2[\{\phi_a\}] = J_2[\{\sum_b U_{ab}\phi_b\}] \quad (2.25)$$

Inactive-inactive rotations do not affect the active space at all. We neglect all mixing with and between occupied orbitals, as it is not within the scope of the OVOS method.

We can write the full matrix $\mathbf{R}$ as:

$$R = \begin{bmatrix} R_{ij} & R_{ia} & R_{ie} \\ R_{aj} & R_{ab} & R_{ae} \\ R_{ej} & R_{eb} & R_{ef} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & R_{ae} \\ 0 & -R_{ae}^T & 0 \end{bmatrix} \quad (2.26)$$

where we relate the active-inactive block to its transpose by the antisymmetry condition, $R_{ea} = -R_{ae}$. The dimension of the non-zero block is $N' \times (N_{\text{unocc.}} - N')$.

Unitary Transformation via Matrix Exponential

The unitary transformation $\mathbf{U}$ that rotates the orbitals is given by the matrix exponential of $\mathbf{R}$:

$$\mathbf{U} = e^{\mathbf{R}} = \sum_{n=0}^{\infty} \frac{\mathbf{R}^n}{n!} \quad (2.27)$$

Unitary by construction, $\mathbf{U}^\dagger \mathbf{U} = \mathbf{I}$, is guaranteed by the antisymmetry of $\mathbf{R}$, $R = -R^\dagger$.

Expanding the virtual orbitals to second order in $\mathbf{R}$ gives:

$$\phi'_a = \sum_b U_{ab}\phi_b + \sum_e U_{ae}\phi_e \quad (2.28)$$

We can express an active orbital after rotation as:

$$\phi'_a = \phi_a + \sum_e R_{ea}\phi_e - \frac{1}{2} \sum_{b,e} R_{ea}R_{eb}\phi_b + \dots \quad (2.29)$$

where the original active orbital is mixed with the inactive orbitals via the first-order term, and the second-order term accounts for the fact that the new active orbital is

no longer orthogonal to the original active orbitals. These terms are redundant when applying the full matrix exponential instead, as it handles all terms, the expansion allows for a tractable expression for the gradient and Hessian of J_2 with respect to the rotation parameters, which are derived in the following sections.

2.3.4 Gradient Expression

The descent direction for OVOS optimisation is given by the gradient of the Hylleraas functional J_2 with respect to the orbital rotation parameters R_{ae} [8]. This section derives the expression for the gradient, which consists of two terms, a direct contribution from the active-inactive mixing and an indirect contribution via the MP2 virtual-virtual density matrix D_{ab} .

MP2 Virtual-Virtual Density Matrix

The second-order density matrix, which captures the correlation between pairs of active virtual orbitals, how strongly active virtual orbitals, how strongly a and b appear together in the MP1 wavefunction, is defined as:

$$D_{ab} = \sum_{i>j} \sum_c t_{ij}^{ac} t_{ij}^{bc} \quad (2.30)$$

where t_{ij}^{ac} are the MP2 amplitudes. The matrix D_{ab} is symmetric, $D_{ab} = D_{ba}$,

Gradient Expression

The gradient of J_2 with respect to the orbital rotation parameters R_{ae} is given by:

$$G_{ae} = \left. \frac{\partial J_2}{\partial R_{ae}} \right|_{R=0} \quad (2.31)$$

Here we evaluate the gradient at $R = 0$ to obtain the expression in terms of the unrotated orbitals. This condition means the Fock matrix and integrals are evaluated in the original orbital basis, such that the orbitals are canonical in the unoccupied space. This means in the Hylleraas functional, the Fock matrix elements f_{ac} are diagonal in the virtual space, $f_{ac} = \epsilon_a \delta_{ac}$, meaning we can express the "zero-order" term of J_2 in terms of the orbital energies:

$$\langle \Psi_{ij}^{(1)} | H_0 - E^{(0)} | \Psi_{ij}^{(1)} \rangle = \sum_{a>b, c>d} t_{ij}^{ab} t_{ij}^{cd} [(\epsilon_a \delta_{ac} \delta_{bd} - \epsilon_a \delta_{ad} \delta_{bc} + \epsilon_b \delta_{bd} \delta_{ac} - \epsilon_b \delta_{bc} \delta_{ad})] \quad (2.32)$$

$$- (\epsilon_i + \epsilon_j) (\delta_{ac} \delta_{bd} - \delta_{ad} \delta_{bc})] \quad (2.33)$$

$$= \sum_{a>b, c>d} t_{ij}^{ab} t_{ij}^{cd} [(\epsilon_a + \epsilon_b - \epsilon_i - \epsilon_j) (\delta_{ac} \delta_{bd} - \delta_{ad} \delta_{bc})] \quad (2.34)$$

using the definition of the MP2 amplitudes, we can express the "zero-order" term as:

$$\langle \Psi_{ij}^{(1)} | H_0 - E^{(0)} | \Psi_{ij}^{(1)} \rangle = - \sum_{a>b, c>d} t_{ij}^{cd} \langle ij || ab \rangle (\delta_{ac} \delta_{bd} - \delta_{ad} \delta_{bc}) \quad (2.35)$$

$$= - \sum_{a>b} t_{ij}^{ab} \langle ij || ab \rangle \quad (2.36)$$

as only the first deltas term contributes, by the summation convention. Recognize the resulting expression as the “perturbative coupling” term, therefore the J_2 functional can be expressed as:

$$J_2 = \sum_{i>j} \sum_{a>b} t_{ij}^{ab} \langle ij||ab \rangle = \sum_{i>j} \sum_{a>b} \langle ij||ab \rangle \langle ij||ab \rangle / (\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b) \quad (2.37)$$

Using the chain rule to evaluate the gradient, regarding the two-electron integrals and the Fock matrix elements, we get the intermediate expression:

$$\frac{\partial J_2}{\partial R_{ae}} = \sum_{i>j} \sum_{a>b} 2 \langle ij||ab \rangle / (\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b) \frac{\partial \langle ij||ab \rangle}{\partial R_{ae}} \quad (2.38)$$

$$- \sum_{i>j} \sum_{a>b} (\langle ij||ab \rangle / (\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b))^2 \frac{\partial}{\partial R_{ae}} (\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b) \quad (2.39)$$

here each term is evaluated to:

$$\frac{\partial \langle ij||ab \rangle}{\partial R_{ae}} = \langle ij||eb \rangle \quad \text{and} \quad \frac{\partial}{\partial R_{ae}} (\epsilon_i + \epsilon_j - \epsilon_a - \epsilon_b) = -f_{ea} \quad (2.40)$$

arriving at the final expression for the gradient:

$$G_{ae} = \frac{\partial J_2}{\partial R_{ae}} = 2 \sum_{i>j} \sum_b t_{ij}^{ab} \langle ij||eb \rangle + 2 \sum_b D_{ab} f_{eb} \quad (2.41)$$

where we have substituted the definition of the density matrix D_{ab} and renamed the indices for clarity. The first term captures the direct contribution of the active-inactive mixing to J_2 , while the second term captures the contribution via the density matrix, which accounts for how optimised orbitals are correlated with each other in the MP1 wavefunction. At convergence, the gradient must vanish, $G_{ae} = 0$ for all a, e , a stationarity condition that defines the optimal orbitals. The gradient is a matrix of dimension $(N_{\text{virt}} - N') \times N'$.

The Stationarity Principle

Optimal orbitals are referred by the stationarity condition that the gradient goes to zero: $\nabla_R E(R) \rightarrow 0$. This condition implies that the energy is at a local extremum with respect to orbital rotations. The stationary point can correspond to a local minimum, maximum, or saddle point in the energy landscape, by examining the second derivative (Hessian) at the stationary point, we can determine the nature of the extremum. For this purpose we derive the Hessian matrix in the following.

2.3.5 Hessian Expression

Taking the second derivative of the Hylleraas functional with respect to the orbital rotation parameters R_{ae} and R_{fb} , we arrive at the Hessian matrix:

$$H_{ae,bf} = \left. \frac{\partial^2 J_2}{\partial R_{ae} \partial R_{bf}} \right|_{R=0} \quad (2.42)$$

If the gradient captures the slope of the energy landscape, the Hessian captures the curvature, providing information about how the energy changes as we move in different directions in orbital space.

Adamowicz and Bartlett derive the full expression for the Hessian, which consists of four terms:

$$H_{ae,bf} = 2 \sum_{i>j} t_{ij}^{ab} \langle ij || ef \rangle - \sum_{i>j} \sum_c [t_{ij}^{ac} \langle ij || bc \rangle + t_{ij}^{bc} \langle ij || ca \rangle] \delta_{ef} \quad (2.43)$$

$$+ D_{ab}(f_{aa} - f_{bb})\delta_{ef} + D_{ab}f_{ef}(1 - \delta_{ef}) \quad (2.44)$$

they note the active orbitals are canonical, and that the Hessian also shows pair separability [8].

Derivation of the Hessian Expression

We start from the MP2 gradient written with consistent dummy indices:

$$G_{ae} = 2 \sum_{i>j} \sum_c t_{ij}^{ac} \langle ij || ec \rangle + 2 \sum_c D_{ac} f_{ec} \quad (2.45)$$

where

$$D_{ac} = \sum_{i>j} \sum_d t_{ij}^{ad} t_{ij}^{cd}, \quad f_{ec} = \epsilon_e \delta_{ec} \quad (2.46)$$

The Hessian is defined as:

$$H_{ea,fb} = \frac{\partial G_{ae}}{\partial R_{bf}} \quad (2.47)$$

We use the following standard orbital-rotation derivatives.

Two-electron integrals:

$$\frac{\partial \langle ij || ec \rangle}{\partial R_{bf}} = \delta_{cb} \langle ij || ef \rangle + \delta_{eb} \langle ij || fc \rangle - \delta_{cf} \langle ij || eb \rangle - \delta_{ef} \langle ij || bc \rangle \quad (2.48)$$

Fock matrix (canonical orbitals):

$$\frac{\partial f_{ec}}{\partial R_{bf}} = \delta_{cb} f_{ef} + \delta_{eb} f_{fc} - \delta_{cf} f_{eb} - \delta_{ef} f_{bc} \quad (2.49)$$

Amplitudes: Rather than expanding the full quotient rule, we use:

$$t_{ij}^{ac} = \frac{\langle ij || ac \rangle}{\Delta_{ijac}} \quad (2.50)$$

and differentiate formally, allowing cancellation later:

$$\frac{\partial t_{ij}^{ac}}{\partial R_{bf}} = \frac{1}{\Delta_{ijac}} \frac{\partial \langle ij || ac \rangle}{\partial R_{bf}} - t_{ij}^{ac} \frac{1}{\Delta_{ijac}} \frac{\partial \Delta_{ijac}}{\partial R_{bf}} \quad (2.51)$$

Step 1: Differentiate First Gradient Term

Define:

$$A = \frac{\partial}{\partial R_{bf}} \left(2 \sum_{i>j} \sum_c t_{ij}^{ac} \langle ij || ec \rangle \right) \quad (2.52)$$

Apply product rule:

$$A = 2 \sum_{i>j} \sum_c \left(\frac{\partial t_{ij}^{ac}}{\partial R_{bf}} \langle ij || ec \rangle + t_{ij}^{ac} \frac{\partial \langle ij || ec \rangle}{\partial R_{bf}} \right) \quad (2.53)$$

Integral derivative contribution

Insert the integral derivative:

$$\sum_c t_{ij}^{ac} \frac{\partial \langle ij || ec \rangle}{\partial R_{bf}} = \sum_c t_{ij}^{ac} (\delta_{cb} \langle ij || ef \rangle + \delta_{eb} \langle ij || fc \rangle - \delta_{cf} \langle ij || eb \rangle - \delta_{ef} \langle ij || bc \rangle) \quad (2.54)$$

Collapse sums:

$$= t_{ij}^{ab} \langle ij || ef \rangle + \delta_{eb} \sum_c t_{ij}^{ac} \langle ij || fc \rangle - t_{ij}^{af} \langle ij || eb \rangle - \delta_{ef} \sum_c t_{ij}^{ac} \langle ij || bc \rangle \quad (2.55)$$

Thus:

$$A_{\text{int}} = 2 \sum_{i>j} \left[t_{ij}^{ab} \langle ij || ef \rangle - t_{ij}^{af} \langle ij || eb \rangle \right] + 2\delta_{eb} \sum_{i>j} \sum_c t_{ij}^{ac} \langle ij || fc \rangle - 2\delta_{ef} \sum_{i>j} \sum_c t_{ij}^{ac} \langle ij || bc \rangle \quad (2.56)$$

Amplitude derivative contribution

The amplitude derivative produces terms of the same structure but with permuted indices. After relabeling dummy indices and combining symmetric contributions, these terms cancel the δ_{eb} terms and generate the symmetric partner of the δ_{ef} term:

$$-2\delta_{ef} \sum_{i>j} \sum_c t_{ij}^{bc} \langle ij || ca \rangle \quad (2.57)$$

Final result for A

$$A = 2 \sum_{i>j} t_{ij}^{ab} \langle ij || ef \rangle - \delta_{ef} \sum_{i>j} \sum_c \left[t_{ij}^{ac} \langle ij || bc \rangle + t_{ij}^{bc} \langle ij || ca \rangle \right] \quad (2.58)$$

Step 2: Differentiate Second Gradient Term

Define:

$$B = \frac{\partial}{\partial R_{bf}} \left(2 \sum_c D_{ac} f_{ec} \right) \quad (2.59)$$

$$B = 2 \sum_c \left(\frac{\partial D_{ac}}{\partial R_{bf}} f_{ec} + D_{ac} \frac{\partial f_{ec}}{\partial R_{bf}} \right) \quad (2.60)$$

Fock derivative term

$$\sum_c D_{ac} \frac{\partial f_{ec}}{\partial R_{bf}} = \sum_c D_{ac} (\delta_{cb} f_{ef} - \delta_{ef} f_{bc}) \quad (2.61)$$

$$= D_{ab} f_{ef} - \delta_{ef} \sum_c D_{ac} f_{bc} \quad (2.62)$$

Density derivative term

After inserting amplitude derivatives and simplifying (using symmetry and relabelling), this yields:

$$\delta_{ef} D_{ab} (f_{aa} - f_{bb}) \quad (2.63)$$

Final result for B

$$B = D_{ab} (f_{aa} - f_{bb}) \delta_{ef} + D_{ab} f_{ef} (1 - \delta_{ef}) \quad (2.64)$$

Final Hessian

$$H_{ea,fb} = 2 \sum_{i>j} t_{ij}^{ab} \langle ij || ef \rangle \quad (2.65)$$

$$- \delta_{ef} \sum_{i>j} \sum_c \left[t_{ij}^{ac} \langle ij || bc \rangle + t_{ij}^{bc} \langle ij || ca \rangle \right] \quad (2.66)$$

$$+ D_{ab} (f_{aa} - f_{bb}) \delta_{ef} + D_{ab} f_{ef} (1 - \delta_{ef}) \quad (2.67)$$

Block Structure of the Hessian

We see a clear simplification of the Hessian if $\delta_{ef} = 0$:

$$H_{ae,bf} = 2 \sum_{i>j} t_{ij}^{ab} \langle ij || ef \rangle + D_{ab} f_{ef} \quad (2.68)$$

such that two orbitals a, b would not rotate to the same orbitals, on the other hand if $\delta_{ef} = 1$:

$$H_{ae,be} = 2 \sum_{i>j} t_{ij}^{ab} \langle ij || ee \rangle - \sum_{i>j} \sum_c \left[t_{ij}^{ac} \langle ij || bc \rangle + t_{ij}^{bc} \langle ij || ca \rangle \right] + D_{ab} (f_{aa} - f_{bb}) \quad (2.69)$$

such that two orbitals a, b would rotate to the same orbital. This translates to a block structure within the Hessian, where each block corresponds to a specific inactive orbital e . Each block is a square matrix of size $N' \times N'$, where N' is the number of optimised orbitals. The off-diagonal blocks (where $e \neq f$) contain terms that couple different inactive orbitals, but these are typically smaller in magnitude than the diagonal blocks. This block structure approximation allows for efficient inversion of the Hessian, as each block can be inverted separately, reducing the computational cost from inverting a large matrix to inverting several smaller matrices.

Negative Eigenvalues and Level-Shift Stabilisation

As the Hessian expresses the curvature of the energy landscape, it may become indefinite (i.e., have negative eigenvalues) near saddle points or in cases with small virtual orbital energy gaps. This can lead to divergence or oscillations in the Newton-Raphson optimisation, see next section. To address this, a level-shift strategy can be employed, if any diagonal block of the Hessian has negative eigenvalues, a positive shift λ is added to the diagonal elements, effectively replacing $\mathbf{H}$ with $\mathbf{H} + \lambda\mathbf{I}$. The choice of λ can be empirical or based on a dynamic threshold related to the smallest eigenvalue. This ensures that the modified Hessian is positive-definite, allowing for stable convergence towards a local minimum. Adamowicz & Bartlett does not discuss this strategy. It is observed that the block approximation does get rid of most negative eigenvalues, but some may still remain, which may be in cases with small virtual gaps $\epsilon_a - \epsilon_i$ from basis sets with diffuse functions. The implementation of OVOS omits the level-shift strategy, but it is a potential avenue for future improvement to ensure robust convergence.

2.3.6 Newton-Raphson

Building off the gradient and Hessian of J_2 , an orbital rotation update $\Delta\mathbf{R}$ can be computed using the Newton-Raphson (NR) method, which iteratively refines the orbital coefficients to minimise J_2 . The NR update is given by:

$$\mathbf{R} = -\mathbf{G} \cdot \mathbf{H}^{-1} \quad (2.70)$$

which is equivalent to:

$$\mathbf{H}\mathbf{R} = -\mathbf{G} \quad (2.71)$$

where $\mathbf{G}$ is the gradient vector and $\mathbf{H}$ is the Hessian matrix. The NR method runs into problems if the energy landscape is flat or has multiple minima, convergence can be slow or may require damping of the update step. The block-diagonal structure of the Hessian allows for efficient inversion, as each block can be treated independently, which is equivalent to solving a reduced linear equation in each inactive-orbital block. To ensure numerical stability, especially in cases where the Hessian may have negative eigenvalues, a level-shift strategy can be applied, as discussed in the previous section.

The convergence criterion for the NR iterations is typically defined as $|J_2(n) - J_2(n-1)| < 10^{-8}$, indicating that the change in the Hylleraas functional between iterations is sufficiently small to consider the solution converged. Supplementing this with the stationarity condition $\nabla_R E(R) < 10^{-8}$ ensures that the orbitals are at a local extremum of the energy landscape, which is the defining characteristic of optimal orbitals in this context.

2.3.7 Orbital Canonicalisation

After a unitary rotation, the virtual Fock block F_{ab}^{active} is generally non-diagonal, meaning the active virtual orbitals are no longer canonical [5]. To restore the canonical form, we compute the active Fock block in the rotated basis:

$$F_{ab} = \mathbf{C}'^\dagger F^{\text{AO}} \mathbf{C}' \quad (2.72)$$

where C' are the coefficients of the virtual orbitals after rotation. Diagonalising F_{ab} yields new canonical virtual orbitals with well-defined orbital energies ε_a from the eigenvalues. This step is crucial for the next iteration of MP2 amplitude calculations, as it ensures that the denominators in the amplitude expressions are based on canonical orbital energies, simplifying the computations and improving numerical stability. It is important to note that only the virtual orbitals are rotated and re-canonicalised at each OVOS iteration; the occupied orbitals remain unchanged throughout the procedure.

With canonical rotated orbitals, the MP2 energy for the new active space can be evaluated, and the energy recovery fraction can be defined to quantify the performance of the OVOS procedure.

2.3.8 Computational Cost and Scaling

A central motivation for OVOS is the potential for significant computational speedup by contracting the orbital space to a smaller active orbital set [8]. To quantify this, we analyse the computational cost of each step in the OVOS iteration and compare it to the cost of traditional full-space correlation methods.

Per-Iteration Cost of OVOS Optimisation

Defining the cost of an action requires introducing the Big O notation, which describes the asymptotic scaling of an algorithm with respect to the size of the input. In the context of OVOS, the relevant input sizes are the number of occupied orbitals (N_{occ}), the number of inactive virtual orbitals (N_{inact}), the number of optimised orbitals (N'), and the number of atomic orbitals (n).

The computational cost of one OVOS iteration can be broken down by the steps involved, see Section 4.1 for an algorithm summary, where the cost are:

- Integral transformation: $O(n^4 + N_{occ}^2 N_{virt}^2)$ for transforming two-electron integrals to the MO basis.
- MP2 amplitude evaluation: $O(N_{occ}^2 N'^2)$ for computing the amplitudes in the active space.
- MP2 energy: $O(N_{occ}^2 N'^4)$ for evaluating the MP2 energy with the active space amplitudes.
- Density matrix D_{ab} : $O(N_{occ}^2 N'^3)$ for computing the virtual-virtual density matrix.
- Gradient G_{ea} : $O(N_{occ}^2 N' N_{inact})$ for computing the gradient with respect to orbital rotations.
- Hessian $H_{ea,fb}$: $O(N_{occ}^2 N_{inact}^2 N'^2)$ for computing the Hessian in block-diagonal form.
- Block diagonal: $O(N_{inact} N'^2)$ for extracting the block-diagonal part of the Hessian.
- Newton-Raphson step: $O(N_{inact} N'^3)$ inverting the block-diagonal Hessian.

- Orbital rotation: $O((N_{occ} + N_{inact} + N')^3)$ for computing the unitary rotation matrix via matrix exponential.
- Canonicalisation: $O(N'^3)$ for diagonalising the active Fock block to obtain canonical orbitals.

Overall, the per-iteration cost is dominated differently for a few cases, summarised in the following table: This reflects the trade-off in OVOS, a smaller active spaces

Active Space	$r = N'/N_{virt}$	Dominant	Cost Scaling	Relative Cost
Small	~ 0.1	Orbital Rotation	$O(N_{total}^3)$	Fastest, but minimal speedup
Medium	$\sim 0.3-0.5$	Newton Solver*	$O((N_{inact}N')^3)$	Optimal window
Large	$\sim 0.7-0.9$	MP2 Energy	$O(N_{occ}^2 N'^4)$	Diminishing returns

Table 2.3: Dominant computational steps in OVOS iterations as a function of the active space size N' . *MP2 close 2nd.

reduce the cost of MP2 energy and gradient evaluations but increase the relative cost of orbital rotations and Hessian inversions (number of blocks). Larger active spaces reduce the cost of orbital rotations but increase the cost of MP2 energy evaluations. The optimal window for OVOS is typically in the medium range, where the cost of the Newton step is manageable and the speedup from reduced MP2 evaluations is significant.

Taken the size of the basis set into account, translating to the number of virtual orbitals, the per-iteration cost of OVOS is dominated by the MP2 energy evaluation step, which scales as $O(N_{occ}^2 N'^4)$. However, since $N' < N_{virt}$, this represents a significant reduction compared to the full-space MP2 cost as the basis set size increases, beating out the cost of orbital rotations and Hessian inversions.

Downstream Speedup Factors

In turn, how well would the reduced active space from OVOS speed up downstream methods like VQE. VQE scaling is dominated by the number of virtual orbitals, as the number of qubits and circuit depth scale with the size of the active space. The cost of VQE with a UtUPS ansatz scales as $O(N_{qubit})$, so the speedup from OVOS is the percentage of lower virtual space size, $O(N_{virt}/N')$. An advantage of OVOS might not be the recovered MP2 energy for a system, but the corresponding reduced virtual orbitals, as a trial state.

Breakeven Analysis and Practical Realisation

Although a speedup appears beneficial, it comes at the cost of reduced correlation energy recovery. The breakeven point for OVOS depends on the desired accuracy in the correlation energy and the size of the system. For small systems with few virtual orbitals, the overhead of OVOS iterations may not be justified, as the full-space MP2 calculation is already manageable. However, for larger systems with many virtual orbitals, OVOS can provide significant speedup while still recovering a substantial fraction of the correlation energy, as will be discussed. The practical breakeven point will differ from performance benchmarks, as it depends on the specific system, basis set, and computational resources available.

2.4 Relevance to Quantum Computing

This section connects the OVOS method to its relevance for quantum computing applications, particularly in the context of variational quantum algorithms like VQE. For VQE, the immediate gain is qubit reduction by contracting the virtual orbital space to a smaller active set, while still capturing a substantial fraction of the correlation energy. A practical necessity on today's noisy near-term quantum devices with limited qubit counts and coherence times. In this context we introduce the Brueckner orbitals, which are a special case of orbital optimisation and are expected to be closely approximated by the OVOS optimised orbitals within the truncated active space. Understanding the connection between OVOS and Brueckner orbitals provides insight into why OVOS can be an effective strategy for generating high-quality trial states for VQE, as Brueckner orbitals are known to maximise the overlap with the exact ground state wavefunction. This section lays the theoretical groundwork for the subsequent analysis of OVOS performance in quantum computing applications, as will be explored in Chapter 5.

2.4.1 Brueckner Orbitals and Connection to OVOS

A special case of orbital optimisation in post-HF methods is the concept of Brueckner orbitals, which are defined as the orbitals that diagonalise the single-particle Green's function [20].

The single-particle Green's function is an advanced quantum mechanical object that encodes off-diagonal elements of the density matrix corrections from correlation [5]. Diagonalizing this function means the correlation-induced density matrix has minimal off-diagonal elements or, equivalently, the orbitals for which the T1 coupled-cluster amplitudes vanish for all occupied-virtual pairs, $t_{ia}^1 = 0$. Brueckner orbitals maximise the overlap of the Hartree-Fock determinant with the exact ground state wavefunction (FCI), $S = |\langle \Phi_{\text{HF}} | \Phi_{\text{exact}} \rangle|^2$, making Brueckner orbitals an expected optimal choice for reference orbitals in correlated calculations [19]. In practice, Brueckner orbitals are computed using the Brueckner-CCD (B-CCD) method [19]. This method iteratively solves the coupled-cluster doubles equations while simultaneously rotating the orbitals to eliminate the T1 amplitudes. The resulting Brueckner orbitals are often found to provide a better starting point for correlation treatments than the canonical HF orbitals, especially in cases where the HF reference is a poor approximation to the true ground state [19, 21]. The presence of non-zero T1 amplitudes in the HF reference indicates that the density matrix obtained from correlation differs from the HF density, leading to computational effort being wasted on single excitations [13].

Connection to Orbital Optimised MP2 methods

Brueckner orbitals are closely related to the concept of orbital optimised MP2 (OO-MP2) [13], which variationally minimises the MP2 energy with respect to all orbital rotations. At convergence, the gradient of the MP2 energy with respect to orbital rotations vanishes, which is the same stationarity condition that defines Brueckner orbitals at the MP2 level [13]. Therefore, OO-MP2 orbitals can be considered as the MP2-level Brueckner orbitals.

We can also draw a connection to the OVOS method, which minimises the MP2

energy in a restricted active virtual space [8]. Conceptually, this is the same orbital stationarity condition as for Brueckner orbitals, but applied only to the active block of virtual orbitals. The OVOS optimised molecular orbitals are hence expected to approximate Brueckner orbitals within the truncated active virtual space.

More precisely, the OVOS procedure enforces the stationary point condition $\nabla_R E(R) = 0$ for all active-inactive orbital rotations R_{ae} (where $a \in$ active virtuals, $e \in$ inactive virtuals). This stationarity condition is mathematically identical to enforcing the Brueckner condition $t_i^a = 0$ within the truncated active virtual subspace. In other words, OVOS optimised orbitals ARE the Brueckner orbitals of the truncated orbital space. This insight has direct implications for higher-order correlation method, just as Brueckner-CCD orbitals are known to dramatically reduce T1 and T2 amplitudes compared to HF, we expect OVOS orbitals to yield smaller T1 amplitudes [19, 21].

OVOS Orbitals as Approximate Brueckner Orbitals

OVOS orbitals satisfy the stationarity condition with respect to orbital rotations between the active and inactive virtual spaces. Specifically, at convergence of the OVOS procedure, the gradient of the Hylleraas functional with respect to these rotations vanishes:

$$\frac{\partial J_2}{\partial R_{ae}} = G_{ae} = 0 \quad \forall a, e \quad (2.73)$$

This condition implies that the MP2 energy is stationary with respect to orbital rotations that mix the active virtual orbitals (those included in the OVOS optimisation) and the inactive virtual orbitals (those excluded from the active space). In the context of orbital optimised MP2 (OO-MP2), stationarity of the energy with respect to all orbital rotations (including occupied-virtual) leads to the condition that the single-excitation amplitudes T_1 vanish, which defines Brueckner orbitals.

We express the single-excitation amplitudes T_1 as the coupling between the reference determinant and singly excited determinants:

$$T_1 = \sum_{i,a} t_i^a |\Phi_i^a\rangle \quad (2.74)$$

where t_i^a are the single-excitation amplitudes, $|\Phi_i^a\rangle$ are the singly excited determinants obtained by replacing an occupied orbital i with a virtual orbital a .

While OVOS does not enforce stationarity with respect to occupied-virtual rotations, it achieves stationarity within the truncated active virtual space, effectively making the OVOS orbitals the Brueckner orbitals of that active space. This connection has important implications for the quality of the reference state and the expected reduction of T_1 amplitudes in subsequent correlation treatments.

2.4.2 Quantum Computing and Orbital Space

Quantum computing is an emerging paradigm that has the potential to revolutionise computational chemistry by enabling the simulation of quantum systems that are inconceivable for classical computers. Bridging from classical electronic structure methods to quantum algorithms requires careful consideration of how molecular orbitals are represented and utilised in the quantum computational framework.

Second Quantisation and Qubit Mapping

In the context of quantum chemistry, the electronic structure problem is often formulated in second quantisation, where the Hamiltonian is expressed in terms of creation and annihilation operators acting on a reference state [5]. To implement quantum algorithms on a quantum computer, the fermionic operators must be mapped to qubit operators using transformations such as the Jordan-Wigner or Bravyi-Kitaev mappings [12]. The second quantised electronic Hamiltonian can be written as [7]:

$$\hat{H} = \sum_{pq} h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} g_{pqrs} a_p^\dagger a_q^\dagger a_r a_s, \quad a_p^\dagger a_q \rightarrow \mathcal{O}_{pq}, \quad a_p^\dagger a_q^\dagger a_r a_s \rightarrow \mathcal{O}_{pqrs}$$

This mapping allows us to represent the electronic Hamiltonian as a sum of Pauli operators, which can then be implemented as quantum circuits [7]. The choice of molecular orbitals used in the second quantised Hamiltonian can significantly impact the efficiency of the quantum algorithm, as it affects the structure of the Hamiltonian and the overlap of the reference state with the true ground state. In particular, using OVOS optimised orbitals as the basis for the second quantised Hamiltonian might lead to a more compact representation and improved convergence properties in quantum algorithms such as the VQE [7, 8].

The Variational Quantum Eigensolver

The VQE [6] is a hybrid quantum-classical algorithm designed to find the ground-state energy of a quantum system [7]. VQE uses a parameterised quantum circuit, known as an ansatz, to prepare trial wavefunctions on a quantum computer. A classical outer loop optimises the parameters of the ansatz to minimise the energy expectation value, which is measured on the quantum device [6, 7].

The choice of reference state in VQE is crucial for the efficiency and accuracy of the algorithm, and OVOS orbitals offer a promising avenue for improving the performance of VQE by providing a better initial guess for the ground state wavefunction [7]. Leaving the choice of ansatz, while Unitary Coupled Cluster Singles and Doubles (UCCSD) is a broadly used chemically-motivated ansatz in VQE [6, 7], the Unrestricted tiled Unitary Product State (UtUPS) ansatz is chosen for its simplified structure and the impact of the reference state. The UtUPS ansatz is constructed as $|\Psi(\theta)\rangle = e^{T(\theta)} e^{-T^\dagger(\theta)} |\Phi_{\text{ref}}\rangle$, where $|\Phi_{\text{ref}}\rangle$ is a reference state, typically the Hartree-Fock determinant, and the Θ 's are parameters, tuning the ansatz [6]. The quality of the reference state significantly impacts the performance of VQE, as a reference with better overlap with the true ground state can lead to faster convergence and more compact orbital spaces [7].

OVOS Molecular Orbitals as a Trial State for VQE

We had VQE as a potential downstream application of OVOS in Section 2.3.8, as the reduced active virtual space directly translates to fewer qubits and shallower circuits. VQE shifts us from classical quantum chemistry to quantum computing, where the quality of the trial state is advantages for convergence and accuracy. This section argues why OVOS trial states are beneficial for variational quantum algorithms on NISQ devices.

Reference-State Overlap

To quantify the quality of the OVOS orbitals as a trial state for VQE, we can consider the overlap (or fidelity) between the reference state constructed from the OVOS orbitals and the Full Configuration Interaction (FCI) ground state. The fidelity F is defined as:

$$F = |\langle \Phi_{ref} | \Psi_{FCI} \rangle|^2 \quad (2.75)$$

where Φ_{ref} is the reference state built from the OVOS orbitals and Ψ_{FCI} is the FCI wavefunction expressed in the natural orbital basis.

The claim is that $F_{OVOS} \geq F_{HF}$ because OVOS orbitals are optimised to minimise the MP2 energy, which should yield a reference state that has a higher overlap with the correlated wavefunction compared to the HF orbitals. This improved overlap can lead to faster convergence in VQE, as the initial state is closer to the true ground state, and can also reduce the required circuit depth to achieve a given accuracy.

Trial states

To evaluate how well OVOS orbitals will do, we formulate the following competitors, as the UMP2 natural orbitals, and canonical UHF orbitals. The former are the natural orbitals obtained from a UMP2 calculation, which are known to provide a good approximation to the correlated wavefunction and often yield improved convergence in VQE, formulated in PySCF [11]. We arrive at the natural orbitals by diagonalizing the one-particle density matrix constructed from the UMP2 amplitudes:

$$D_{pq} = \sum_{ijab} t_{ij}^{ab} \langle pq | ij \rangle \quad (2.76)$$

where t_{ij}^{ab} are the UMP2 amplitudes and $\langle pq | ij \rangle$ are the two-electron integrals. Diagonalizing D_{pq} yields the natural orbitals:

$$SD_{pq}SC = SCn \quad (2.77)$$

where S is the overlap matrix, C are the coefficients of the natural orbitals, and n are the natural orbital occupation numbers. The natural orbitals are expected to have a higher overlap with the FCI ground state compared to the canonical UHF orbitals, which are obtained from a standard Hartree-Fock calculation and may not capture correlation effects as effectively.

Reduction of T_1 Amplitudes and Circuit Depth

Within VQE the reduction of T_1 amplitudes in the OVOS orbitals implies that the single-excitation contributions to the wavefunction are minimised. This has significant implications for the choice of ansatz in VQE, as it allows for the use of simple ansatz dropping single excitations (e.g., UCCD instead of UCCSD), which can significantly reduce the circuit depth and complexity. The reduction in T_1 amplitudes might reflect a good approximation to the reference state, which can lead to faster convergence and improved accuracy in VQE calculations.

Chapter 3

OVOS Implementation

In this chapter we build on the theoretical foundation laid out in Chapter 2 to describe the practical implementation of the OVOS algorithm. The implementation was written up for this project in Python. The goal is to provide a clear and detailed overview of the algorithmic steps, the computational strategies employed, and the choices made in the implementation, for the full implementation details see Appendix C. This chapter serves as a bridge between the abstract theoretical concepts and the concrete computational details, enabling readers to understand how the OVOS method is realised and how it can be reproduced or extended in future work. It is to be made clear here that the OVOS algorithm was coded up from scratch by myself, following the original proposal by Adamowicz & Bartlett, but with some modifications and optimisations as described in the sections below.

3.1 Algorithm

Here we outline the full OVOS algorithm as implemented in this work, following the general structure proposed by Adamowicz & Bartlett [8]. The algorithm consists of an initialisation phase followed by an iterative loop that optimises the active virtual orbitals to maximise the MP2 correlation energy. The key steps are summarised as pseudocode in Algorithm 1, which serves as a roadmap for the detailed descriptions in the subsequent sections.

Algorithm 1 OVOS Iterative Optimisation

- 1: Perform SCF calculation to obtain initial MO coefficients $\mathbf{C}$
 - 2: Transform AO integrals to MO basis: $\langle ij|ab \rangle$
 - 3: Select initial active virtual space
 - 4: **repeat**
 - 5: Compute MP1 amplitudes t_{ij}^{ab} and MP2 energy J_2
 - 6: Assemble gradient $\mathbf{G}$ and Hessian $\mathbf{H}$
 - 7: Solve Newton-Raphson equation: $\Delta \mathbf{R} = -\mathbf{H}^{-1} \mathbf{G}$
 - 8: Generate unitary matrix $\mathbf{U} = \exp(\mathbf{R})$
 - 9: Update MO coefficients: $\mathbf{C} \leftarrow \mathbf{C} \mathbf{U}$
 - 10: Canonicalize active Fock block
 - 11: Evaluate new J_2
 - 12: **until** $|J_2^{(n)} - J_2^{(n-1)}| < \epsilon_{\text{conv}}$
-

Below we provide a detailed description of each step in the algorithm. For an illustrative figure of the loop structure in OVOS see Figure B.12 in Appendix B.

3.1.1 Integral Transformation

Transforming the two-electron integrals from the atomic orbital (AO) basis to the molecular orbital (MO) basis is a critical step in the OVOS algorithm. This transformation is necessary to compute the MP1 amplitudes and the gradient of the MP2 energy with respect to orbital rotations. However, performing a full 4-index transformation of all integrals would be computationally prohibitive. To mitigate this, we only transform the subset of integrals that are required for the MP2 amplitude equations, specifically those of the form $\langle ij|ab \rangle$ where i, j are occupied orbitals and a, b are active virtual orbitals, as these are the only ones that contribute directly to the correlation energy and its gradient. The transformation is performed using the `ao2mo.kernel()` function from the PySCF library, which efficiently computes the required blocks of integrals. After obtaining the spatial integral blocks for $\alpha\alpha$, $\beta\beta$, and $\alpha\beta$ spin combinations, we construct the full spin-orbital ERI tensor by interleaving the α and β contributions according to the index mapping. Finally, we convert the chemists' notation integrals output by PySCF to the physicists' antisymmetrised form used in our formulas, ensuring consistency between the code and the theoretical expressions. These steps are detailed in Code 3.1.

Code 3.1: AO-to-MO integral transformation with spin-orbital antisymmetrisation

```

1  def _eri_vovo_antisym(self, mo_coeffs: List[np.ndarray]) -> np.
    ndarray:
2      """Transform AO integrals -> spin-orbital."""
3      nocc_a, nocc_b = self.mol.nelec
4      C_occ_a, C_occ_b = mo_coeffs[0][:, :nocc_a], mo_coeffs
        [1][:, :nocc_b]
5      C_vir_a, C_vir_b = mo_coeffs[0][:, nocc_a:], mo_coeffs
        [1][:, nocc_b:]
6
7      def _ao2mo(C_left, C_row, C_right, C_col):
8          # Returns (left,row/right,col) in chemist notation
9          return ao2mo.kernel(self.eri_4fold_ao,
10                             [C_left, C_row, C_right, C_col],
11                             compact=False)
12 # Compute spatial blocks: (vir,occ/vir,occ)
13     vovo_aaaa = _ao2mo(C_vir_a, C_occ_a, C_vir_a, C_occ_a)
14     vovo_bbbb = _ao2mo(C_vir_b, C_occ_b, C_vir_b, C_occ_b)
15     vovo_aabb = _ao2mo(C_vir_a, C_occ_a, C_vir_b, C_occ_b)
16 # Assemble spin-orbital chemist tensor (interleaved alpha/beta)
17     nvir_spin, nocc_spin = 2 * nvir_a, nocc_a + nocc_b
18     vovo_spin = np.zeros((nvir_spin, nocc_spin, nvir_spin,
        nocc_spin))
19     vovo_spin[0::2, 0::2, 0::2, 0::2] = vovo_aaaa
20     vovo_spin[1::2, 1::2, 1::2, 1::2] = vovo_bbbb
21     vovo_spin[0::2, 0::2, 1::2, 1::2] = vovo_aabb
22     vovo_spin[1::2, 1::2, 0::2, 0::2] = vovo_aabb.T
23 # Convert to physicist notation and antisymmetrise:
24     eri_phys = vovo_spin.transpose(0, 2, 1, 3)
25     return eri_phys - eri_phys.transpose(0, 1, 3, 2)

```


With the integrals transformed to the MO basis, the next step is to select the initial active virtual orbitals, which is described in the following section.

3.1.2 Initial Active Space Selection

Before the iterative optimisation process can begin, an initial set of active virtual orbitals must be selected. In the original algorithm proposed by Adamowicz & Bartlett, the selection is based on ranking the virtual orbitals according to their estimated contribution to the correlation energy, which involves evaluating both the diagonal and off-diagonal contributions from the MP2 amplitude equations [8]. In the current implementation, we adopt a simpler approach where the active virtual orbitals are selected sequentially based on their spin-orbital indices. Specifically, after the occupied orbitals are assigned indices from 0 to $n_{\text{occ}} - 1$, the next N'_{virt} spin-orbitals are designated as active virtuals, and the remaining ones are considered inactive virtuals. This approach is straightforward and avoids the computational overhead at the initial stage.

Once the active space is defined, the next step is to compute the MP1 amplitudes and the MP2 correlation energy. This process is described in detail in the following section.

3.1.3 MP1 Amplitudes and MP2 Energy

At each iteration, the first-order MP (MP1) amplitudes (T2 excitations, "D") are re-computed in the current MO basis and used to evaluate the second-order Hylleraas functional J_2 . In the code, see Code 3.2 and 3.3, the amplitudes are computed using the formulas derived in Chapter 2, with explicit spin-orbital index labelling that matches the implementation. The amplitudes are stored in a tensor of shape $(n_{\text{occ}}, n_{\text{occ}}, N'_{\text{virt}}, N'_{\text{virt}})$, where N'_{virt} is the number of active virtual orbitals.

Code 3.2: MP1 amplitude computation

```
1 def _mp1_amplitudes(self, fock_diag: np.ndarray,
2                       eri_as: np.ndarray) -> np.ndarray:
3     ...
4     return -eri_as[:nvir_act, :nvir_act, :, :] / denom
```

The MP2 correlation energy is then evaluated using these amplitudes and the antisymmetrised integrals, see Appendix C for the full implementation.

Code 3.3: MP2 correlation energy

```
1 def _mp2_energy(self, fock_spin: np.ndarray, t_abij: np.ndarray
2                 ,
3                 eri_as: np.ndarray) -> float:
4     """MP2 correlation energy J_2"""
5     ...
6     energy = 0.0
7     for loc_i, loc_j in [(i, j) for i in range(nocc) for j in
8                           range(i)]:
9         abs_i = occ_abs[loc_i]
10        abs_j = occ_abs[loc_j]
```



```

9         eps_ij_sum = fock_spin[abs_i, abs_i] + fock_spin[abs_j,
10                abs_j]
11
12         t_ab = t_abij[a_loc, b_loc, loc_i, loc_j]
13         g_ab = eri_as[a_loc, b_loc, loc_i, loc_j]
14
15         # Diagonal: 2 * t * g
16         term_diag = 2.0 * np.dot(t_ab, g_ab)
17
18         # Off-diagonal: carefully handled with Fock matrix
19         elements
20         term_offdiag = ...
21
22         energy += term_diag + term_offdiag
23     return energy

```

Alongside the MP2 energy, the gradient and Hessian matrices required for the Newton-Raphson step are assembled from the amplitudes computed here, as detailed in the following section.

3.1.4 Gradient and Hessian Assembly

The gradient vector G and Hessian matrix H of the MP2 energy functional J_2 are computed at each iteration using the stored MP1 amplitudes and the antisymmetrised integrals. The gradient G_{ea} , as detailed in Section 2.3.4, is implemented by looping over the inactive orbital index e and the active orbital index a , summing contributions from the amplitudes and integrals, see Code 3.4.

Code 3.4: Gradient computation

```

1  def _gradient(self, t_abij: np.ndarray, eri_as: np.ndarray,
2         D_ab: np.ndarray, fock_spin: np.ndarray) -> np.
3         ndarray:
4         """Gradient G[a,e] = 2*sum_{i>j,b} t_ab^ij <eb||ij> + 2*
5         sum_b D_ab F_be"""
6         ...
7         # Extract <eb||ij> for e in inactive, b in active, i>j
8         i_idx, j_idx = np.tril_indices(nocc, k=-1)
9         eri_eb = eri_as[nvir:, :nvir, :, :][:, :, i_idx, j_idx] #
10        (ninact, nvir, pairs)
11        t_ij = t_abij[:, :, i_idx, j_idx]
12
13        # Term 1: gradient from integrals
14        G = 2.0 * np.einsum('abp,ebp->ae', t_ij, eri_eb, optimise=
15        True)
16
17        # Term 2: density contribution
18        F_be = fock_spin[np.ix_(vir, inactive)]
19        G += 2.0 * D_ab @ F_be
20
21    return G

```

The Hessian $H_{ea,fb}$ is assembled with a block structure that allows for efficient inversion. Specifically, for each inactive orbital e , we construct a diagonal block $H^{(e)}$ of size $N'_{\text{virt}} \times N'_{\text{virt}}$ corresponding to the active virtual orbitals. This means that instead of inverting a large $(N'_{\text{virt}} \times n_{\text{inact}}) \times (N'_{\text{virt}} \times n_{\text{inact}})$ matrix, we only need to invert n_{inact} smaller blocks, which is computationally more tractable, see Code 3.5. Each index of the Hessian corresponds to a pair of active and inactive orbitals, and the contributions are computed according to the formulas derived in Chapter 2.

Code 3.5: Hessian computation with block structure

```

1 def _hessian(self, t_abij: np.ndarray, eri_as: np.ndarray,
2               D_ab: np.ndarray, fock_spin: np.ndarray) -> np.
3                   ndarray:
4     """Hessian H_{ae,bf} with separation by inactive orbital
5         index."""
6     ...
7     fock_vir_diag = np.diag(fock_spin)[vir]
8     fock_inact = fock_spin[np.ix_(inactive, inactive)]
9     i_idx, j_idx = np.tril_indices(nocc, k=-1)
10    t_ij = t_abij[:, :, i_idx, j_idx]
11    eri_ef = eri_as[nvir:, nvir:, i_idx, j_idx]
12    eri_ab = eri_as[:, nvir, :nvir, i_idx, j_idx]
13    # Term 1: 2 * t_ab ~p <ef||ij> ~p
14    term1 = 2.0 * np.einsum('abp,efp->aebf', t_ij, eri_ef,
15                            optimise=True)
16    H = term1.reshape(nvir * ninact, nvir * ninact)
17    # Terms 2-4: Fock matrix contributions (diagonal in e=f and
18    #           mixed)
19    term2 = -(np.einsum('acp,bcp->ab', t_ij, eri_ab, optimise=
20                      True) +
21             np.einsum('cbp,cap->ab', t_ij, eri_ab, optimise=
22                      True))
23    term3 = D_ab * (fock_vir_diag[:, None] + fock_vir_diag[None, :])
24    H += np.kron(term2 + term3, np.eye(ninact)).reshape(...)
25    # Term 4: off-diagonal Fock terms (e != f)
26    fock_inact_offdiag = fock_inact.copy()
27    np.fill_diagonal(fock_inact_offdiag, 0.0)
28    H += np.kron(D_ab, fock_inact_offdiag).reshape(...)
29    return H

```

With the gradient and Hessian assembled, we compute the Newton-Raphson step. Which is then used to generate the orbital rotation matrix. The details of this process are described in the following section.

3.1.5 Orbital Rotation and Update

The Newton-Raphson step is computed by solving the linear equation $\Delta R = -H^{-1}G$, where H is the Hessian matrix and G is the gradient vector. To the Hessian a block diagonal approximation RLE is applied `_apply_block_diagonal_rle(H)`, see Appendix C. Utilizing the `numpy.linalg.solve()` function, we need not invert the Hessian matrix

directly, as it is computationally more efficient to solve the linear system directly, see Code 3.6.

Code 3.6: Newton-Raphson step and orbital rotation

```

1 def _newton_step(self, G: np.ndarray, H: np.ndarray,
2                 iteration: int, start_counting: bool) -> np.
3                     ndarray:
4     """Solve  $H \cdot R = -G$  using block-diagonal RLE approximation.
5     """
6     g_vec = G.flatten()
7     dim = len(g_vec)
8     grad_norm = np.linalg.norm(g_vec)
9
10    # ===== APPLY BLOCK-DIAGONAL RLE APPROXIMATION =====
11    # Use block-diagonal approximation for better
12    # conditioning
13    H = self._apply_block_diagonal_rle(H)
14
15    # Pure Newton
16    R = np.linalg.solve(H, -g_vec)
17
18    return R

```

Once ΔR is obtained, we generate the unitary rotation matrix $U = \exp(\Delta R)$ using `scipy.linalg.expm()` for matrix exponentials. The rotation is applied to the MO coefficient matrices for both α and β spins, ensuring that only the virtual block is affected while the occupied orbitals remain unchanged, see Code 3.7.

Code 3.7: Orbital rotation via matrix exponential

```

1 def _rotate_orbitals(self, mo_coeffs: List[np.ndarray],
2                     fock_spin: np.ndarray,
3                     R_vec: np.ndarray) -> Tuple[List[np.ndarray],
4                     np.ndarray]:
5     """Apply orbital rotation  $U = \exp(R)$  where  $R$  is
6     antisymmetric."""
7
8     ...
9
10    # Reshape R_vec into local matrix R_local[a, e]
11    R_local = R_vec.reshape(nvir_act, ninact)
12
13    # Build full antisymmetric R in spin-orbital space
14    # (occupied block stays ZERO)
15    R_full = np.zeros((n_spin, n_spin), dtype=np.float64)
16    for a in range(nvir_act):
17        for e in range(ninact):
18            abs_a = n_occ + a
19            abs_e = n_occ + nvir_act + e
20            R_ae = R_local[a, e]
21            R_full[abs_e, abs_a] = R_ae
22            R_full[abs_a, abs_e] = -R_ae

```

```

21
22     # Verify antisymmetry:  $R = -R^T$ 
23     assert np.allclose(R_full, -R_full.T, atol=1e-12), \
24         f"R matrix not antisymmetric!"
25
26     # Generate unitary:  $U = \exp(R)$ 
27     U_spin = scipy.linalg.expm(R_full)
28
29     # Apply to MO coefficients (separate alpha/beta)
30     U_alpha = U_spin[0::2, 0::2]
31     U_beta = U_spin[1::2, 1::2]
32
33     mo_rot = [
34         mo_coeffs[0] @ U_alpha,
35         mo_coeffs[1] @ U_beta
36     ]
37
38     # Transform Fock matrix:  $F' = U^T F U$ 
39     fock_rot = U_spin.T @ fock_spin @ U_spin
40
41     return mo_rot, fock_rot, U_spin

```

After applying the orbital rotation, we re-canonicalize the active Fock block by diagonalising it to obtain new orbital energies ε_a for the active virtual orbitals. This step ensures that the orbitals remain orthogonal and that the Fock matrix is properly represented in the new basis, see Code ??.

Code 3.8: Re-canonicalization of active virtual block

```

1 def _canonicalize_active(self, mo_coeffs: List[np.ndarray],
2                           fock_spin: np.ndarray,
3                           U_applied) -> Tuple[List[np.ndarray],
4                                                np.ndarray]:
5     """Canonicalize ONLY the active virtual block (not inactive)"""
6     n_occ_a, n_occ_b = self.mol.nelec
7     nact_spatial = self.num_opt_virtual_orbs // 2
8
9     # Build canonicalization unitaries for alpha and beta
10    separately
11    U_alpha_full = np.eye(fock_spin.shape[0] // 2)
12    U_beta_full = np.eye(fock_spin.shape[0] // 2)
13
14    # Diagonalize ONLY active-active block (not inactive)
15    sl_a = slice(n_occ_a, n_occ_a + nact_spatial)
16    if nact_spatial > 0:
17        fock_alpha_spatial = fock_spin[0::2, 0::2]
18        F_alpha_active = fock_alpha_spatial[sl_a, sl_a]
19        evals_a, evects_a = np.linalg.eigh(F_alpha_active)
20        U_alpha_full[sl_a, sl_a] = evects_a
21
22    # Same for beta

```

```

21     sl_b = slice(n_occ_b, n_occ_b + nact_spatial)
22     if nact_spatial > 0:
23         fock_beta_spatial = fock_spin[1::2, 1::2]
24         F_beta_active = fock_beta_spatial[sl_b, sl_b]
25         evals_b, evecs_b = np.linalg.eigh(F_beta_active)
26         U_beta_full[sl_b, sl_b] = evecs_b
27
28         # Build spin-orbital canonicalization unitary
29         U_canon_spin = np.zeros_like(fock_spin)
30         U_canon_spin[0::2, 0::2] = U_alpha_full
31         U_canon_spin[1::2, 1::2] = U_beta_full
32
33         # Apply to MO coefficients and Fock
34         mo_canon = [mo_coeffs[0] @ U_alpha_full, mo_coeffs[1] @
35                     U_beta_full]
36         fock_canon = U_canon_spin.T @ fock_spin @ U_canon_spin
37
38     return mo_canon, fock_canon, [evals_a, evals_b]

```

Having the new MO coefficients and orbital energies, the algorithm proceeds to the next iteration where the MP1 amplitudes and MP2 energy are recomputed, and the convergence is assessed as described in the following section.

3.1.6 Convergence and Iteration Control

The OVOS iteration continues until convergence is achieved or the maximum number of iterations is reached. The primary convergence criterion is based on the change in the MP2 correlation energy J_2 between successive iterations, which must be less than 10^{-8} Hartree. Additionally, we monitor the norm of the gradient and the magnitude of the T1 amplitudes to ensure that the solution is not only energetically converged but also corresponds to a stationary point in the orbital space. We aid in ensuring "true" convergence by requiring that the criteria are met for a set amount of iterations, specified by the `keep_track_max`. The specific implementation of these checks and controls is detailed in Code 3.9.

Code 3.9: Convergence checking loop

```

1  while iter_count < self.max_iter:
2      iter_count += 1
3      # ... compute MP2 energy and gradient (as above) ...
4      if iter_count > 1:
5          # Check convergence criteria
6          dE = abs(energy_hist[-1] - energy_hist[-2])
7          grad_norm = np.linalg.norm(G.flatten())
8          t1_norm = self._t1_norm(t1_abij)
9
10         # Convergence criterion (all three must be satisfied):
11         if (dE < self.conv_energy and
12             grad_norm < self.conv_grad and
13             t1_norm < self.t1_conv):
14             keep_track += 1

```

```

15         if keep_track >= self.keep_track_max and iter_count
16             > 100:
17             converged = True
18             break
19     else:
20         keep_track = 0

```

The rate and value of convergence will vary depending on the initial orbital guess. An example of different convergence criteria, for the same system, with initial RHF orbitals is shown in Figure 3.1, in which the choice of criteria is illustrated by the red point. This plot explored the iteration count required for convergence across a grid of gradient norm and MP2 energy change criteria, showing how the choice of these parameters can affect the convergence behaviour of the algorithm. The landscape reveals regions where MP2 correlation energy convergence is negligible between different sets of criteria, but iteration counts vary significantly, highlighting the importance of selecting appropriate criteria for efficient optimisation. The relationship between the criteria is also evident, at a lower gradient norm threshold, the MP2 correlation energy change criterion needs to be multiple orders of magnitude tighter to achieve a noticeable change in the iteration count. At a fixed MP2 correlation energy change criterion, just one order of magnitude tightening of the gradient norm criterion leads to a significant increase in the iteration count, indicating that the gradient norm is a more sensitive criterion for convergence in this case.

OVOS Convergence Landscape: HF/6-31G, Method: RHF, Optimized Virtual Orbitals: 4

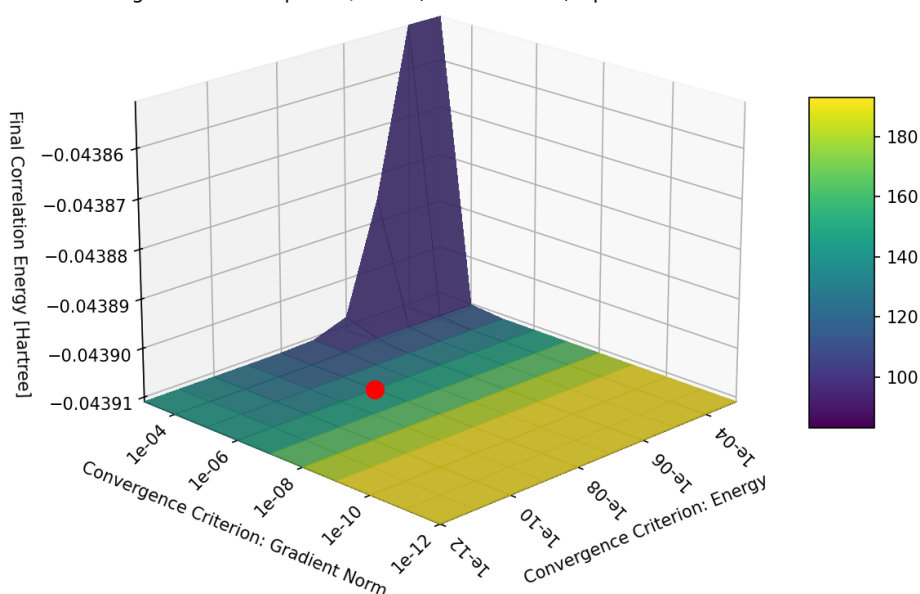


Figure 3.1: Convergence iteration count landscape for OVOS with convergence criteria gradient norm and MP2 correlation energy. Explored for the HF molecule in the 6-31G basis with initial RHF orbitals. Active space of 7 molecular orbitals. The red point indicates the final converged solution with the default criteria.

The next section describes the different initialisation strategies implemented and their impact on convergence.

3.2 Initialisation Strategies

Here we detail the three distinct initialisation strategies implemented for the OVOS algorithm, which can significantly affect convergence speed and the quality of the final orbital set found by the Newton-Raphson iteration.

3.2.1 Default SCF Initialisation

We can utilise the canonical MO coefficients obtained from a preceding RHF or UHF SCF calculation as the initial guess for the OVOS iteration, like `PySCF scf.RHF()`. This is the default strategy and is computationally efficient, as it leverages the existing SCF machinery to provide a reasonable starting point. As seen in Code 3.10, the initial MO coefficients, the SCF object, and the Fock matrices are passed directly to the OVOS constructor, and the iteration proceeds from there.

Code 3.10: OVOS initialization with RHF/UHF starting guess

```

1  ovos = OVOS(
2      mol=mol,
3      scf="scf",
4      Fao=[Fao_alpha, Fao_beta],
5      num_opt_virtual_orbs=4,
6      mo_coeff=[mo_a, mo_b],
7      init_orbs="RHF",
8      verbose=1,
9      max_iter=1000,
10     conv_energy=1e-8,
11     conv_grad=1e-4,
12     keep_track_max=50
13 )
14
15 # Run from scratch
16 E_corr, E_hist, iters, mo_final, fock_final, reason = ovos.run(
    mo_coeff, fock_spin=None)

```

If no initial Fock matrix is provided in `ovos.run` in Code 3.10, it is constructed from the AO Fock matrices and the initial MO coefficients, as shown in Code 3.11. This ensures that the MP1 amplitudes and MP2 energy can be computed correctly in the first iteration.

Code 3.11: Initial Fock matrix construction

```

1  if fock_spin is None:
2      # Transform AO Fock to MO basis
3      Fmo_a = mo_coeffs[0].T @ self.Fao[0] @ mo_coeffs[0]
4      Fmo_b = mo_coeffs[1].T @ self.Fao[1] @ mo_coeffs[1]
5
6      # Build spin-orbital Fock (block-diagonal alpha/beta)
7      fock_spin = self._build_spin_fock(Fmo_a, Fmo_b)
8
9      # Extract diagonal energies for MP2 denominators
10     eig_a = scipy.linalg.eigh(Fmo_a, eigvals_only=True)

```

```

11 eig_b = scipy.linalg.eigh(Fmo_b, eigvals_only=True)
12 self.eps = self._spatial_to_spin_energies(eig_a, eig_b)

```

3.2.2 Warm-Start from Previous Orbital Set

For a systematic scan over different numbers of active virtual orbitals $N'_{\text{virt}} = 2, 4, 6, \dots$, we can adopt a warm-start strategy where the converged orbitals from a previous run with $N'_{\text{virt}} = k$ active virtuals are used as the starting guess for the next run with $N'_{\text{virt}} = k + 2$ active virtuals. The rationale behind this approach is that nearby orbital spaces should have similar optimal MOs, and thus using the previously optimised orbitals can provide a better starting point for convergence, potentially improving the recovery fraction and reducing the iteration count. The pseudo-code for this strategy is shown in Code 3.12, see Appendix C for the full implementation.

Code 3.12: Warm-start strategy (pseudo-code for expansion)

```

1 # Previous run with  $N'_{\text{virt}}=2$  active virtuals
2 ovos_prev = OVOS(mol, scf, Fao, num_opt_virtual_orbs=2,
3   mo_coeff=..., ...)
4 E_2, _, _, mo_prev, fock_prev, _ = ovos_prev.run(mo_coeff_scf,
5   fock_spin=None)
6
7 # Use optimised orbitals as starting guess for  $N'_{\text{virt}}=4$ 
8 ovos_curr = OVOS(mol, scf, Fao, num_opt_virtual_orbs=4,
9   mo_coeff=mo_prev, # Start from previous
10   optimum
11   ...)
12 E_4, _, _, mo_curr, fock_curr, _ = ovos_curr.run(mo_prev,
13   fock_spin=fock_prev)

```

3.2.3 Random Unitary Sampling

To explore the orbital space landscape more thoroughly and potentially avoid local minima, we implement a random unitary sampling strategy. In this approach, we generate a number of random unitary rotations in the virtual orbital space by creating random matrices and orthogonalizing them using QR decomposition. Each random unitary is applied to the virtual orbital block of the initial MO coefficients, and the OVOS iteration is run from each of these starting points. The result with the lowest MP2 correlation energy J_2 at convergence is selected as the final output. This method is computationally expensive, as it requires running the full OVOS iteration n_{samples} times. The conceptual code for this strategy is shown in Code 3.13, see Appendix C for the full implementation.

Code 3.13: Random unitary initialization

```

1 # Generate random unitary rotations in virtual space
2 n_samples = 500
3 best_energy = np.inf
4 for sample in range(n_samples):
5     # Generate random matrix and orthogonalize via QR

```



```
6 random_matrix = np.random.randn(nvir, nvir)
7 Q, R = np.linalg.qr(random_matrix)
8 # Apply to virtual orbital block
9 mo_rot = mo_coeff.copy()
10 mo_rot[:, nelec:] = mo_coeff[:, nelec:] @ Q
11 # Run OVOS from this starting point
12 ovos = OVOS(mol, scf, Fao, num_opt_virtual_orbs, mo_rot,
13             ...)
14 E, E_hist, _, mo_best, _, _ = ovos.run(mo_rot)
15 # Track lowest energy
16 if E < best_energy:
17     best_energy = E
18     mo_best_overall = mo_best
19 # Use best result across all 500 random starts
20 return mo_best_overall, best_energy
```

Chapter 4

Computational Details

In this chapter, we provide a comprehensive account of the computational details underlying all calculations presented in this thesis. This includes the software and libraries used, the molecular geometries and basis sets employed, the settings for the reference SCF calculations, and the specific hyperparameters of the OVOS optimisation. The goal is to ensure that all results are fully reproducible by any reader with access to similar computational resources.

4.0.1 Software and Libraries

In this section we detail the software environment used for all calculations in this thesis. The primary quantum chemistry calculations were performed using PySCF version 2.10 [11], and the SlowQuant branch Unrestricted VQE [22]. Numerical operations were carried out using NumPy version 1.26.4 and SciPy version 1.17, a complete list of which is provided in Table 4.1.

Table 4.1: Software packages used in this work.

Package	Version	Purpose	Citation
PySCF	2.10.0	Quantum chemistry calculations	[11]
SlowQuant	-	Alternative quantum chemistry package	[22]
NumPy	1.26.4	Numerical operations	-
SciPy	1.17.0	Scientific computing	-

The OVOS method was implemented as a standalone Python class, with source code provided in Appendix C. All calculations were executed in a single-threaded mode to ensure reproducibility, with the environment variables `OPENBLAS_NUM_THREADS` and `OMP_NUM_THREADS` set to 1. The operating system used was Windows, running in Microsoft Visual Studio Code’s WSL Ubuntu virtual machine, as PySCF requires a Unix-like environment, and the Python distribution was the standard Python 3.11.

4.0.2 Molecular Geometries

The thesis focuses on a test set of five molecules that represent a variety of electronic structures, all abundantly used in quantum chemistry benchmarks, and found in nature. These include diatomic molecules (CO, HF), small polyatomic molecules (H₂O, NH₃), and a simple open-shell molecule (Li₂). All geometries were kept fixed throughout the calculations, with no geometry optimisation performed. The specific coordinates used for each molecule are listed in Table 4.2, along with their spin multiplicity, charge, and point group symmetry. The bond lengths and angles were

taken from experimental values reported in the literature, ensuring that the test set is representative of real molecular systems. The coordinate unit used is Ångström.

Table 4.2: Molecular geometries and electronic structure parameters used in this work.

Molecule	Geometry (Å)
Li ₂	$r(\text{Li} - \text{Li}) = 2.673$
H ₂ O	$r = 0.957, \angle = 104.5^\circ$
CO	$r(\text{C} - \text{O}) = 1.128$
HF	$r(\text{H} - \text{F}) = 0.917$
NH ₃	$r(\text{N} - \text{H}) = 1.012$

The basis-set-specific orbital space dimensions for each molecule are given in the following section, along with a description of the basis sets employed.

4.0.3 Basis Sets

The choice of basis set determines the size of the virtual orbital space and hence the difficulty and accuracy of the OVOS calculation. This section describes the basis sets employed and provides the resulting orbital space dimensions for each molecule. See Basis Set Exchange [23] for the source of all basis sets used in this work.

6-31G: A split-valence double-zeta basis set developed by Pople et. al. [15–17]. It consists of a minimal basis for core electrons and a double-zeta basis for valence electrons, providing greater flexibility in describing electron correlation than minimal basis sets.

cc-pVDZ: The correlation-consistent polarised valence double-zeta basis set developed by Dunning [18]. It includes polarisation functions that allow for a better description of electron correlation effects, particularly important for accurate energy calculations.

4.1 OVOS Calculation Settings

Running OVOS concerns two main steps: the OVOS object and the OVOS.run() method. The OVOS object is initialised with arguments that specify the molecule, the SCF object, the Fock matrix in the AO basis, the number of active virtual spin-orbitals to optimise, the initial MO coefficients from SCF, the type of initial orbitals (RHF or UHF), verbosity level, maximum number of iterations, energy and gradient convergence thresholds, and the number of iterations to keep track of for consistent convergence checks. The initialisation is shown in Code 4.1.

Code 4.1: OVOS Object

```

1  ovos = OVOS(
2      mol=mol,
3      scf="scf",
4      Fao="Fock matrix in AO basis",
5      num_opt_virtual_orbs="number of active virtual spin-
        orbitals to optimise",
6      mo_coeff="initial MO coefficients from SCF",
7      init_orbs="RHF",
8      verbose="verbosity level (0, 1)",
9      max_iter="maximum number of iterations",
10     conv_energy="energy convergence threshold (e.g., 1e-8)",
11     conv_grad="gradient convergence threshold (e.g., 1e-4)",
12     keep_track_max="maximum number of iterations to keep track
        of"
13 )

```

The `OVOS.run()` method is then called to execute the orbital optimisation, as shown in Code 4.2. This method takes the initial MO coefficients and an optional spin-specific Fock matrix as arguments.

Code 4.2: `OVOS.run()`

```

1  ovos.run("initial MO coefficients from SCF", fock_spin="
        optional Fock matrix for spin-specific calculations")

```

For initialisation strategies see Section 3.2.

We would like to express that the specific hyperparameters chosen for the OVOS optimisation, such as the energy convergence threshold of 10^{-8} Hartree and the maximum number of iterations, were selected based on preliminary tests to ensure a balance between computational cost and convergence reliability, see Figure 3.1. The choice of these parameters can significantly affect the performance of the OVOS method, and they were kept consistent across all calculations to allow for a fair comparison between different orbital optimisation strategies.

The Hessian indefiniteness checks can be turned on or off with verbosity level 1, and the specific implementation of these checks is shown in Code ???. These checks are crucial for ensuring that the optimisation is proceeding in a direction that leads to a minimum energy solution, they help to prevent convergence to saddle points or maxima, and identify potential issues with the optimisation landscape in forms of negative eigenvalues and condition numbers. The checks are performed at each iteration, and the results are printed to the console when verbosity level 1 is set. It was found that when convergence issues arise, they are often associated with a highly indefinite Hessian. The block-diagonal approximation of the Hessian helped reduce negative eigenvalues, which could be identified through these checks.

The T1 diagnostic, also enable by setting verbosity level 1, is calculated at each iteration to monitor the quality of the reference wavefunction and checked against the HF reference.

4.2 VQE Calculation Settings

All VQE calculations were performed using the SlowQuant branch for unrestricted VQE [22], coded up by Pernille Volsgaard Christensen Ph.D. student at the Sauer Group at UCPH. The VQE wavefunction is an unrestricted wavefunction with a specific ansatz. Here, we use tiled unitary product states (tUPS) with 1 layer and no active kappa parameters. These settings were chosen to ensure a manageable computational cost while still providing a meaningful comparison between different orbital optimisation strategies. 1 layer means the ansatz consists of a single layer of unitary operations, a common choice for initial VQE studies. The decision to exclude active kappa parameters was made to simplify the wavefunction and focus on the effects of orbital optimisation. We vary the number of electrons, the active space, the MO coefficients, and the integrals for each molecule/basis set combination, while keeping the ansatz and hyperparameters fixed across all calculations. The specific implementation of the wavefunction is shown in Code 4.3.

Code 4.3: VQE Wavefunction

```

1  UnrestrictedWaveFunctionUPS(
2      num_electrons,
3      active_space,
4      mo_coeff_list,
5      h_core_array,
6      g_eri_array,
7      "utups",
8      {"n_layers": 1},
9      include_active_kappa=False,
10 )

```

We then run the optimisation using the BFGS algorithm, with orbital optimisation either enabled or disabled, and the same energy convergence threshold as OVOS (10^{-8} Hartree). The specific implementation of the VQE run is shown in Code 4.4.

Code 4.4: Running VQE

```

1  stats_ovos_opt = run_ucc_and_get_stats(WF_ovos, "BFGS", oo,
2      atol)
3  def run_ucc_and_get_stats(wf, str_, oo, atol):
4      ...
5      wf.run_wf_optimization_1step(str_, orbital_optimization=oo,
6          tol=atol, maxiter=5000)
7      ...

```

In the results, we run VQE for different settings to better clarify these combinations of orbital optimisation and initialisation strategies, based on the following Flowchart 4.1.

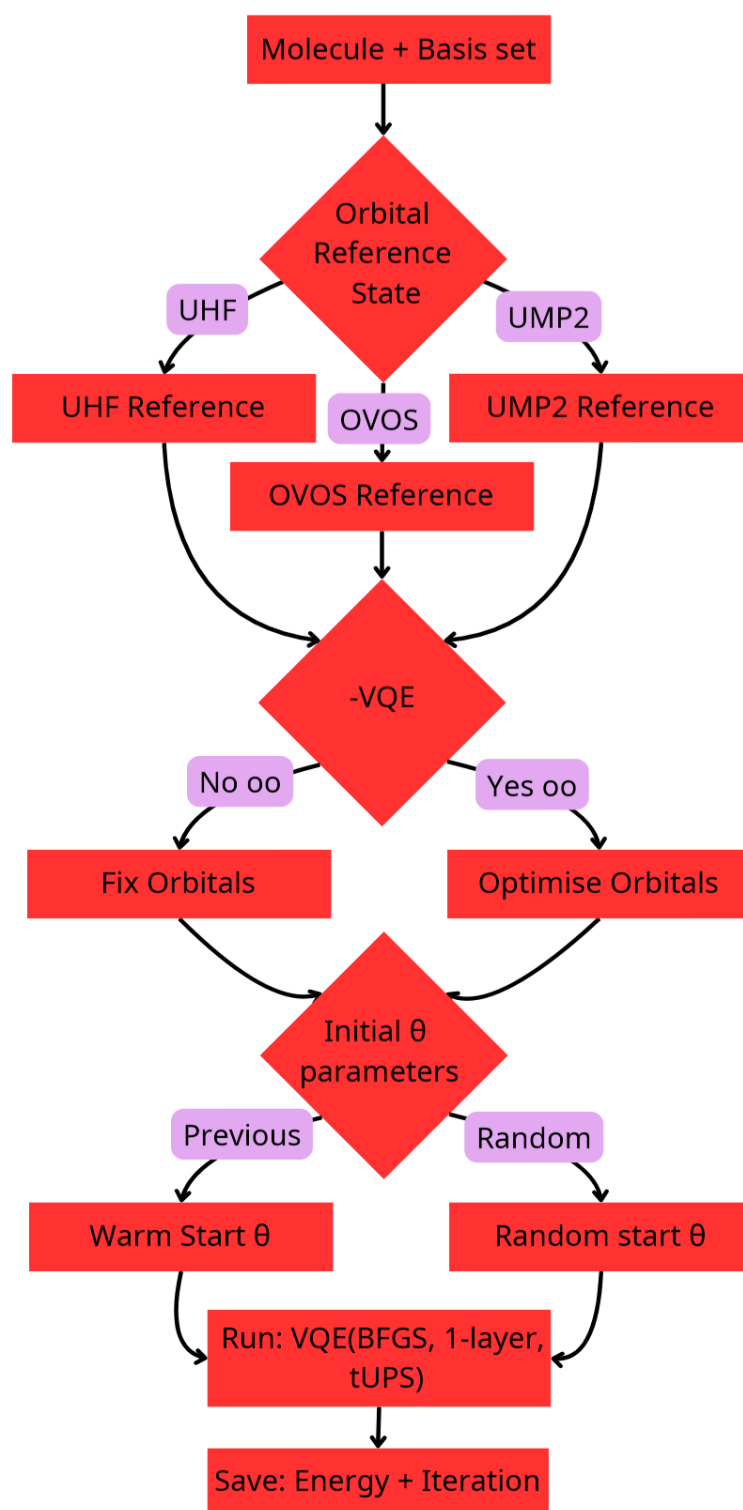


Figure 4.1: Flowchart of VQE calculation settings. The flowchart illustrates the different combinations of orbital optimisation (OVOS, no optimisation, and canonical HF orbitals) used in the VQE calculations.

Chapter 5

Results

In this chapter, the results of the OVOS implementation are presented and analysed. The content is structured into several sections, each reflecting a key aspect of the results. The main OVOS results are presented in Section 5.1, reflecting on the effect of the number of active virtual orbitals, initialization strategy, and basis set choice. Next, the performance of OVOS orbitals as a VQE reference state is presented in Section 5.2.1, where we comment on convergence behaviour and energy recovery, and how well the OVOS orbitals capture Brueckner orbitals. Finally, it is reflected on if we allow orbital optimisation in VQE, how the OVOS orbitals perform as a starting point for orbital optimisation in VQE, and if they can help to reach better minima in the orbital optimisation landscape.

5.1 OVOS

This section presents the MP2 correlation energy recovered as a function of the number of active virtual orbitals for each test molecule and basis set. The results are presented for all test molecules in the cc-pVDZ basis. The convergence behaviour of the OVOS algorithm is also analysed, with a focus on cases where convergence issues arise. The effect of different initialization strategies on the convergence and energy recovery is explored. Lastly, the cc-pVDZ results are compared against the 6-31G results to assess the basis set dependence of the OVOS algorithm. Here we use $N'_{\text{act.unocc}}$ to denote the number of active virtual orbitals, previously N'_{virt} , here N' will always denote the number of active virtual orbitals, as described in Notation and Integral Conventions 1.6.

The T1 amplitudes during the OVOS algorithm showed similar values to the T1 amplitudes of HF reference in orders of magnitude at each iteration at 10^{-16} . For lower numbers of active virtual orbitals, the T1 amplitudes were generally smaller than those of HF, while for higher numbers of active virtual orbitals, the T1 amplitudes were generally larger than those of HF. This suggests that the OVOS algorithm is effectively optimising the orbitals to minimise the T1 amplitudes, which is consistent with the goal of finding optimal orbitals for correlation energy recovery.

5.1.1 Correlation energy of OVOS

The results of the OVOS algorithm for all test molecules in the cc-pVDZ basis sets for start guess RHF, previous, and random are presented in this section. See Appendix B for OVOS on the same molecules in the 6-31G basis. We explore the results of the correlation energy recovered by OVOS as a function of the number of active unoccupied orbitals $N'_{\text{act.unocc}}$.

CO

We observe that OVOS smoothly recovers more of the total correlation energy $E_{\text{corr}}^{\text{OVOS}}$ as $N'_{\text{act.unocc}}$ increases, with the percentage of correlation energy recovered reaching 100% at full space $N'_{\text{act.unocc}} = 21$ for start guess RHF. Reaching 90% recovery at $N'_{\text{act.unocc}} = 13$ for all start guesses, which is approximately 62% of the total number of active unoccupied orbitals for previous start guess. The OVOS algorithm results for CO in the cc-pVDZ basis through each $N'_{\text{act.unocc}}$ to full space of active unoccupied orbitals for the three start guesses are shown in Figure 5.1.

We mark the difference the OVOS algorithm make, the amount of correlation energy recovered, at each $N'_{\text{act.unocc}}$ by an initial point with iterations to the final point for each $N'_{\text{act.unocc}}$ in Figure 5.1 as the points for each $N'_{\text{act.unocc}}$. The most recovered correlation energy is between $N'_{\text{act.unocc}} = 3$ and $N'_{\text{act.unocc}} = 10$ with $\Delta E_{\text{corr}}^{\text{OVOS}} \simeq 0.1$ Hartree for start guess RHF. It is observed that start guess random consistently finds higher initial correlation energies, and in contrast start guess previous finds consistently lower correlation energies. It is observed that the change in converged correlation energy from previous number of active unoccupied orbitals falls as $N'_{\text{act.unocc}}$ increases, with a few exceptions like between $N'_{\text{act.unocc}} = 4$ and $N'_{\text{act.unocc}} = 5$. Convergence is nearly achieved in all cases, only $N'_{\text{act.unocc}} = 6$ for RHF reaches maximum iterations and is not converged, see Appendix B. Iteration counts show no clear pattern for any of the start guesses as $N'_{\text{act.unocc}}$ reaches the total number of active unoccupied orbitals. The correlation energy iterations are plotted in Appendix B to show the convergence behaviour of the OVOS algorithm for CO in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$.

Shortcomings of the OVOS algorithm are observed at $N'_{\text{act.unocc}} = 11$ through to $N'_{\text{act.unocc}} = 17$ for the different start guesses, where the algorithm fails to converge at an agreeable correlation energy, and instead converges to a local minimum with a much higher correlation energy. It is however observed in this interval that RHF performs more similar to the trend of other numbers of active unoccupied orbitals. This suggests that the orbital optimisation landscape in OVOS has multiple local minima, which can trap the algorithm if the initial guess is not close enough to the global minimum. This is backed up by the fact it was observed that the Hessian matrix has negative eigenvalues in these regions, as the blocks within grow in size, capturing more of the off-diagonal elements.

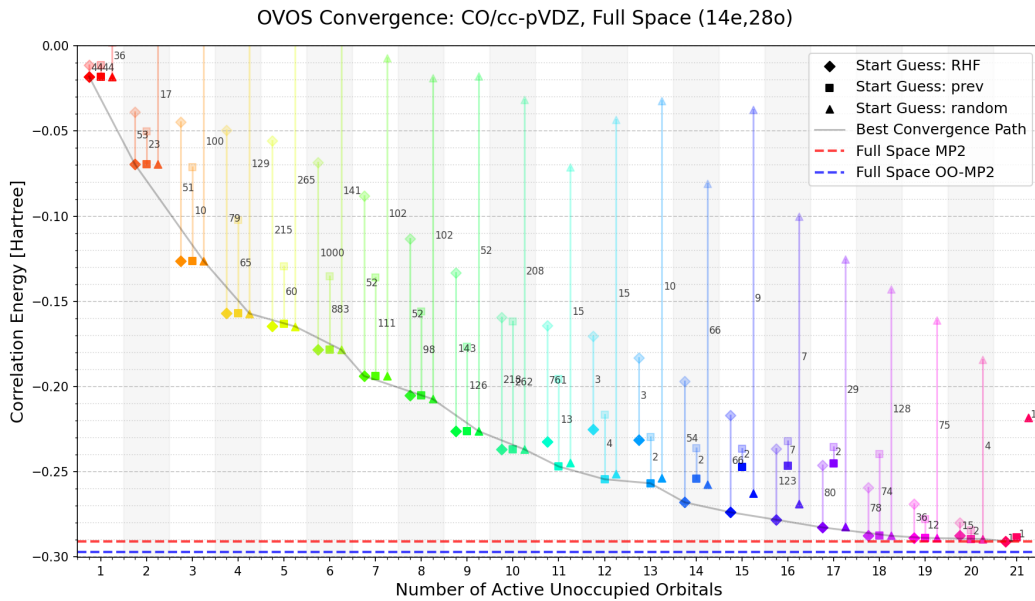


Figure 5.1: Convergence of the MP2 correlation energy for CO in the cc-pVDZ basis at each number of active unoccupied orbitals ($N'_{\text{act.unocc}}$). The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red) and OO-MP2 (blue) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

H₂O

OVOS recovers increasingly more of the total correlation energy $E_{\text{corr}}^{\text{OVOS}}$ as $N'_{\text{act.unocc}}$ increases, with the percentage of correlation energy recovered reaching 100% at $N'_{\text{act.unocc}} = 19$ for start guess RHF. Reaching 90% recovery at $N'_{\text{act.unocc}} = 13$, which is approximately 68% of the total number of active unoccupied orbitals. It is seen that from $N'_{\text{act.unocc}} = 17$ the start guesses approximate the full-space correlation energy more closely. The OVOS algorithm results for H_2O in the cc-pVDZ basis through each $N'_{\text{act.unocc}}$ to full space of active unoccupied orbitals are shown in Figure 5.2.

The difference OVOS makes for H_2O at each $N'_{\text{act.unocc}}$ is marked in Figure 5.2. The most recovered correlation energy is for $N'_{\text{act.unocc}} \in \{3, 4, 9, 10\}$ with $\Delta E_{\text{corr}}^{\text{OVOS}} \simeq 0.1$ Hartree for start guess RHF. Likewise for CO it is observed start guess previous finds consistently lower correlation energies, and start guess random finds consistently higher correlation energies. Convergence is achieved in nearly all cases, except three cases for $N'_{\text{act.unocc}} = 13$ for RHF, and $N'_{\text{act.unocc}} = 12, 8$ for previous, which is observed to agree with the values for the other start guesses, as seen in Figure 5.2. There is no region of non-convergence to a local minimum as $N'_{\text{act.unocc}}$ increases like seen for CO . Except for a little deviation for $N'_{\text{act.unocc}} = 11$ for previous. Iteration counts show no clear pattern as $N'_{\text{act.unocc}}$ increases nor between the start guesses, with the highest iteration count at $N'_{\text{act.unocc}} = 13$ for random only accounting for converged cases. The correlation energy iterations are plotted in Appendix B to show the convergence behaviour of the OVOS algorithm for H_2O in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$.

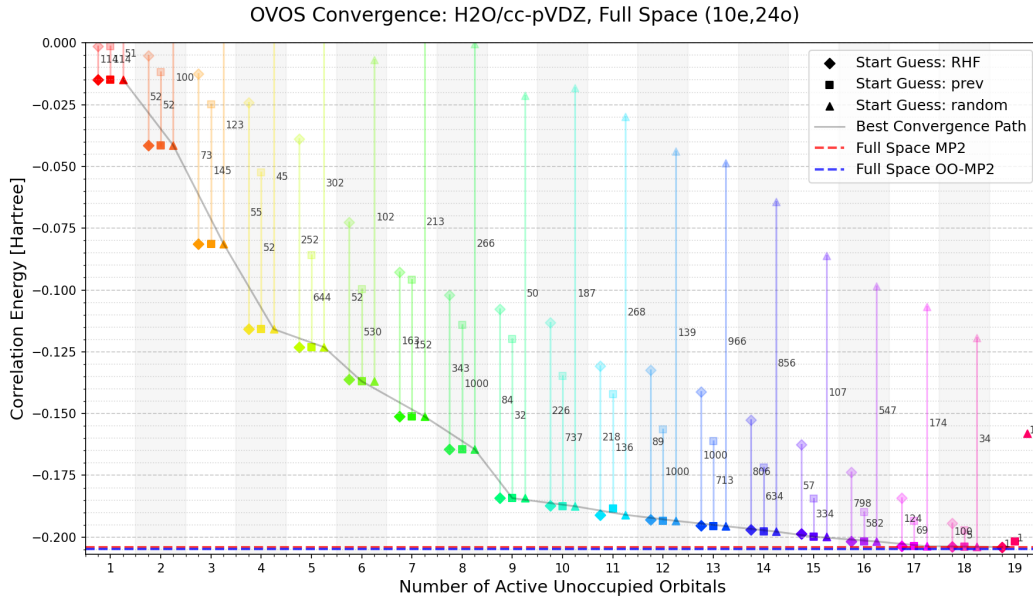


Figure 5.2: Convergence of the MP2 correlation energy for H_2O in the cc-pVDZ basis at each number of active unoccupied orbitals ($N'_{\text{act.unocc}}$). The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red) and OO-MP2 (blue) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

HF

OVOS recovers increasingly more of the total correlation energy $E_{\text{corr}}^{\text{OVOS}}$ as $N'_{\text{act.unocc}}$ increases, with the percentage of correlation energy recovered reaching 100% at $N'_{\text{act.unocc}} = 14$ for RHF. Reaching 90% recovery at $N'_{\text{act.unocc}} = 9$, consistently for all start guesses, which is approximately 64% of the total number of active unoccupied orbitals. The OVOS algorithm results for HF in the cc-pVDZ basis through each $N'_{\text{act.unocc}}$ to full space of active unoccupied orbitals are shown in Figure 5.3.

The most correlation energy recovered by OVOS for HF is at $N'_{\text{act.unocc}} = 3, 4, 9$ with $\Delta E_{\text{corr}}^{\text{OVOS}} \simeq 0.625$ Hartree for RHF. Here we notice some previous start guesses find higher initial correlation energies than RHF, while the random start guess consistently finds higher correlation energies. Convergence is achieved in all cases for RHF, with iteration counts increasing and decreasing as $N'_{\text{act.unocc}}$ increases. Previous finds it reaches max iterations at $N'_{\text{act.unocc}} = 5$ and random at $N'_{\text{act.unocc}} = 10, 12, 6$. Iteration counts show no clear pattern as $N'_{\text{act.unocc}}$ increases, with the highest iteration count at $N'_{\text{act.unocc}} = 6$ for previous reaching 867, undergoing both oscillations and divergence. The correlation energy iterations are plotted in Appendix B to show the convergence behaviour of the OVOS algorithm for HF in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$.

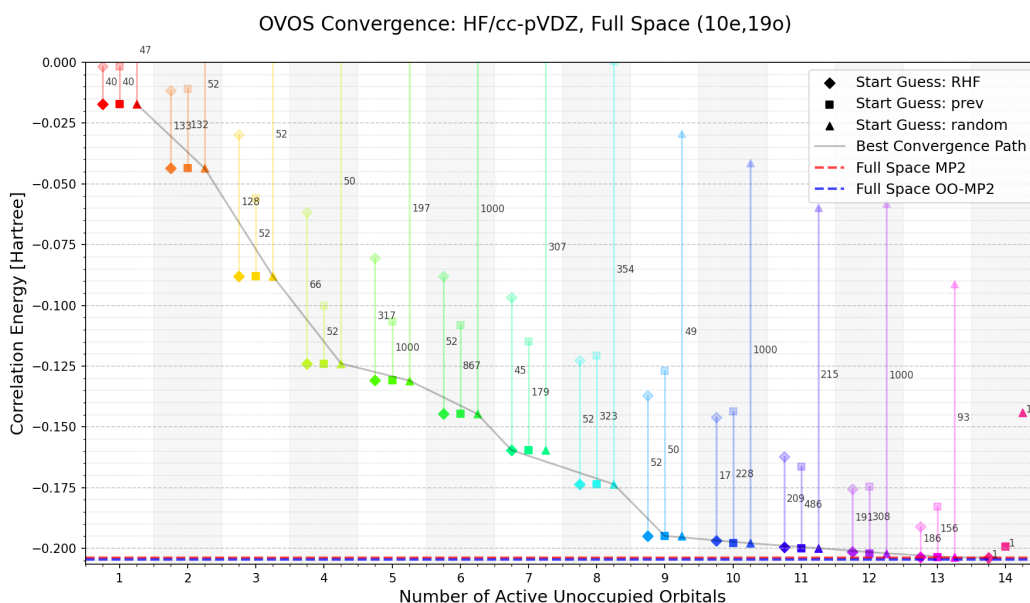


Figure 5.3: Convergence of the MP2 correlation energy for HF in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red) and OO-MP2 (blue) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

NH₃

OVOS recovers increasingly more of the total correlation energy $E_{\text{corr}}^{\text{OVOS}}$ as $N'_{\text{act.unocc}}$ increases, except for $N'_{\text{act.unocc}} = 20$ which struggle, to the amount that the best of

the three start guesses is no lower than that of the previous $N'_{\text{act.unocc}}$. It still holds that the percentage of correlation energy recovered reaches 100% at $N'_{\text{act.unocc}} = 24$ for RHF, and we reach 90% recovery at $N'_{\text{act.unocc}} = 12$ for all start guesses, which is approximately 50% of the total number of active unoccupied orbitals. It is seen that from $N'_{\text{act.unocc}} = 21$ the start guesses approximate the full-space correlation energy more closely. The OVOS algorithm results for NH_3 in the cc-pVDZ basis through each $N'_{\text{act.unocc}}$ to full space of active unoccupied orbitals are shown in Figure 5.4.

The most correlation energy recovered by OVOS for NH_3 is at $N'_{\text{act.unocc}} = 3, 4, 5$ with $\Delta E_{\text{corr}}^{\text{OVOS}} \simeq 0.75$ Hartree for RHF. It is observed previous starts at a lower correlation energy than RHF, in contrast random shows consistent higher initial correlation energies, likewise the previous molecules. At $N'_{\text{act.unocc}} = 8$ for RHF and $N'_{\text{act.unocc}} = 6, 10$ for previous OVOS does not converge, reaching max iterations, which is observed to stop at an agreeable correlation energy, between $N'_{\text{act.unocc}} = 9$ and $N'_{\text{act.unocc}} = 10$, as seen in Figure 5.4. Iteration counts show no clear pattern as $N'_{\text{act.unocc}}$ increases, with the highest iteration count at $N'_{\text{act.unocc}} = 23$ for RHF of 797, undergoing both oscillations and divergence. The correlation energy iterations are plotted in Appendix B to show the convergence behaviour of the OVOS algorithm for NH_3 in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$.

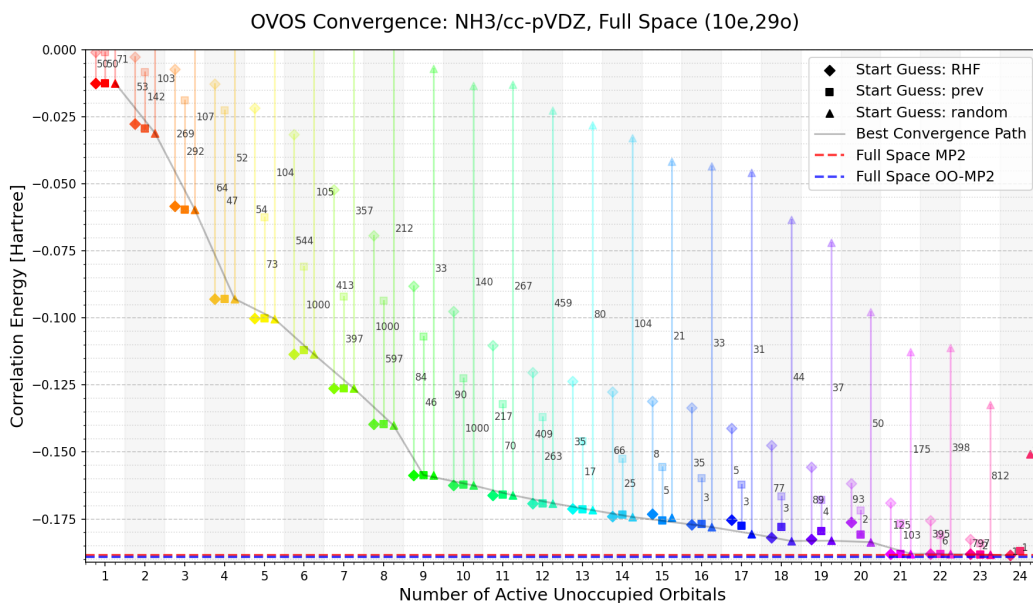


Figure 5.4: Convergence of the MP2 correlation energy for NH_3 in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red) and OO-MP2 (blue) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

Li₂

OVOS increasingly recovers more of the total correlation energy $E_{\text{corr}}^{\text{OVOS}}$ as $N'_{\text{act.unocc}}$ increases, quietly slowing down as $N'_{\text{act.unocc}}$ approaches 9, hereafter convergence

issues show up at $N'_{\text{act.unocc}} = 12$, as multiple points deviate from the expected correlation energy behavior toward full space. The percentage of correlation energy recovered reaches 100% at $N'_{\text{act.unocc}} = 25$ for RHF, and we reach 90% recovery at $N'_{\text{act.unocc}} = 7$, which is approximately 28% of the total number of active unoccupied orbitals. It is seen that from $N'_{\text{act.unocc}} = 22$ the RHF start guess approximates the full-space correlation energy more closely. The OVOS algorithm results for Li_2 in the cc-pVDZ basis through each $N'_{\text{act.unocc}}$ to full space of active unoccupied orbitals are shown in Figure 5.5.

The most correlation energy recovered by OVOS for Li_2 is at $N'_{\text{act.unocc}} = 5, 6, 7$ with $\Delta E_{\text{corr}}^{\text{OVOS}} \simeq 0.0125$ Hartree for RHF. Disregarding the convergence issues at higher $N'_{\text{act.unocc}}$ values, previous and random show consistent behavior as found for the other molecules. OVOS converges for all $N'_{\text{act.unocc}}$, no max iterations show up for any start guess. Iteration counts show no clear pattern as $N'_{\text{act.unocc}}$ increases, with the highest iteration count at $N'_{\text{act.unocc}} = 23$ of 668 for random start, undergoing both oscillations and divergence. The region, as seen for CO , shows similar bad convergence behavior, here the region is from $N'_{\text{act.unocc}} = 12$ to $N'_{\text{act.unocc}} = 23$, and only a few points of start guess RHF look to follow the original trend at $N'_{\text{act.unocc}} = 15, 19, 20, 22, 23$. The correlation energy iterations are plotted in Appendix B to show the convergence behaviour of the OVOS algorithm for Li_2 in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$.

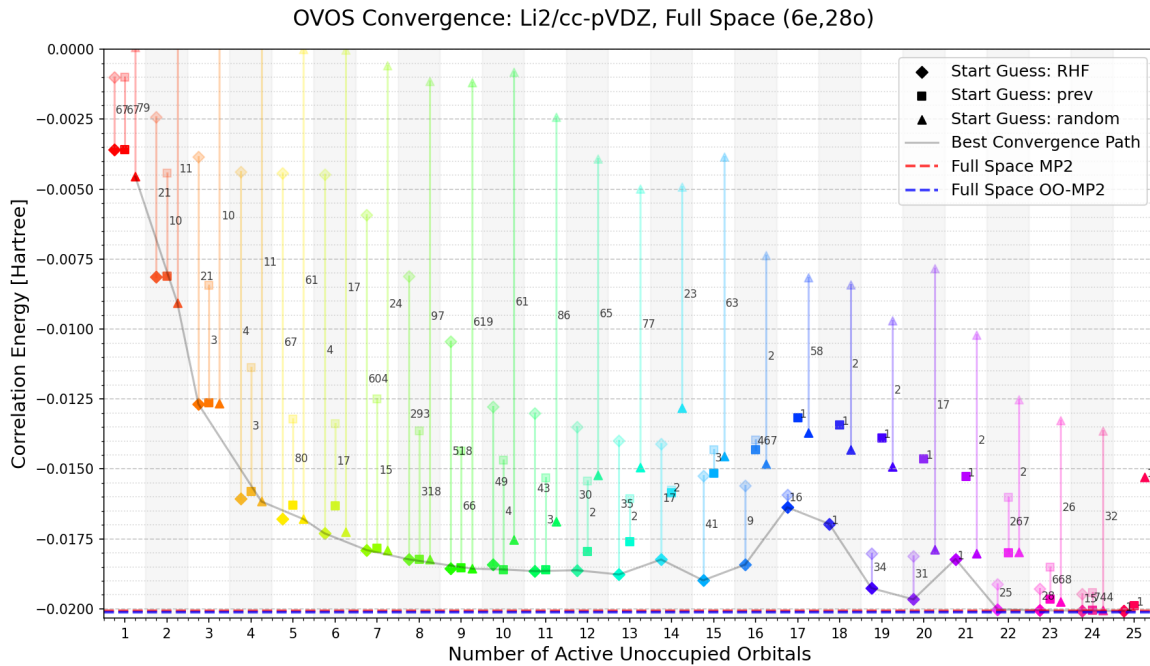


Figure 5.5: Convergence of the MP2 correlation energy for Li_2 in the cc-pVDZ basis at each $N'_{\text{act.unocc}}$. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red) and OO-MP2 (blue) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

5.1.2 Basis Set Dependence

As the OVOS algorithm optimises the orbitals to recover correlation energy, the choice of basis set influences the number of virtual orbitals and their character, which in turn affects the energy recovery. In this section, I compare the energy recovery curves for different basis sets (6–31G, cc–pVDZ) for the benchmarked molecules. We use the best results of initialization strategies based on the results from the previous section.

CO

Observing the correlated energy curves for CO in the 6–31G (11) and cc–pVDZ (21) basis sets, we see that the larger cc–pVDZ basis set, which has more virtual orbitals, allows for a higher fraction of correlation energy to be recovered at lower $N'_{\text{act.unocc}}$. At $N'_{\text{act.unocc}} = 25\%$ of the total active virtual orbitals, the cc–pVDZ basis recovers less than 6–31G basis, but as $N'_{\text{act.unocc}}$ increases, the two basis sets show similar trends. The energy recovery curves for CO in the 6–31G and cc–pVDZ basis sets are shown in Figure 5.6.

If instead we looked at a fixed number of active virtual orbitals, $N'_{\text{act.unocc}} = 5$, their percentages of the total active virtual orbitals are different for the two basis sets, with 6–31G having a higher percentage than cc–pVDZ. We can find this point by counting 5 points along the line for each basis set in Figure 5.6. At this fixed number of active virtual orbitals, the 6–31G basis recovers a higher fraction of correlation energy than the cc–pVDZ basis, as the total number of virtual orbitals is smaller in 6–31G.

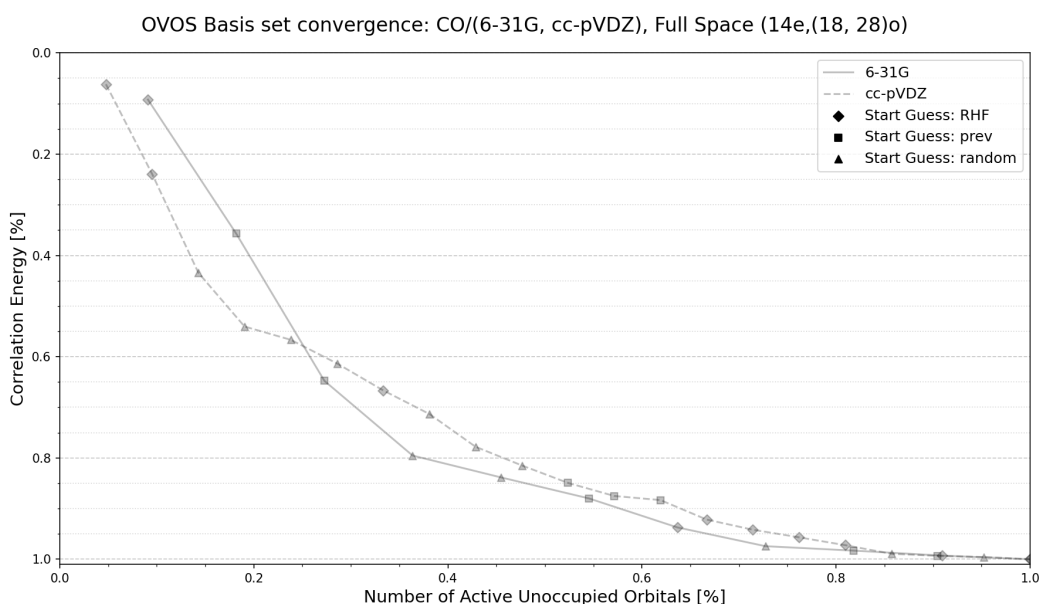


Figure 5.6: Basis set dependence of energy recovery for CO. The MP2 energy correlation is plotted against the active virtual orbitals for the basis sets: 6–31G (stripped) and cc–pVDZ (full) line.

H₂O

The correlated energy curves for H₂O in the 6–31G (8) and cc–pVDZ (19) basis sets show a stronger trend in the larger cc–pVDZ basis set, beating the smaller 6–31G basis set at most points along the curve, see Figure 5.7. At $N'_{\text{act.unocc}} = 25\%$ of the total active virtual orbitals, the cc–pVDZ basis recovers more correlation energy than the 6–31G basis. As $N'_{\text{act.unocc}}$ increases, the two basis sets show similar trends, with the cc–pVDZ basis meeting the 6–31G basis. At a fixed number of active virtual orbitals, $N'_{\text{act.unocc}} = 5$, the 6–31G basis recovers a higher correlation energy than the cc–pVDZ basis, as the total number of virtual orbitals is smaller in 6–31G.

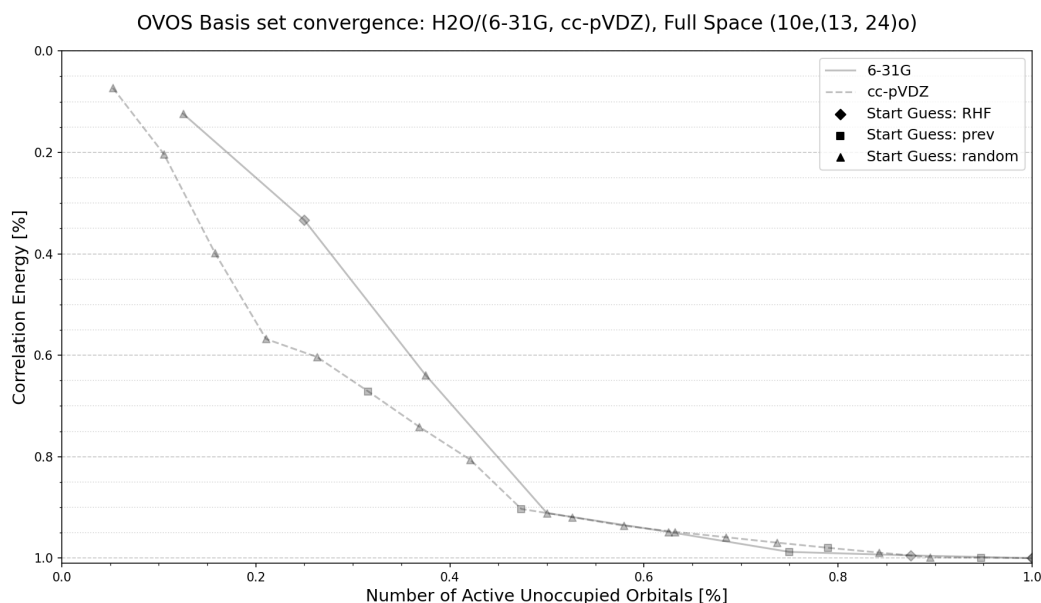


Figure 5.7: Basis set dependence of energy recovery for H₂O. The MP2 energy correlation is plotted against the active virtual orbitals for the basis sets: 6–31G (stripped) and cc–pVDZ (full) line.

HF

The correlated energy curves for HF in the 6–31G (6) and cc–pVDZ (14) basis sets show a stronger trend in the larger cc–pVDZ basis set, beating the smaller 6–31G basis set at most points along the curve, see Figure 5.8. At $N'_{\text{act.unocc}} = 25\%$ of the total active virtual orbitals, the cc–pVDZ basis recovers more correlation energy than the 6–31G basis. As $N'_{\text{act.unocc}}$ increases, the two basis sets show similar trends, with the cc–pVDZ basis meeting the 6–31G basis. At a fixed number of active virtual orbitals, $N'_{\text{act.unocc}} = 5$, the 6–31G basis recovers a higher correlation energy than the cc–pVDZ basis, as the total number of virtual orbitals is smaller in 6–31G.

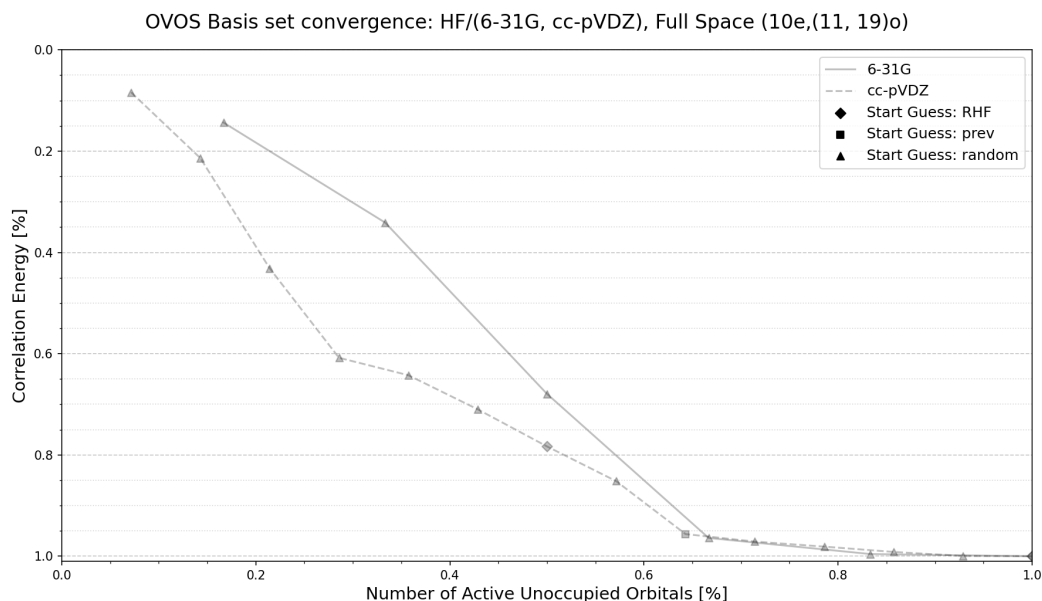


Figure 5.8: Basis set dependence of energy recovery for HF. The MP2 energy correlation is plotted against the active virtual orbitals for the basis sets: 6–31G (stripped) and cc–pVDZ (full) line.

NH₃

The correlated energy curves for NH₃ in the 6–31G (10) and cc–pVDZ (24) basis sets show a stronger trend in the larger cc–pVDZ basis set, beating the smaller 6–31G basis set at most points along the curve, see Figure 5.9. At $N'_{\text{act.unocc}} = 25\%$ of the total active virtual orbitals, the cc–pVDZ basis recovers more correlation energy than the 6–31G basis. As $N'_{\text{act.unocc}}$ increases, the two basis sets show similar trends, with the cc–pVDZ basis meeting the 6–31G basis. At a fixed number of active virtual orbitals, $N'_{\text{act.unocc}} = 5$, the 6–31G basis recovers a higher correlation energy than the cc–pVDZ basis, as the total number of virtual orbitals is smaller in 6–31G.

Li₂

The correlated energy curves for Li₂ in the 6–31G (15) and cc–pVDZ (25) basis sets show a similar trend, with the cc–pVDZ basis set generally recovering more correlation energy than the 6–31G basis set. This is consistent with the observations for previous molecules, where larger basis sets tend to perform better in terms of energy recovery, especially at lower $N'_{\text{act.unocc}}$. At $N'_{\text{act.unocc}} = 25\%$ of the total active virtual orbitals, the cc–pVDZ basis recovers more correlation energy than the 6–31G basis. As $N'_{\text{act.unocc}}$ increases, the two basis sets show similar trends, with the cc–pVDZ basis meeting the 6–31G basis. Where when they get to the bad points of cc–pVDZ, the 6–31G basis performs better. At a fixed number of active virtual orbitals, $N'_{\text{act.unocc}} = 5$, the 6–31G basis recovers a higher correlation energy than the cc–pVDZ basis, as the total number of virtual orbitals is smaller in 6–31G.

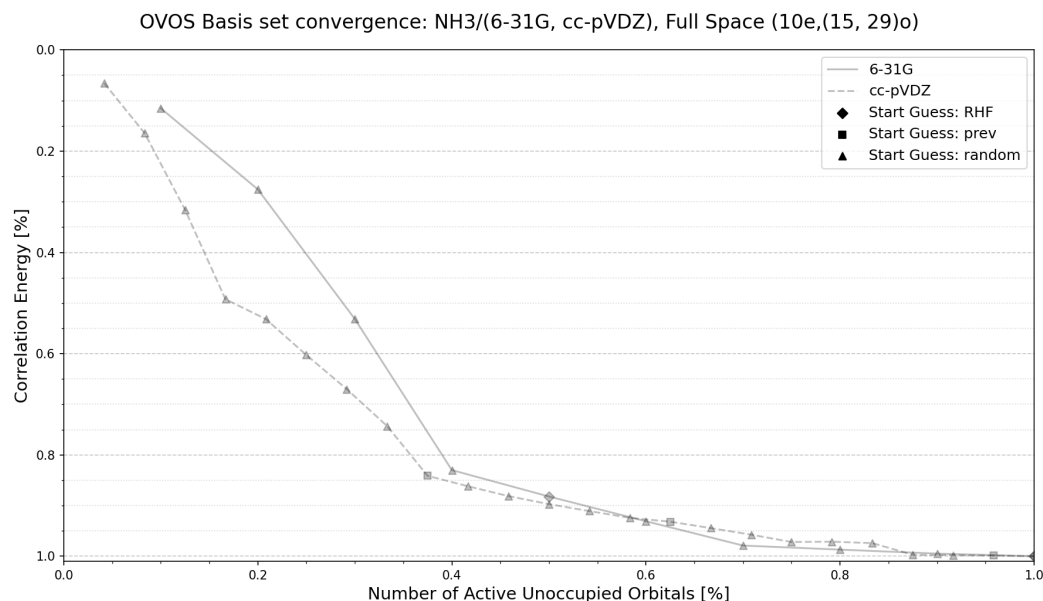


Figure 5.9: Basis set dependence of energy recovery for NH₃. The MP2 energy correlation is plotted against the active virtual orbitals for the basis sets: 6–31G (stripped) and cc–pVDZ (full) line.

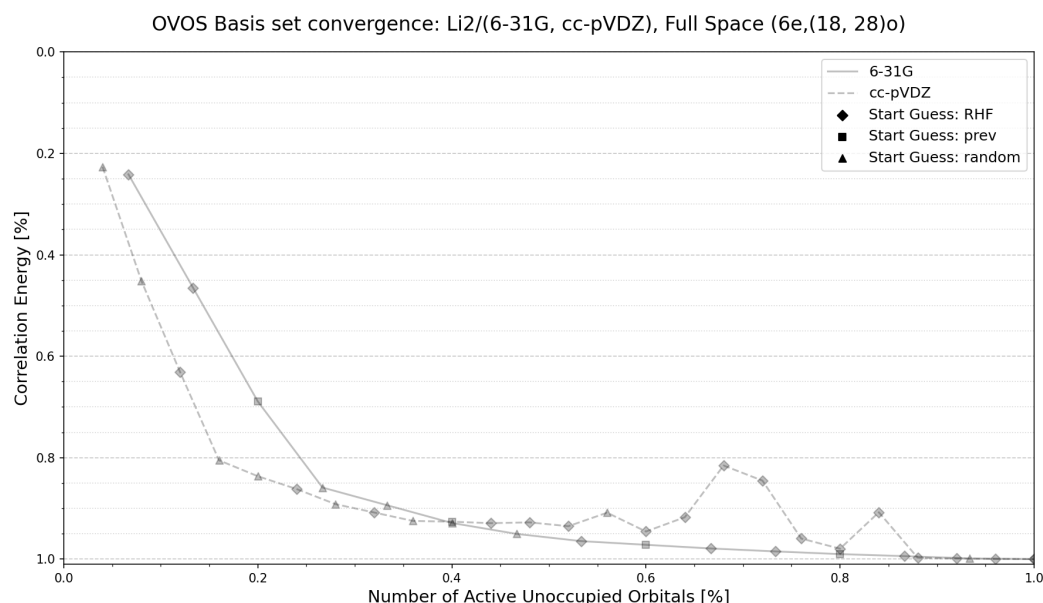


Figure 5.10: Basis set dependence of energy recovery for Li₂. The MP2 energy correlation is plotted against the active virtual orbitals for the basis sets: 6–31G (stripped) and cc–pVDZ (full) line.

Comparison with Adamowicz and Bartlett (1987)

Adamowicz & Bartlett's 1987 paper presents OVOS results for the CH₂ (¹A₁) molecule in a (9s7p2d 1f,5s2p) contracted Gaussian basis set in their Table I, where they note that it has the size of 70 active virtual orbitals [8]. I include their results here to show the potential of OVOS at bigger basis sets, and to compare with the results

obtained in this work. However, it is important to note that there are differences in the computational details between their work and this one, such as their handling of convergence (which does not seem to be detailed, e.g. ill-conditioned Hessian, iteration count, ...), and the geometry of the molecule. These differences can lead to variations in the correlation energy values, making a direct comparison challenging.

Molecule / N'	100%	~50%	~30%	~10%
CH_2^+ (70)	99%	~96%	~88%	~50%
Li_2 (25)	100%	~92%	~90%	~50%
NH_3 (24)	100%	~90%	~66%	~18%
CO (21)	100%	~86%	~70%	~22%
H_2O (19)	100%	~90%	~65%	~15%
HF (14)	100%	~89%	~61%	~16%

Table 5.1: Comparison of my implementation of OVOS for other molecules in cc-pvCVZ basis set against the values from Adamowicz & Bartlett [8] (noted by a †). The energies are in percentages of the full space correlation energy for each molecule, of which in parentheses their active virtual orbital counts are given.

Looking past not being able to replicate the exact setup of Adamowicz & Bartlett, the basis sets behaviour shows some similarities with their findings. It is agreeable that around 90% of the correlation energy can be recovered with around 50% of the active virtual orbitals. It is less agreeable for the 30% and 10% of the active virtual orbitals, where we see only Li_2 finds similar findings, despite its regions of bad convergence, see Section 5.1.1. The other molecules show a much lower percentage of correlation energy recovery at these lower active virtual orbital counts, which is likely due to molecular and basis set differences. Bigger basis sets might be necessary for these molecules. The comparison of the energy recovery percentages for the different molecules in this work against the values from Adamowicz & Bartlett [8] are shown in Table 5.1.

5.2 VQE

This section presents the non orbital optimised VQE results for the test molecules, using the OVOS optimised orbitals as the reference state. The goal is to quantify the practical benefit of OVOS MOs for quantum computing by computing the potential energy surface, the reference-state overlap, and comment on their relation to Brueckner orbitals. The VQE results are compared to those obtained using the UHF orbitals and UMP2 natural orbitals as reference states, to assess the improvement in energy convergence and resource savings.

5.2.1 VQE potential energy surfaces plot

Below I present the potential energy surfaces for H_2O , HF , and Li_2 for the VQE calculations using the OVOS optimised orbitals as reference states. Here, the VQE calculations are performed without orbital optimisation, as detailed in Section 2.4.2, and the results are compared to those obtained using the UHF orbitals and UMP2

natural orbitals as reference states. When it comes to potential energy surfaces, a common way to visualize the performance of different reference states is to plot the energy as a function of a reaction coordinate, such as bond length or angle, for each reference state. This allows us to see how well the VQE calculations converge to the true ground state energy across different geometries. The plot will highlight if the orbitals find lower states correctly, and if the energy convergence is smooth or if there are any irregularities. The goal is to assess the improvement in energy convergence when using OVOS optimised orbitals for VQE calculations.

H₂O

For H₂O, in 6-31G basis set, the VQE calculations using the OVOS optimised orbitals as reference states show a consistent small improvement over the UHF orbitals, with a lower energy convergence. Both the OVOS optimised orbitals and UHF orbitals show a smooth potential energy surface, along with a dip into a lower energy state, with the OVOS optimised orbitals showing a slightly better convergence to the true ground state energy across different geometries. The UMP2 natural orbitals, show an even lower energy convergence than the UHF orbitals and OVOS optimised orbitals, which is expected given that they are typically considered to be better reference states for correlated calculations. The OVOS optimised orbitals seem to catch up to the UMP2 natural orbitals in terms of energy convergence after finding the lower energy state. Observing the plots compared to the reference UHF curve, the VQE calculations all show improved convergence across the potential energy surface. The potential energy surfaces for H₂O using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.11.

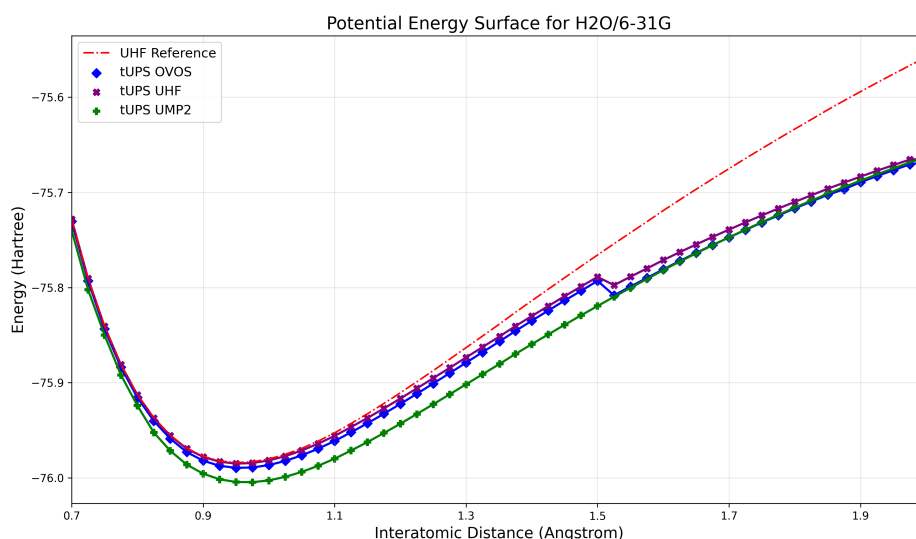


Figure 5.11: Potential energy surfaces for H₂O in 6-31G basis set, using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The geometries vary the interatomic bond length, the angle between the bonds are kept constant. The active space is the same for all calculations with 10 electrons and 6 virtual orbitals (75% of the full virtual space). The dashed lines represent the reference energies from PySCF. The plot shows the energy convergence across different geometries.

HF

For HF, in 6-31G basis set, the VQE calculations using the OVOS optimised orbitals as reference states show a consistent improvement over the UHF orbitals, with a lower energy convergence, up until it shows a dip down into a lower energy state, where the UHF orbitals show a better convergence. Both the OVOS optimised orbitals and UHF orbitals show a smooth potential energy surface, along with a dip into a lower energy state at the same interatomic distance at 1.32 Å. The UMP2 natural orbitals, are caught up to by the UHF orbitals after finding the lower energy state, and show a better convergence than the OVOS optimised orbitals across the potential energy surface. Observing the plots compared to the reference UHF curves, the VQE calculations, except for the UHF orbitals before dipping into the lower energy state, show improved convergence across the potential energy surface. The potential energy surfaces for HF using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.12.

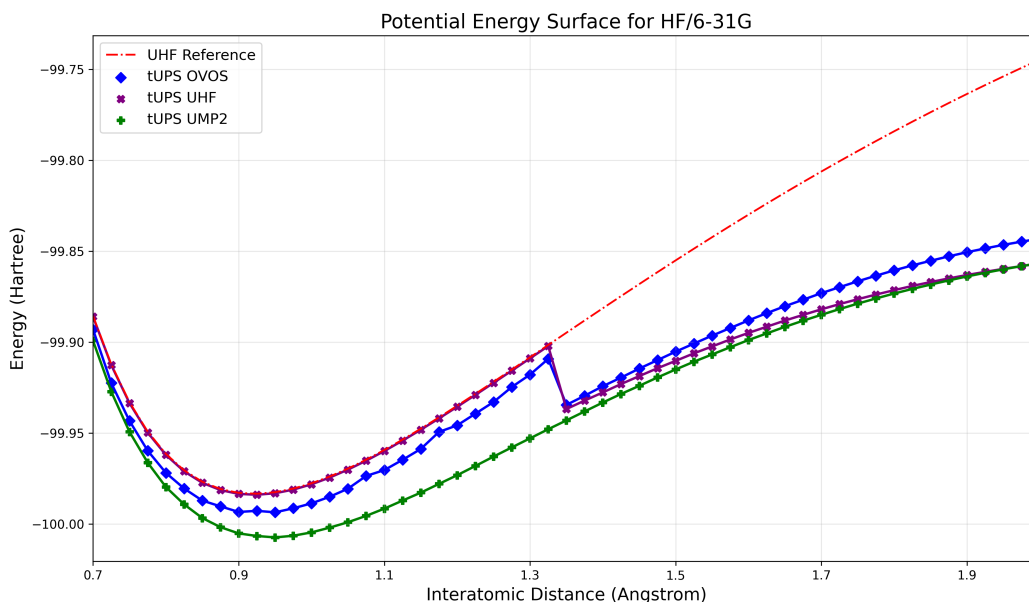


Figure 5.12: Potential energy surfaces for HF in 6-31G basis set, using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The active space is the same for all calculations with 10 electrons and 5 virtual orbitals (75% of the full virtual space rounded up). The dashed lines represent the reference energies from PySCF. The plot shows the energy convergence across different geometries.

Li₂

For Li₂, in 6-31G basis set, the VQE calculations using the OVOS optimised orbitals as reference states show a consistent improvement over the UHF orbitals, with a lower energy convergence. Both the OVOS optimised orbitals and UHF orbitals do not show a smooth potential energy surface, still the UMP2 natural orbitals show a smooth potential energy surface. The UMP2 natural orbitals, show a better energy convergence than the UHF orbitals and OVOS optimised orbitals, which is expected

given that they are typically considered to be better reference states for correlated calculations. The OVOS optimised orbitals never catch up to the UMP2 natural orbitals in terms of energy convergence after dipping into the lower energy state. Observing the plots compared to the reference UHF curve, all VQE calculations show improved convergence across the potential energy surface. The potential energy surfaces for Li_2 using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.13.

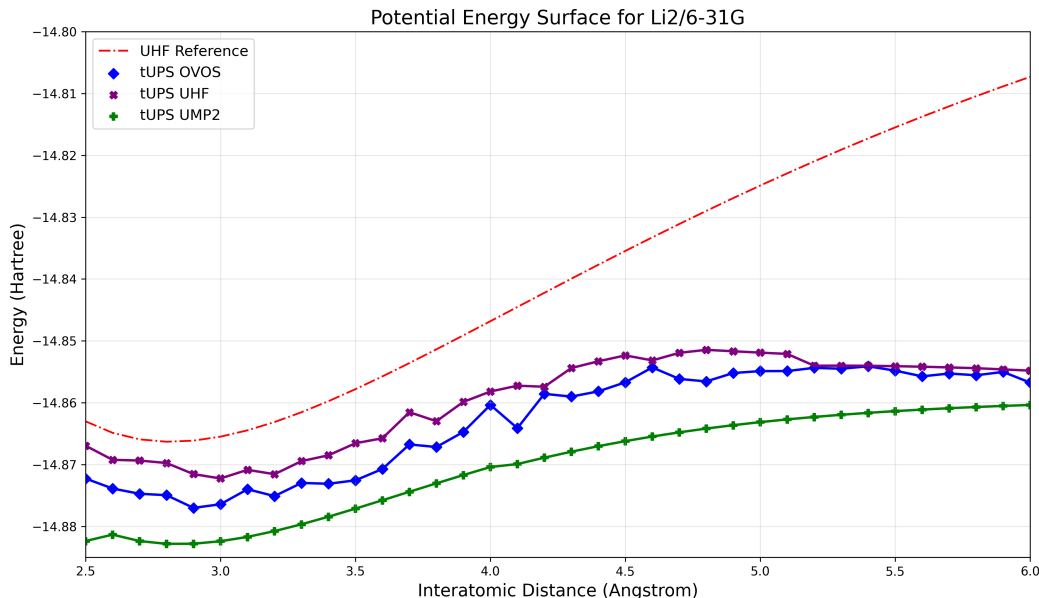


Figure 5.13: Potential energy surfaces for Li_2 in 6-31G basis set, using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The active space is the same for all calculations with 6 electrons and 12 virtual orbitals (75% of the full virtual space rounded up). The dashed lines represent the reference energies from PySCF. The plot shows the energy convergence across different geometries.

5.3 ooVQE

Different to the VQE calculations presented in the previous section, here we present the results in regard to convergence iteration count for the orbital optimised VQE (ooVQE) calculations, where the orbitals are optimised in the VQE loop, as detailed in Section ???. These results are compared to the non orbital optimised VQE calculations, where the orbitals are not optimised in the VQE loop, and are instead fixed to the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states. Further we distinguish between using the previous geometry final thetas as initialization for the VQE calculations, and using random initialization of thetas, to see if there is a difference in convergence iteration count between these two approaches. These iteration counts are presented for all the geometries along the potential energy surface in a boxplot, to see the distribution of iteration counts across the potential energy surface. The energy convergence is disregarded but look to Section 5.3 and appendix B for more details. There is a lack of entries for the ooVQE random

geometry final thetas initialization for H_2O and Li_2 as I ran out of time. The goal is to assess the improvement in convergence iteration count between using OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states in the non orbital optimised VQE calculations.

H_2O

For H_2O , in 6-31G basis set, the ooVQE calculations show a similar convergence iteration count to the non orbital optimised VQE calculations when using the previous geometry final values as initialization. Against no orbital optimisation and random initialization of thetas, the ooVQE calculations show a significantly faster convergence, with a much lower iteration count to reach convergence. Within the no orbital optimisation and random initialization of thetas, the mean convergence iteration count for the OVOS optimised orbitals is lower than that for the UHF orbitals and UMP2 natural orbitals, with the UHF orbitals showing a lower mean convergence iteration count than the UMP2 natural orbitals. Overall, there is a significant spread in the convergence iteration counts for OVOS orbitals, with some geometries calculations showing a very fast convergence, and others showing a much slower convergence. The potential energy surfaces for H_2O using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.14.

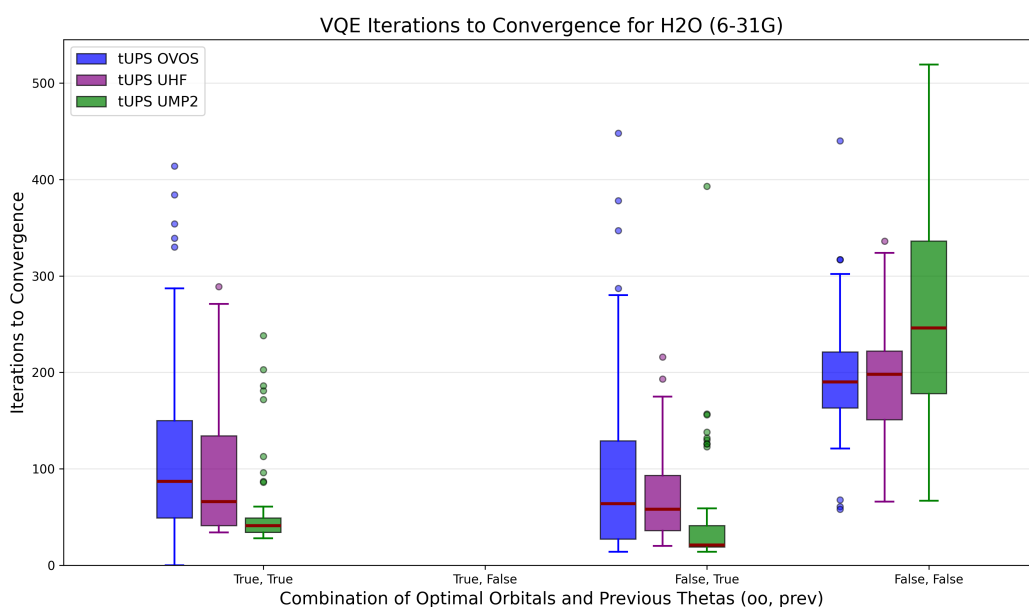


Figure 5.14: Convergence iteration count for H_2O in 6-31G basis set, (10e, 11o), using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The boxplots represent the distribution of convergence iteration counts across different geometries along the potential energy surface, with the red lines representing the mean convergence iteration count for each reference state.

HF

For HF, in 6-31G basis set, the ooVQE calculations show a worse spread of convergence iteration counts compared to the non orbital optimised VQE calculations when using the previous geometry final values as initialization. The best of the four combinations of orbital optimisation and initialization, in terms of convergence iteration count, is the non orbital optimised VQE calculations with the previous initialization of thetas, where the mean convergence iteration count is significantly lower than that for the other three combinations. For which we can observe that the OVOS optimised orbitals show a similar to lower mean convergence iteration count than the UHF orbitals and UMP2 natural orbitals, in three of the four combinations of orbital optimisation and initialization, with the non orbital optimised VQE calculations with the random initialization of thetas being the only combination where the OVOS optimised orbitals show a higher mean convergence iteration count than the UHF orbitals and UMP2 natural orbitals. Overall, there is a significant spread in the convergence iteration counts for OVOS orbitals, with some geometries calculations showing a very fast convergence, and others showing a much slower convergence. The potential energy surfaces for HF using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.15.

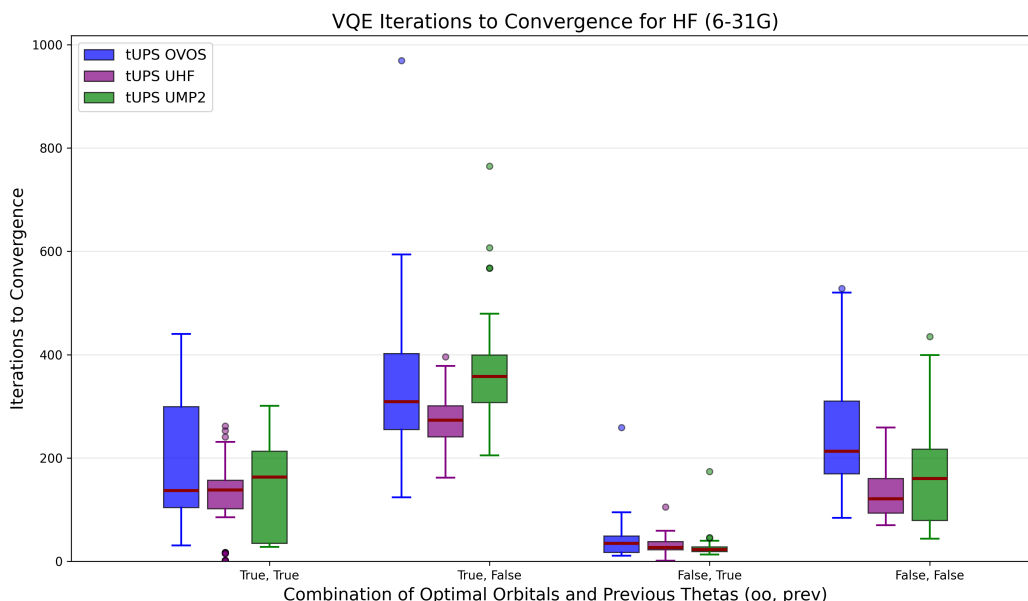


Figure 5.15: Convergence iteration count for HF in 6-31G basis set, (10e, 10o), using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The boxplots represent the distribution of convergence iteration counts across different geometries along the potential energy surface, with the red lines representing the mean convergence iteration count for each reference state.

Li₂

For Li₂, in 6-31G basis set, all three combinations of orbital optimisation and initialization show a similar small spread of convergence iteration counts, with the non orbital

optimised VQE calculations with the random initialization of thetas showing slightly more outliers than the other two combinations. The best of the three combinations of orbital optimisation and initialization, in terms of convergence iteration count, is the non orbital optimised VQE calculations with the previous initialization of thetas, where the mean convergence iteration count is a bit lower than that for the other two combinations. We observe that the OVOS optimised orbitals show a higher mean convergence iteration count than the UHF orbitals and UMP2 natural orbitals, in both combinations of orbital optimisation and initialization, with the non orbital optimised VQE calculations with the random initialization of thetas being the only combination where the OVOS optimised orbitals show a lower mean convergence iteration count than the UHF orbitals and UMP2 natural orbitals. Overall, there is a significant spread in the convergence iteration counts for OVOS orbitals, with some geometries calculations showing a very fast convergence, and others showing a much slower convergence. The potential energy surfaces for Li_2 using the OVOS optimised orbitals, UHF orbitals, and UMP2 natural orbitals as reference states are shown in Figure 5.16.

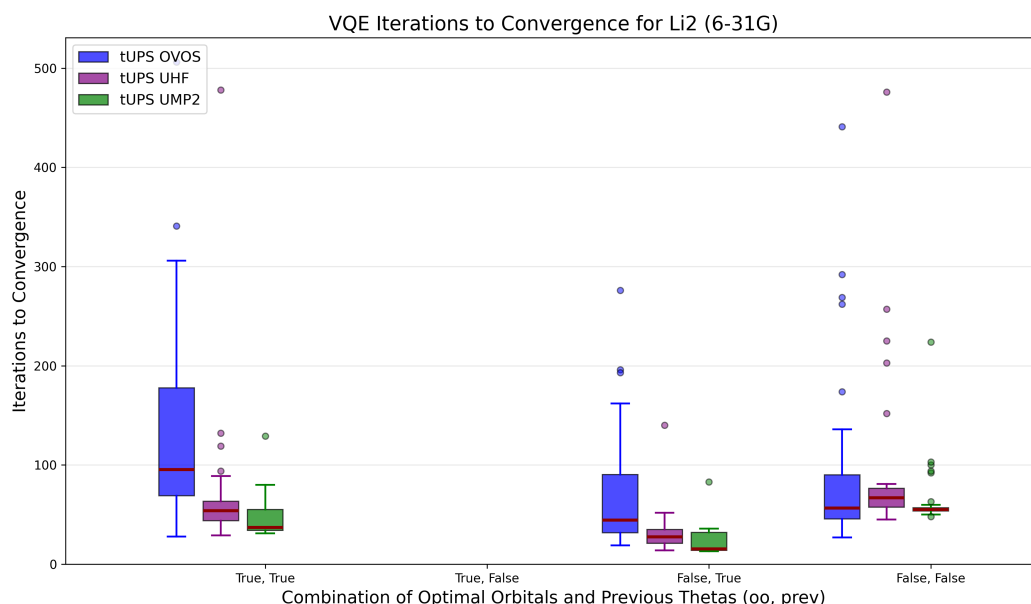


Figure 5.16: Convergence iteration count for Li_2 in 6-31G basis set, (6e, 15o), using the OVOS optimised orbitals (blue), UHF orbitals (purple), and UMP2 natural orbitals (green) as reference states in VQE calculations. The boxplots represent the distribution of convergence iteration counts across different geometries along the potential energy surface, with the red lines representing the mean convergence iteration count for each reference state.

Chapter 6

Conclusion and Outlook

In this chapter, we present what the OVOS implementation achieved, critically assess the limitations of the current work, and outline promising directions for future research in the area of optimised virtual orbitals and their applications in quantum computing.

6.1 Conclusion

In this thesis, we have implemented the Optimized Virtual Orbital Space (OVOS) method of Adamowicz & Bartlett [8] by coding it up in Python. The following conclusions can be drawn from the results presented in Chapter 5. Below we further discuss the implications of these conclusions for quantum computing applications.

6.1.1 Summary of Findings

This section summarises the main findings of this thesis in relation to the research objectives outlined in the Introduction 1. The implementation of the OVOS method was successfully validated against both PySCF and the original results of Adamowicz & Bartlett, demonstrating that our code correctly captures the expected energy recovery behaviour. The key quantitative finding is that with only a fraction of the virtual orbitals (e.g., $N'_{\text{virt}} \approx 50 - 60\%$ of N_{virt}), OVOS can recover approximately 90% of the MP2 correlation energy across all tested molecules and basis sets. Li_2 reached 90% recovery at only 28% of virtual orbitals, matching the efficiency seen in the original work. Going up in basis set size, for the same molecule, showed mostly an improvement in correlation energy for small numbers of N'_{virt} . This will translate well to even larger basis sets, it will promote further truncation of the space. This indicates that, even with a larger basis set, as we approach the full virtual space, the correlation energy does not improve only slightly.

The initialisation strategies for the OVOS optimisation mostly agreed, and convergence was generally robust. However, as the number of active virtual orbitals approaches the full virtual space, convergence becomes more prone to getting stuck in local minima. Here, it was generally observed across molecules that the different initialisation strategies (RHF, warm-start, random sampling) could lead to different convergence behaviours, with random sampling and RHF occasionally finding lower energy solutions than the warm-start. This suggests that while the OVOS method is generally robust, there may be cases where exploring multiple initialisations could yield better results, especially for larger active spaces. Thus, OVOS retains the essential physics while shrinking the virtual space by a factor of two.

The OVOS optimised orbitals as the VQE reference state consistently reached a lower energy before the energy kink appeared. Only for HF was it found that the converged energy was higher than that of the UHF orbitals. This suggests that the OVOS optimised orbitals generally provide a better reference state for VQE type

algorithms than the default UHF orbitals, consistent with the theoretical expectation that OVOS should yield orbitals with better overlap with the correlated ground state. However, the fact that, for HF, the OVOS optimised orbitals did not outperform the UHF orbitals indicates that there may be cases where OVOS does not provide a significant advantage, and further investigation into the factors influencing this behaviour would be valuable. It was found that UMP2 natural orbitals consistently had lower energy than the OVOS optimised orbitals, as expected, given that UMP2 natural orbitals provide a good approximation to the correlated ground state. Only for H₂O did we observe, at some geometries, a performance similar to that of the UMP2 natural orbitals.

The kinks, the drop in energy at certain geometries, were due to the optimisation crossing a barrier in the orbital rotation space, allowing it to find a lower energy solution. The OVOS optimised orbitals and UHF orbitals showed similar kinks for molecules H₂O and HF, while for Li₂ the OVOS optimised orbitals showed no clear kink. The UMP2 natural orbitals showed no kinks for any of the molecules, which is consistent with the fact that they are derived from a perturbative treatment of correlation and are not subject to the same optimisation landscape as the OVOS method. The presence of kinks in the OVOS optimised orbitals suggests that the optimisation landscape can be complex, with multiple local minima, and that the choice of initialisation can influence whether the optimisation finds the global minimum or gets stuck in a local minimum. This highlights the importance of exploring different initialisation strategies and potentially developing more robust optimisation algorithms to navigate this landscape effectively.

Observing the different configurations of VQE, the OVOS optimised orbitals seem to show a bigger spread in iteration count, while the UHF and UMP2 natural orbitals seem to show a more consistent iteration count across different configurations. This suggests that while OVOS optimised orbitals can provide a better reference state for VQE in some cases, their convergence behaviour may be more variable than that of UMP2 natural orbitals, with implications for the reliability of VQE calculations using OVOS orbitals. However, the mean iteration count shows that the OVOS optimised orbitals perform similarly to the UHF and UMP2 natural orbitals, with some configurations where they perform better and others where they perform worse. So despite OVOS optimised orbitals not beating the UMP2 natural orbitals in terms of energy convergence, it seems to perform similarly in terms of iteration count, which is an interesting result that suggests that OVOS optimised orbitals may still be a viable option for VQE reference states, especially considering the potential qubit savings they offer.

6.1.2 Critical Assessment

Several practical limitations became apparent during implementation and testing, and they directly affect how much one can trust the present results.

The OVOS method **implementation** was not one to one with the original implementation by Adamowicz & Bartlett. Here we took a different approach to their initial active space. More importantly, the lack of a direct comparison with their code and differences in computational details (e.g., integral evaluation, convergence criteria, ill-conditioning) mean that while the agreement in energy recovery behaviour is

reassuring, it does not guarantee that the same local minima are being found.

Despite the overall success of the OVOS method in recovering a significant fraction of the correlation energy with a reduced virtual space, we observed some limitations and caveats. The **convergence behaviour** was found to be oscillatory for larger active spaces, suggesting that the optimisation landscape may contain multiple local minima. This could be mitigated by implementing more robust optimisation techniques, such as trust-region methods or line-search algorithms, rather than relying solely on the Newton-Raphson method with level-shifting.

The use of a UHF reference for closed-shell molecules introduces **spin contamination**, potentially affecting the quality of the optimised orbitals. An RHF reference would be cleaner and may lead to more stable convergence for closed-shell systems. Additionally, the **block-diagonal approximation** of the Hessian used in the optimisation may introduce errors, especially in larger active spaces, where off-diagonal coupling between virtual orbitals can be significant.

The VQE calculations in SlowQuant were performed using a one step wavefunction optimisation, which may not fully capture the potential benefits of OVOS optimised orbitals. A more thorough **exploration of the VQE** optimisation on quantum simulators or hardware, including multiple optimisation steps and different ansatz choices, would provide a more comprehensive assessment of the advantages of OVOS orbitals for quantum algorithms. Furthermore, the comparison with UMP2 natural orbitals highlights that while OVOS can provide a better reference state than UHF orbitals, it may not always outperform more sophisticated orbital choices, and understanding the conditions under which OVOS provides the most benefit would be an important area for future investigation.

The combination of orbital-space optimisation and quantum algorithms represents a promising strategy for extending the reach of near-term quantum chemistry calculations beyond what either approach alone can achieve. This thesis provides a validated foundation for further exploration of OVOS assisted quantum computation, and the insights gained here will inform future developments in this exciting area of research. For availability of the code, please refer to the GitHub repository at https://github.com/Born-ship-it/optimized_virtual_orbitals.

6.2 Outlook

This section outlines promising directions for future research building on the results of this thesis. The OVOS method has demonstrated significant potential for reducing the virtual orbital space while retaining most of the correlation energy, but there are many avenues for improving the algorithm, extending the method to new contexts, and applying it to larger and more complex systems. Below we discuss these opportunities in detail.

6.2.1 Algorithmic Improvements

The current OVOS implementation serves as a proof-of-concept, but there are several algorithmic improvements that could enhance its numerical stability and computational efficiency. The selection of initial active orbitals is crucial for the success of the optimisation. It would be advantageous to explore different non-trivial approaches

to orbital selection. Additionally, augmenting the optimisation with trust-region or line-search techniques could help prevent overshooting in regions with large rotation, thereby improving robustness and convergence. Exploiting parallelism and GPU acceleration could significantly reduce the computational time, especially for larger basis sets.

6.2.2 Method Extensions

The OVOS framework developed in this thesis is formulated at the MP2 level within an unrestricted reference, but there are several promising extensions to higher levels of theory and alternative electronic structure contexts. Extending OVOS to CCSD or CCSD(T) would allow for a more accurate description of correlation effects, and could be achieved by minimising the CCSD energy with respect to orbital rotations. Implementing a restricted (RHF) formalism for closed-shell molecules would provide a cleaner approach that avoids UHF spin contamination and reduces the number of virtual orbitals by half.

6.2.3 Quantum Computing Applications

The truncation of the virtual orbital space by OVOS is directly relevant to quantum chemistry on near-term and fault-tolerant quantum devices. The reduction in the number of virtual orbitals translates into fewer qubits required to represent the electronic structure problem, as well as a potential reduction in circuit depth for variational algorithms like VQE. Running actual VQE calculations using OVOS orbitals on quantum simulators or hardware would provide concrete evidence of OVOS's benefits in a quantum computing context. If OVOS orbitals indeed approximate Brueckner orbitals, we would expect a significant reduction in T1 amplitudes, allowing us to use a UCCD ansatz instead of UCCSD and further reducing circuit depth. Quantifying the T1 reduction and benchmarking the performance of VQE with OVOS orbitals against traditional HF orbitals would be an important next step toward demonstrating the practical utility of OVOS for quantum chemistry on quantum devices.

6.2.4 Larger Molecular Systems

The OVOS method has been benchmarked on small molecules with up to 4 atoms, but extending the method to larger and more complex systems would demonstrate its practical utility for real-world applications in catalysis, drug discovery, and materials science. Transition-metal complexes, which often exhibit strong correlation effects, would be a particularly interesting test case for the limits of OVOS. Additionally, applying OVOS to biological systems such as amino acids and small peptides could provide insights into its performance in more realistic chemical environments. For periodic systems, implementing OVOS in the solid-state MP2 calculations would be a natural extension. Finally, combining OVOS truncation with basis-set extrapolation techniques to approach the complete basis set (CBS) limit could further enhance the accuracy of correlation energy estimates while keeping computational costs manageable.

Bibliography

- [1] Tobias B. Clausen. Theory of correlated optimized virtual orbitals (covo) and a preliminary implementation of the method. 2025. Preliminary PREP project, University of Copenhagen.
- [2] Attila Szabo and Neil S. Ostlund. *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*. Dover Publications, 1989. ISBN 978-0-486-69186-2.
- [3] C. David Sherrill and Henry F. Schaefer. The configuration interaction method: Advances in highly correlated approaches. In Per-Olov Löwdin, editor, *Advances in Quantum Chemistry*, volume 34, pages 143–269. Academic Press, 1999. doi: 10.1016/S0065-3276(08)60532-8.
- [4] Chr. Møller and M. S. Plesset. Note on an approximation treatment for many-electron systems. *Phys. Rev.*, 46:618–622, 1934. doi: 10.1103/PhysRev.46.618.
- [5] Trygve Helgaker, Poul Jørgensen, and Jeppe Olsen. *Molecular Electronic-Structure Theory*. Wiley, 2000. doi: 10.1002/9781119019572.
- [6] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O’Brien. A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.*, 5:4213, 2014. doi: 10.1038/ncomms5213.
- [7] Yudong Cao, Jonathan Romero, Jonathan P. Olson, Matthias Degroote, Peter D. Johnson, Maria Kieferova, Ian D. Kivlichan, Tim Menke, Borja Peropadre, Nicolas P. D. Sawaya, Sukin Sim, Libor Veis, and Alán Aspuru-Guzik. Quantum chemistry in the age of quantum computing. *Chem. Rev.*, 119:10856–10915, 2019. doi: 10.1021/acs.chemrev.8b00803.
- [8] L. Adamowicz and R. J. Bartlett. Optimized virtual orbital space for high-level correlated calculations. *J. Chem. Phys.*, 86(11):6314–6324, 1987. doi: 10.1063/1.452468.
- [9] Per-Olov Löwdin. Quantum theory of many-particle systems. I. physical interpretations by means of density matrices, natural spin-orbitals, and convergence problems in the method of configurational interaction. *Phys. Rev.*, 97:1474–1489, 1955. doi: 10.1103/PhysRev.97.1474.
- [10] Andrew G. Taube and Rodney J. Bartlett. Frozen natural orbitals: Systematic basis set truncation for coupled-cluster theory. *Collect. Czech. Chem. Commun.*, 70:837–850, 2005. doi: 10.1135/cccc20050837.
- [11] Qiming Sun, Xing Zhang, Samraghi Banerjee, Peng Bao, Marc Barbry, Nick S. Blunt, Nikolay A. Bogdanov, George H. Booth, Jia Chen, Zhi-Hao Cui, Janus Juul Eriksen, Yang Gao, Sheng Guo, Jan Hermann, Matthew R. Hermes, Kevin Koh, Peter Koval, Susi Lehtola, Zhendong Li, Junzi Liu, Narbe Mardirossian,

- James D. McClain, Mario Motta, Bastien Mussard, Hong-Zhou Pham, Artem Pulkin, Wirawan Purwanto, Paul J. Robinson, Enrico Ronca, Elvira R. Sayfutyarova, Maximilian Scheurer, Henry F. Schurkus, James E. T. Smith, Chong Sun, Shi-Ning Sun, Shiv Upadhyay, Lucas K. Wagner, Xiao Wang, Alec White, James Daniel Whitfield, Mark J. Williamson, Sebastian Wouters, Jun Yang, Jason M. Yu, Tianyu Zhu, Timothy C. Berkelbach, Sandeep Sharma, Alexander Yu. Sokolov, and Garnet Kin-Lic Chan. Recent developments in the PySCF program package. *J. Chem. Phys.*, 153:024109, 2020. doi: 10.1063/5.0006074.
- [12] Jacob T. Seeley, Martin J. Richard, and Peter J. Love. The Bravyi–Kitaev transformation for quantum computation of electronic structure. *J. Chem. Phys.*, 137:224109, 2012. doi: 10.1063/1.4768229.
- [13] Joonho Lee and Martin Head-Gordon. Regularized orbital-optimized second-order Møller–Plesset perturbation theory: A reliable fifth-order scaling electron correlation model with orbital energy dependent regularizers. *J. Chem. Theory Comput.*, 14:5203–5219, 2018. doi: 10.1021/acs.jctc.8b00731. arXiv:1807.06185.
- [14] Frank Jensen. *Introduction to Computational Chemistry*. Wiley, 3rd edition, 2017. ISBN 978-1-118-82599-0.
- [15] R. Ditchfield, W. J. Hehre, and J. A. Pople. Self-consistent molecular-orbital methods. ix. an extended gaussian-type basis for molecular-orbital studies of organic molecules. *J. Chem. Phys.*, 54:724–728, 1971. doi: 10.1063/1.1674902.
- [16] W. J. Hehre, R. Ditchfield, and J. A. Pople. Self-consistent molecular orbital methods. xii. further extensions of gaussian-type basis sets for use in molecular orbital studies of organic molecules. *J. Chem. Phys.*, 56:2257–2261, 1972. doi: 10.1063/1.1677527.
- [17] James D. Dill and John A. Pople. Self-consistent molecular orbital methods. xv. extended gaussian-type basis sets for lithium, beryllium, and boron. *J. Chem. Phys.*, 62:2921–2923, 1975. doi: 10.1063/1.430801.
- [18] Thom H. Dunning. Gaussian basis sets for use in correlated molecular calculations. I. the atoms boron through neon and hydrogen. *J. Chem. Phys.*, 90:1007–1023, 1989. doi: 10.1063/1.456153.
- [19] Nicholas C. Handy, John A. Pople, Martin Head-Gordon, Krishnan Raghavachari, and Gary W. Trucks. Size-consistent Brueckner theory limited to single and double substitutions. *Chem. Phys. Lett.*, 164:185–192, 1989. doi: 10.1016/0009-2614(89)85013-4.
- [20] Keith A. Brueckner. Two-body forces and nuclear saturation. III. details of the structure of the nucleus. *Phys. Rev.*, 97:1353–1366, 1954. doi: 10.1103/PhysRev.97.1353.
- [21] George D. Purvis and Rodney J. Bartlett. A full coupled-cluster singles and doubles model: The inclusion of disconnected triples. *J. Chem. Phys.*, 76:1910–1918, 1982. doi: 10.1063/1.443164.

- [22] E. Kjellgren and K. M Ziems. Slowquant, 2026.
- [23] Benjamin P. Pritchard, Doaa Altarawy, Brett Didier, Tara D. Gibbsom, and Theresa L. Windus. A new basis set exchange: An open, up-to-date resource for the molecular sciences community. *J. Chem. Inf. Model.*, 59:4814–4820, 2019. doi: 10.1021/acs.jcim.9b00725.

Appendix A

Implementation Validation

PySCF gives me an opportunity to validate my implementation of full space MP2 by comparing with their built-in MP2 module, `pyscf.mp.MP2`. I present the deviation $\Delta E_{\text{corr}} = E_{\text{corr}}^{\text{code}} - E_{\text{corr}}^{\text{PySCF}}$ is within numerical precision for all test molecules in cc-pVDZ, see Figure A.1.

Molecule	$E_{\text{corr}}^{\text{OVOS}}$ (Hartree)	$E_{\text{corr}}^{\text{PySCF}}$ (Hartree)	ΔE_{corr} (Hartree)
CO	-2.90778E-01	-2.90778E-01	4.25617E-8
H ₂ O	-2.04003E-01	-2.04003E-01	2.29192E-9
HF	-2.03781E-01	-2.03781E-01	1.23463E-10
NH ₃	-1.88825E-01	-1.88825E-01	8.31300E-10
Li ₂	-2.00076E-02	-2.00076E-02	1.11200E-9

Table A.1: Validation of the OVOS implementation against PySCF mp.MP2. Deviations ΔE are within numerical precision ($|\delta E| < \sim 10^{-8}$ Hartree). All molecules are of the cc-pVDZ basis for this validation.

It is observed that CO's deviation is barely at acceptable range of 10^{-8} Hartree, on the other hand, the other molecules show deviations below the acceptable range, which confirms that my implementation of the MP2 correlation energy is correct, and that the OVOS code correctly reproduces the full-space MP2 limit when $N'_{\text{act.unocc}} = N_{\text{act.unocc}}$.

Verifying that the full-space MP2 energies match PySCF's results, any differences observed in the OVOS results are due to the orbital optimisation and active space truncation, rather than a bug in the MP2 implementation.

Appendix B

Additional Data and Figures

B.1 Full MP2 Reference Energies

Here we include the MP2 correlation energy OVOS plots for the smaller 6–31G basis set.

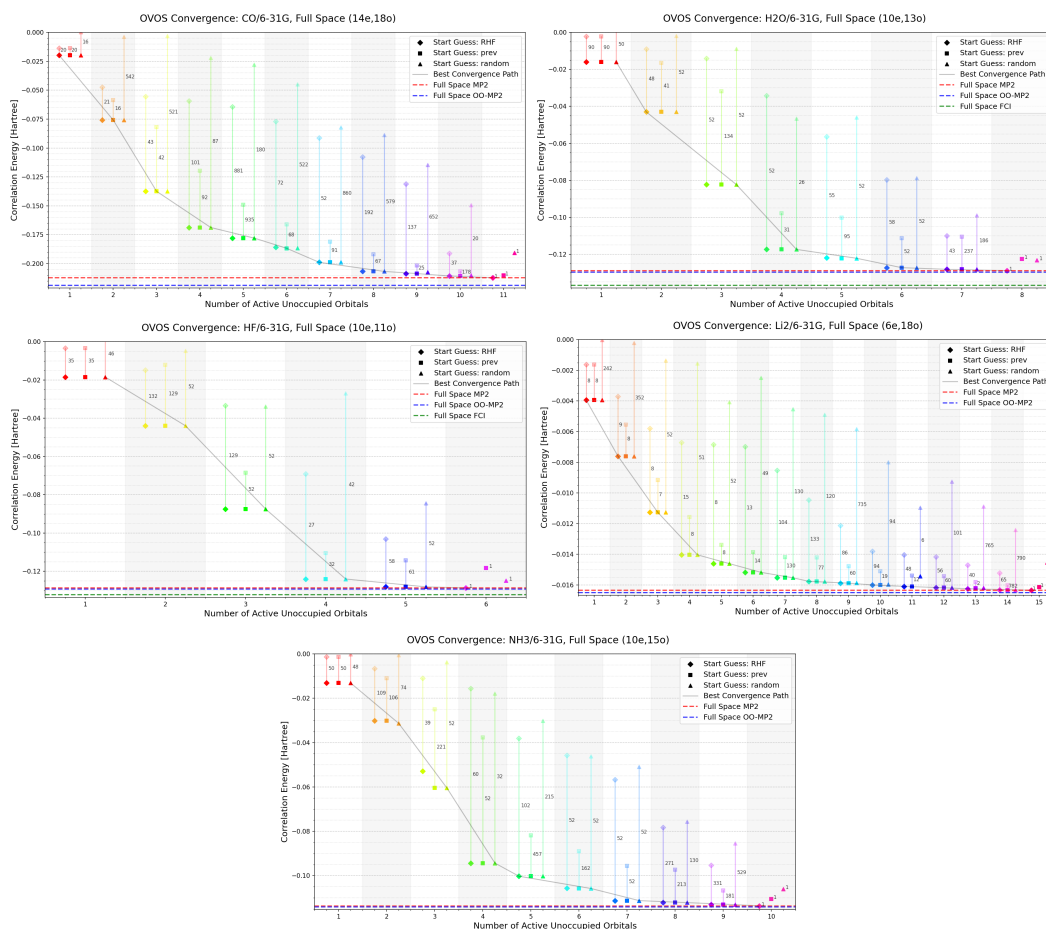


Figure B.1: Convergence of the MP2 correlation energy for CO, H₂, HF, NH₃, and Li₂ in 6–31G basis, at each number of active unoccupied orbitals $N'_{act.unocc}$. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed red line marks the MP2 correlation energy at full space, PySCF reference. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references. A gray line is drawn to connect the best points of the start guesses. A line for each OVOS connects initial and final correlation energies, at which the number of iterations to convergence is annotated.

B.2 Convergence Plots for All Systems

Here we include the convergence plots for all systems, which show the energy recovery as a function of iteration number for each (molecule, basis) combination. These plots provide insight into the convergence behaviour of the OVOS algorithm across different systems and basis sets.

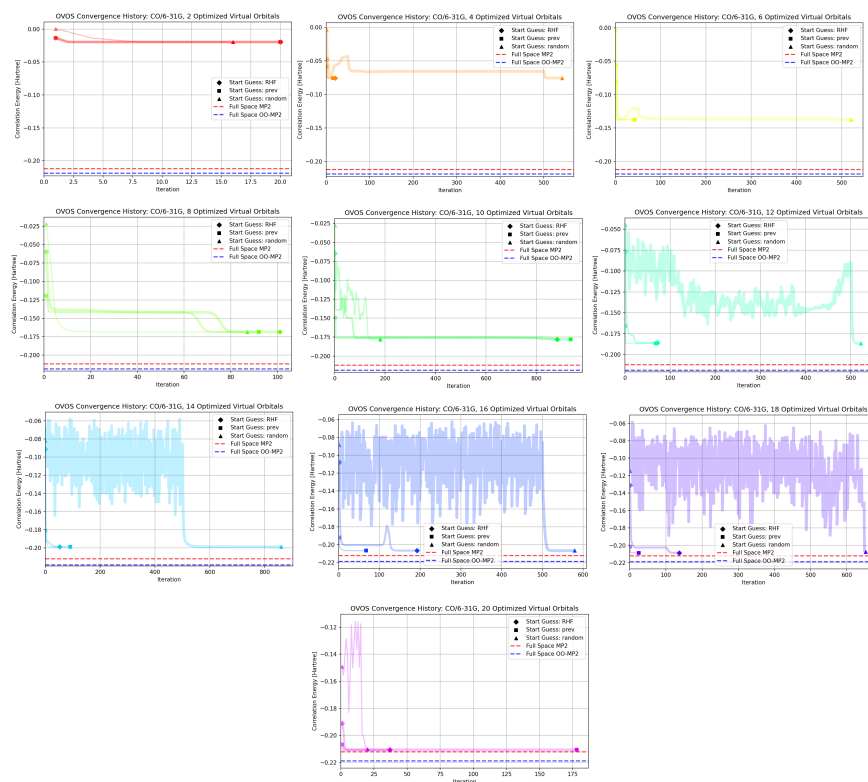


Figure B.2: CO/6–31G OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

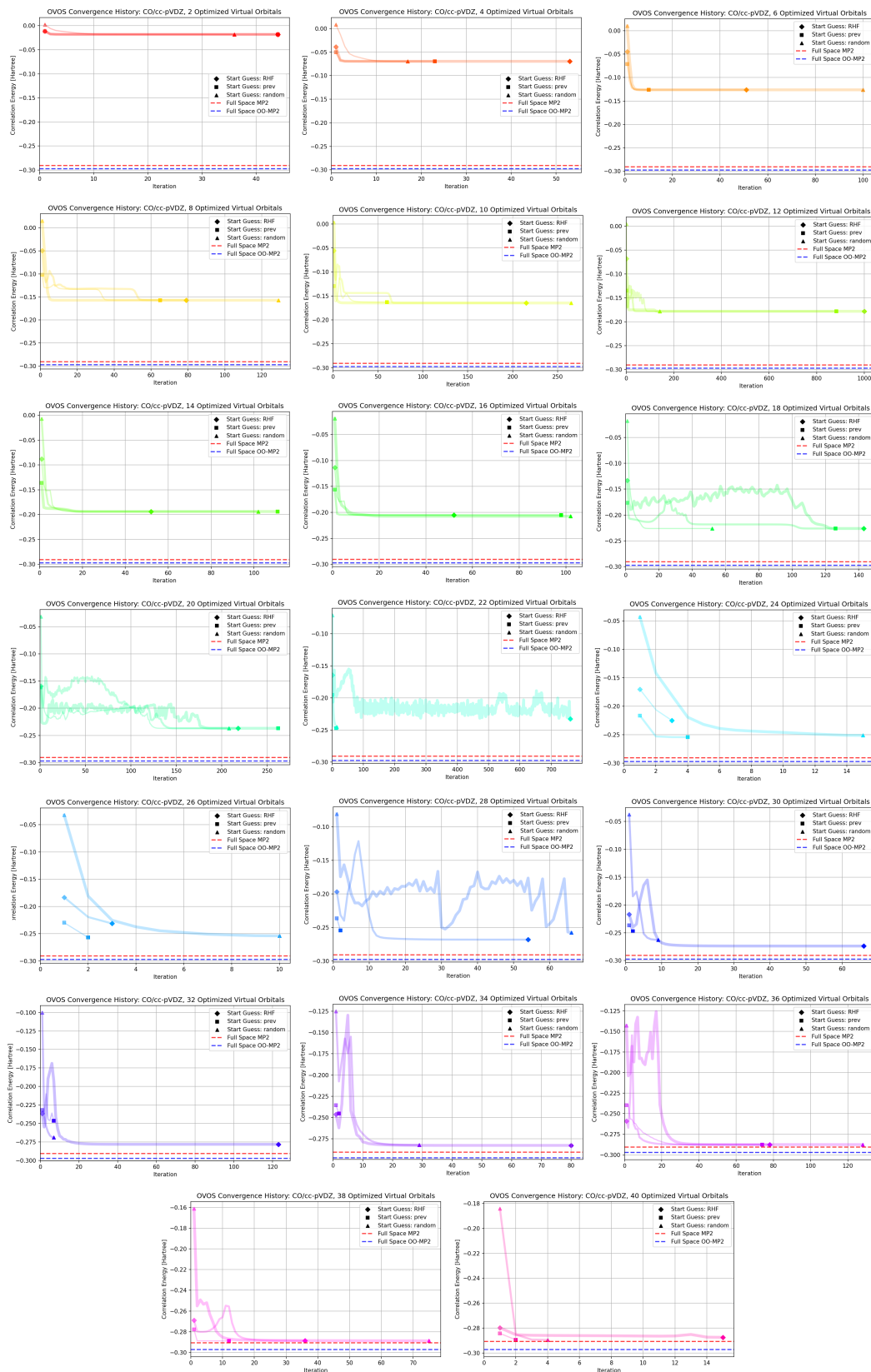


Figure B.3: CO/cc-pVDZ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

B.2. CONVERGENCE PLOTS FOR ALL SYSTEMS

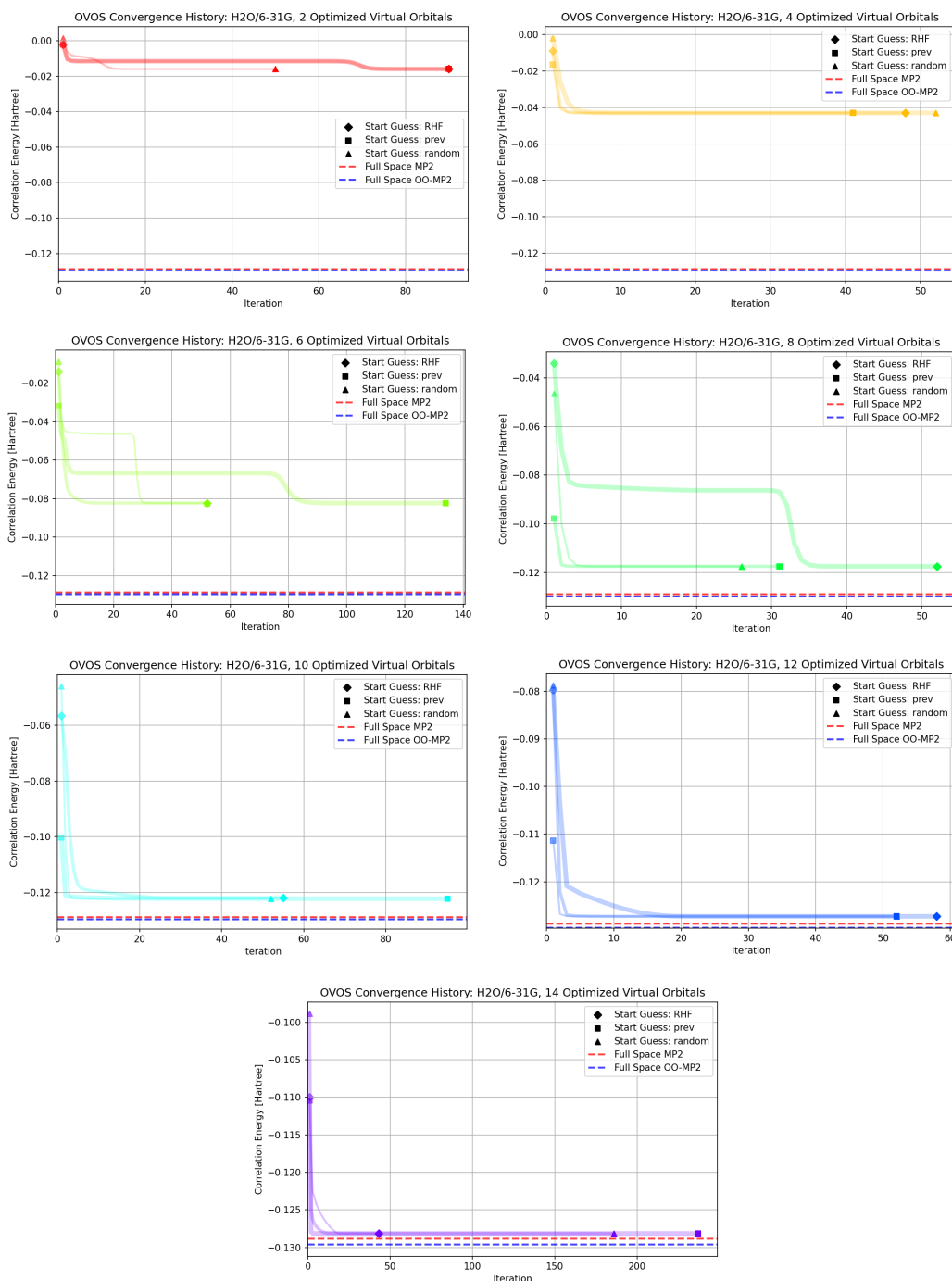


Figure B.4: H₂O/6-31G OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

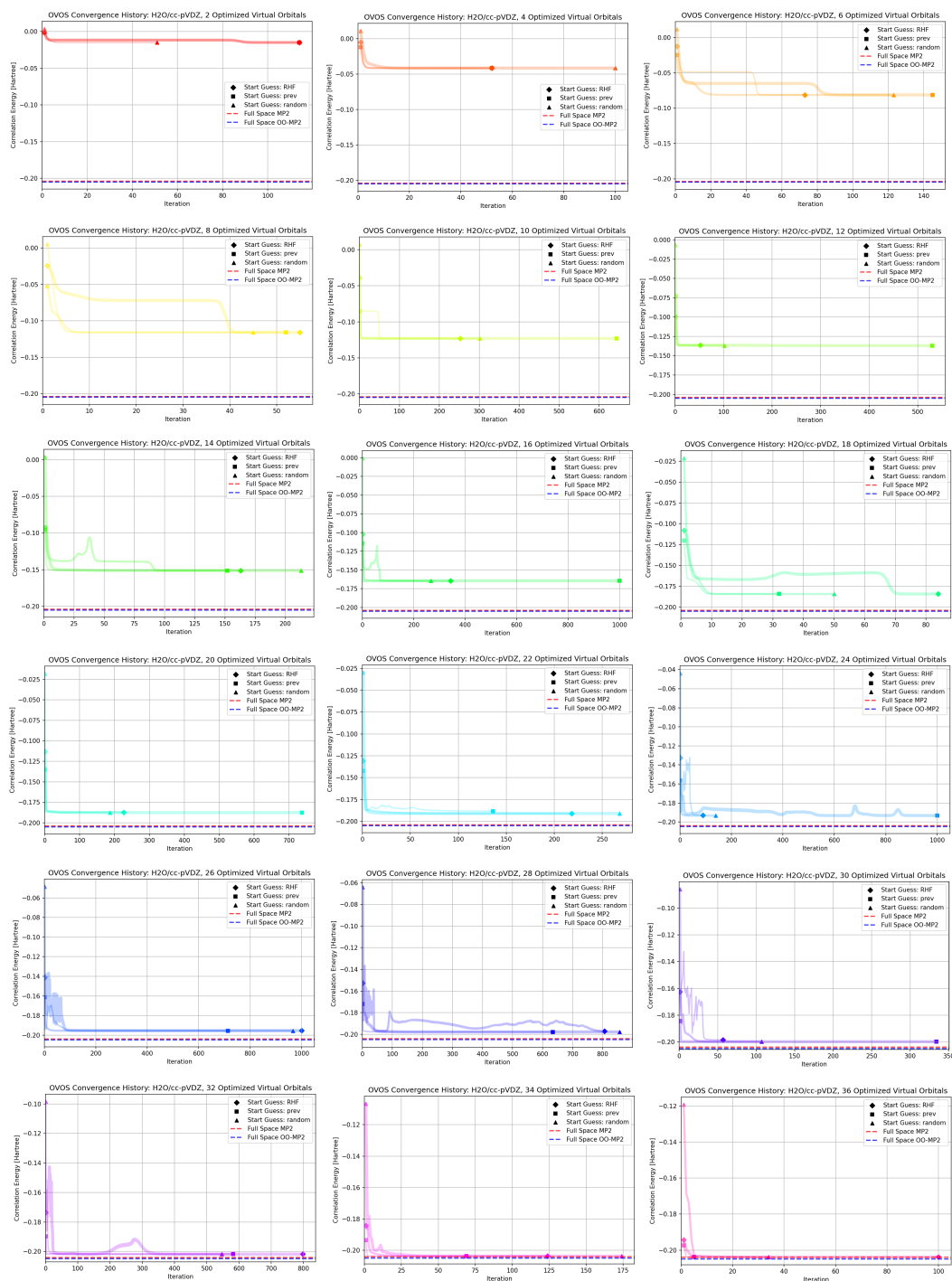


Figure B.5: $\text{H}_2\text{O}/\text{cc-pVDZ}$ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

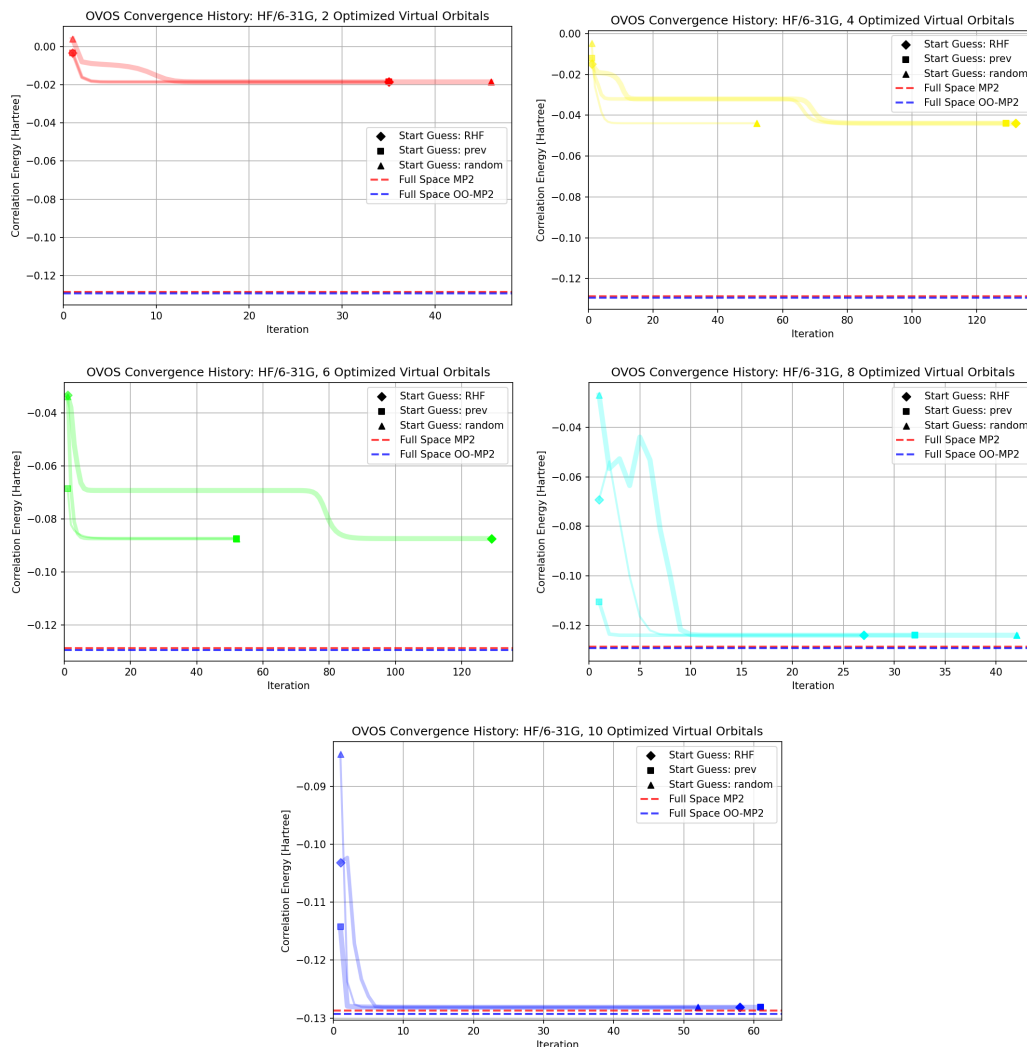


Figure B.6: HF/6–31G OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.



Figure B.7: HF/cc-pVDZ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

B.2. CONVERGENCE PLOTS FOR ALL SYSTEMS



Figure B.8: $\text{Li}_2/6\text{--}31\text{G}$ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

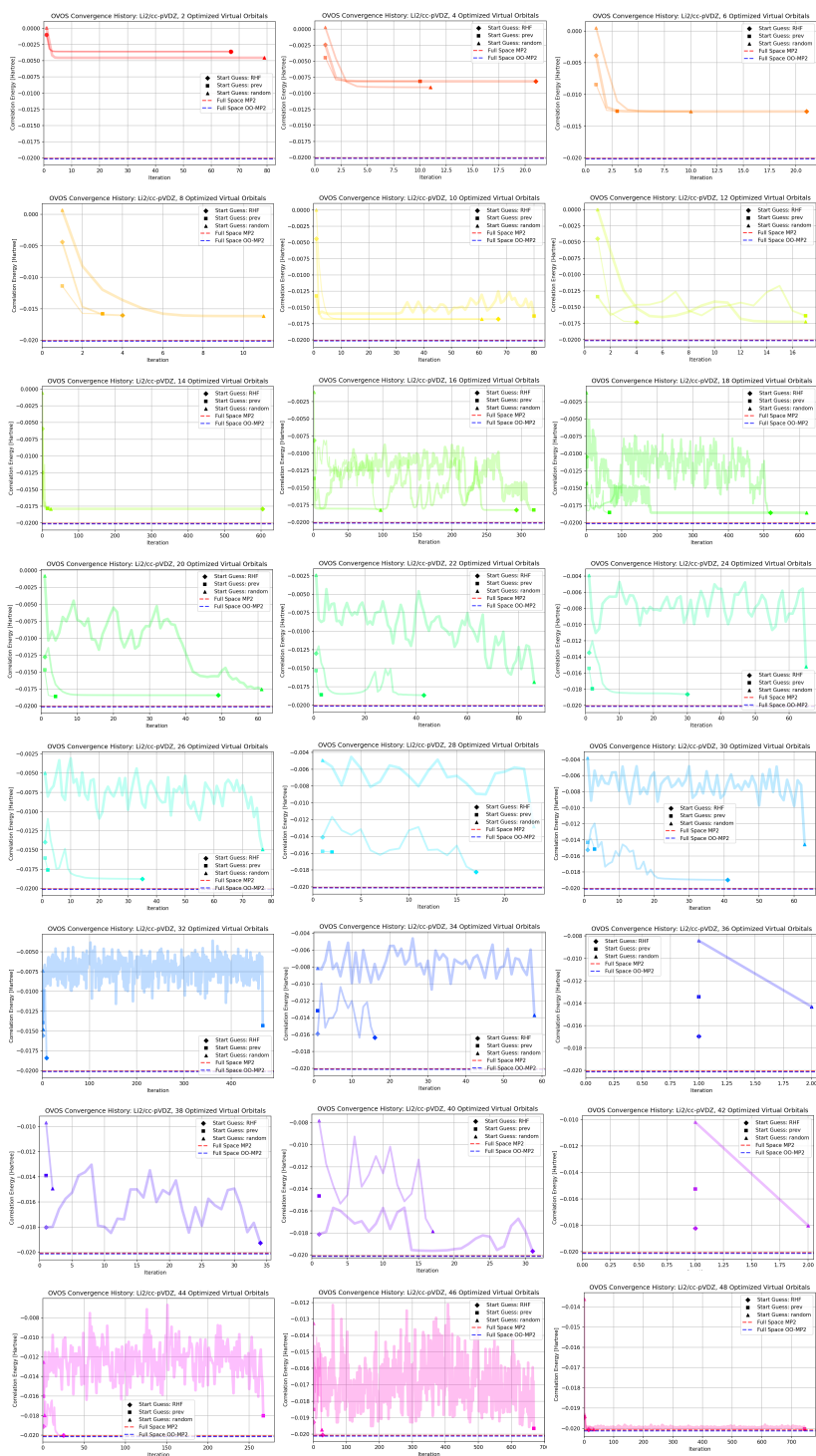


Figure B.9: $\text{Li}_2/\text{cc-pVDZ}$ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

B.2. CONVERGENCE PLOTS FOR ALL SYSTEMS

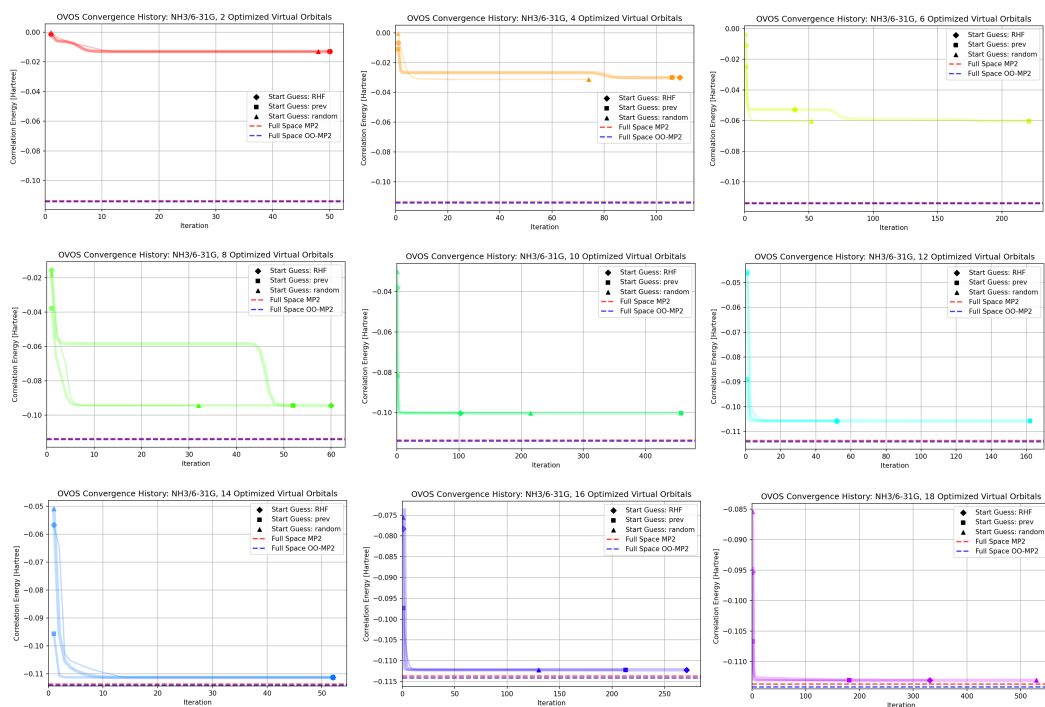


Figure B.10: NH₃/6-31G OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

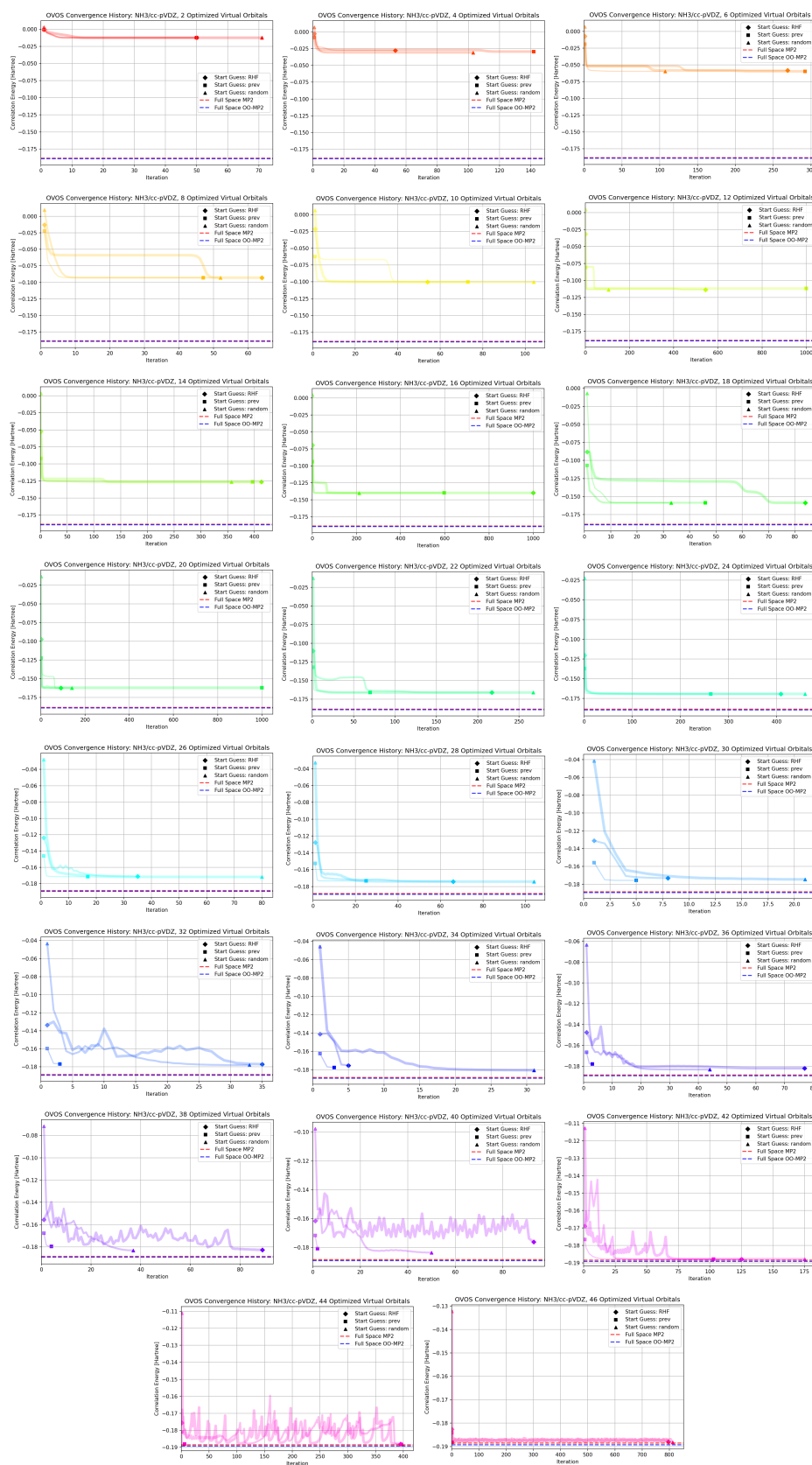


Figure B.11: $\text{NH}_3/\text{cc-pVDZ}$ OVOS convergence history for each active unoccupied orbitals, colour coded to fit the OVOS convergence plot in Figure B.1 and in Section 5.1.1. The OVOS algorithm takes RHF (diamond), previous (square), and random (triangle) as start guesses. Dashed lines: full-space MP2 (red), OO-MP2 (blue), FCI (green) references.

B.3 OVOS Figures

Here we include illustrative figures of the OVOS loop structure, which provide a visual representation of the algorithm's workflow.

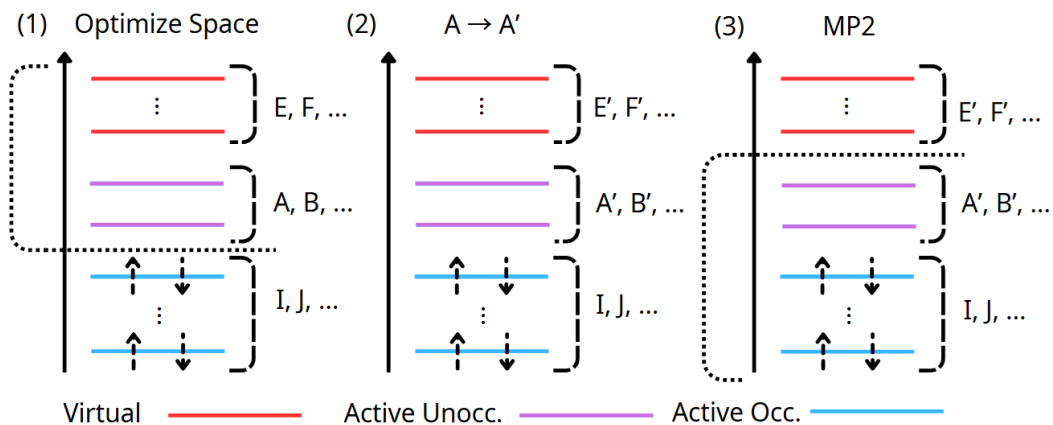


Figure B.12: Illustration of the OVOS loop structure, showing the flow of the algorithm's loop, the optimization of virtual orbitals.

Appendix C

Implementation Details and Source Code

C.1 Repository and Version Information

Here we give the full code implementation for OVOS in Python, also see the GitHub repository at

Code C.1: ...

```
1 """
2 OVOS (Optimised Virtual Orbital Space) method
3
4 Minimises the second-order correlation energy (MP2) by rotating
  virtual orbitals.
5
6 Spaces:
7     Active occupied    (I,J): "Inactive"
8     Active virtual     (A,B): "Active"
9     Inactive virtual   (E,F): "Virtual"
10
11 Reference:
12     L. Adamowicz & R. J. Bartlett (1987), J. Chem. Phys. 86,
13     6314
14     https://pubs.aip.org/aip/jcp/article/86/11/6314/93345
15 """
16
17 import os
18 from typing import List, Optional, Tuple
19 import numpy as np
20 import scipy.linalg
21 import pyscf
22 from pyscf import ao2mo
23 # Module metadata
24 import time
25 _LOAD_TIME = time.time()
26 _MODULE_VERSION = "dev_build"
27 # Limit OpenBLAS threads to avoid oversubscription in parallel
  runs
28 os.environ['NUMBA_THREADING_LAYER'] = 'omp' # Use OpenMP (
  thread-safe)
29 os.environ['NUMBA_NUM_THREADS'] = '1'
30 os.environ['OPENBLAS_NUM_THREADS'] = '1'
31 os.environ['OMP_NUM_THREADS'] = '1'
32 # Set printoptions
33 np.set_printoptions(precision=4, suppress=False, linewidth=200)
```

```

34 class OVOS:
35     """
36     Minimises the MP2 correlation energy via orbital rotations
      (OVOS algorithm).
37
38     Parameters
39     -----
40     mol : pyscf.gto.Mole
41     scf : RHF or UHF object
42     Fao : list of np.ndarray
43           AO Fock matrices [alpha, beta].
44     num_opt_virtual_orbs : int
45           Number of optimised virtual spin orbitals (active
      virtuals).
46     mo_coeff : list of np.ndarray
47           Initial MO coefficients [alpha, beta].
48     init_orbs : str, optional
49           'RHF' or 'UHF' (unused, kept for compatibility).
50     verbose : int, optional
51           Verbosity level. 0 = silent, 1 = normal output.
52     max_iter : int, optional
53           Maximum number of OVOS iterations.
54     conv_energy : float, optional
55           Convergence threshold on absolute change in MP2 energy.
56     conv_grad : float, optional
57           Convergence threshold on gradient norm.
58     keep_track_max : int, optional
59           Number of consecutive converged iterations required to
      stop.
60     lambda_init : float, optional
61           Initial Levenberg Marquardt damping parameter.
62     lambda_max : float, optional
63           Maximum allowed damping parameter.
64     trust_radius : float, optional
65           Initial trust radius for step size control.
66     hessian_reg : float, optional
67           Regularisation added to Hessian diagonal when singular.
68     """
69
70     def __init__(self, mol, scf, Fao, num_opt_virtual_orbs,
71                 mo_coeff,
72                 init_orbs="RHF", verbose=1,
73                 max_iter=1000, conv_energy=1e-8, conv_grad=1e-4,
74                 keep_track_max=50,
75                 lambda_init=1e-3, lambda_max=1e3,
76                 trust_radius=5, hessian_reg=1e-8):
77
78         self.verbose = verbose
79         self.mol = mol

```

```

79     self.num_opt_virtual_orbs = num_opt_virtual_orbs
80     self.init_orbs = init_orbs
81     self.mo_coeffs = mo_coeff
82     self.Fao = Fao
83
84     self.scf = scf
85     self.e_rhf = scf.e_tot
86     self.h_nuc = mol.energy_nuc()
87
88     # Convergence parameters
89     self.max_iter = max_iter
90     self.conv_energy = conv_energy
91     self.conv_grad = conv_grad
92     self.t1_conv = conv_grad
93     self.keep_track_max = keep_track_max
94
95     # Optimization parameters
96     self.lambda_init = lambda_init
97     self.lambda_max = lambda_max
98     self.trust_radius = trust_radius
99     self.hessian_reg = hessian_reg
100
101     # AO integrals
102     self.S = mol.intor('int1e_ovlp')
103     self.hcore_ao = mol.intor("int1e_kin") + mol.intor("
        int1e_nuc")
104     self.eri_4fold_ao = mol.intor('int2e_sph', aosym=1)
105
106     # Orbital counts
107     n_spatial = self.mo_coeffs[0].shape[0]
108     self.tot_num_spin_orbs = 2 * n_spatial
109     assert self.mo_coeffs[0].shape[0] == self.mo_coeffs[1].
        shape[0], \
110         "Alpha and beta MO coeff matrices must have same
        number of rows"
111
112     self.nelec = mol.nelec[0] + mol.nelec[1]
113
114     # Index lists for subspaces
115         # Occupied: 0 to nelec-1 (active)
116     self.active_occ_indices = list(range(self.nelec))
117         # Active virtual: next num_opt_virtual_orbs after
        occupied
118     self.active_inocc_indices = list(range(
119         self.active_occ_indices[-1] + 1,
120         self.nelec + self.num_opt_virtual_orbs
121     ))
122         # Inactive virtual: the rest of the virtual
        orbitals
123     self.virtual_inocc_indices = list(range(

```

```

124         self.active_inocc_indices[-1] + 1,
125         self.tot_num_spin_orbs
126     ))
127     self.inactive_indices = self.virtual_inocc_indices
128     self.full_indices = list(range(self.tot_num_spin_orbs))
129
130     assert self.tot_num_spin_orbs >= self.
131         num_opt_virtual_orbs + self.nelec, \
132         f"'num_opt_virtual_orbs' ({num_opt_virtual_orbs})
133         too large for basis set"
134
135     if self.verbose:
136         self._print()
137         self._print("#### Active and inactive spaces ####")
138         self._print("Total number of spin-orbitals:  ",
139                     self.tot_num_spin_orbs)
140         self._print("Active occupied spin-orbitals:  ",
141                     self.active_occ_indices)
142         self._print("Active unoccupied spin-orbitals: ",
143                     self.active_inocc_indices)
144         self._print("Inactive unoccupied spin-orbs:  ",
145                     self.inactive_indices)
146         self._print()
147
148     #
149     -----
150
151     # Print helper
152     #
153     -----
154
155     def _print(self, *args, **kwargs):
156         """Print with CRLF line endings and immediate flush."""
157         kwargs.setdefault('end', '\r\n')
158         kwargs.setdefault('flush', True)
159         print(*args, **kwargs)
160
161     #
162     -----
163
164     # Spin orbital conversion helpers
165     #
166     -----
167
168     def _spatial_to_spin_energies(self, mo_energy_a: np.ndarray
169     ,
170                                   mo_energy_b: np.ndarray) ->
171                                   np.ndarray:
172         """Interleave alpha/beta spatial MO energies into spin
173         orbital order."""

```



```

157     eps = np.empty(2 * len(mo_energy_a))
158     eps[0::2] = mo_energy_a
159     eps[1::2] = mo_energy_b
160     return eps
161
162 def _build_spin_fock(self, Fmo_a: np.ndarray, Fmo_b: np.
163 ndarray) -> np.ndarray:
164     """Build block diagonal spin orbital Fock matrix (even
165     alpha, odd beta)."""
166     n_spin = 2 * self.Fao[0].shape[0]
167     Fspin = np.zeros((n_spin, n_spin))
168     Fspin[0::2, 0::2] = Fmo_a
169     Fspin[1::2, 1::2] = Fmo_b
170     return Fspin
171
172 #
173 -----
174
175 # Integral transformation
176 #
177 -----
178
179 def _eri_vovo_antisym(self, mo_coeffs: List[np.ndarray]) ->
180 np.ndarray:
181     """
182     Transform (vir,occ|vir,occ) AO integrals to MO basis,
183     assemble spin orbital
184     and antisymmetrise. Returns <ab||ij> in physicist
185     notation (a,b,i,j).
186     """
187     nocc_a, nocc_b = self.mol.nelec
188     C_occ_a = mo_coeffs[0][:, :nocc_a]
189     C_occ_b = mo_coeffs[1][:, :nocc_b]
190     C_vir_a = mo_coeffs[0][:, nocc_a:]
191     C_vir_b = mo_coeffs[1][:, nocc_b:]
192     nvir_a = C_vir_a.shape[1]
193     nvir_b = C_vir_b.shape[1]
194
195     def _ao2mo(C_left, C_row, C_right, C_col):
196         # returns (left,row/right,col) in chemist notation
197         return ao2mo.kernel(self.eri_4fold_ao,
198                             [C_left, C_row, C_right, C_col
199                             ],
200                             compact=False)
201
202     # Chemist blocks (vir,occ|vir,occ)
203     vovo_aaaa = _ao2mo(C_vir_a, C_occ_a, C_vir_a, C_occ_a).
204         reshape(
205             nvir_a, nocc_a, nvir_a, nocc_a)
206     vovo_bbbb = _ao2mo(C_vir_b, C_occ_b, C_vir_b, C_occ_b).

```

```

196         reshape(
197             nvir_b, nocc_b, nvir_b, nocc_b)
198     vovo_aabb = _ao2mo(C_vir_a, C_occ_a, C_vir_b, C_occ_b).
199         reshape(
200             nvir_a, nocc_a, nvir_b, nocc_b)
201
202     nvir_spin = 2 * nvir_a
203     nocc_spin = nocc_a + nocc_b
204
205     # Assemble spin orbital chemist tensor
206     vovo_spin = np.zeros((nvir_spin, nocc_spin, nvir_spin,
207         nocc_spin))
208     vovo_spin[0::2, 0::2, 0::2, 0::2] = vovo_aaaa
209     vovo_spin[1::2, 1::2, 1::2, 1::2] = vovo_bbbb
210     vovo_spin[0::2, 0::2, 1::2, 1::2] = vovo_aabb
211     vovo_spin[1::2, 1::2, 0::2, 0::2] = vovo_aabb.transpose
212         (2, 3, 0, 1)
213
214     # Convert to physicist (a,i,b,j) -> (a,b,i,j) and
215     antisymmetrize
216     eri_phys = vovo_spin.transpose(0, 2, 1, 3)
217     eri_as = eri_phys - eri_phys.transpose(0, 1, 3, 2)
218     return eri_as
219
220 #
221 -----
222
223 # MP2 energy and amplitude computation
224 #
225 -----
226
227 def _mp1_amplitudes(self, fock_diag: np.ndarray,
228     eri_as: np.ndarray) -> np.ndarray:
229     """
230     Compute MP1 amplitudes  $t_{ij}^{ab} = -\langle ab || ij \rangle / (f_{aa} +$ 
231          $f_{bb} - f_{ii} - f_{jj})$ .
232     Only the active occupied and active virtual subspaces
233         are used.
234     """
235     occ_idx = self.active_occ_indices
236     vir_idx = self.active_inocc_indices
237     nvir_act = len(vir_idx)
238
239     eps_occ = fock_diag[occ_idx]
240     eps_vir = fock_diag[vir_idx]
241
242     denom = (eps_vir[:, None, None, None] +
243         eps_vir[None, :, None, None] -
244         eps_occ[None, None, :, None] -
245         eps_occ[None, None, None, :])

```

```

235         return -eri_as[:nvir_act, :nvir_act, :, :] / denom
236
237
238 def _mp2_energy(self, fock_spin: np.ndarray, t_abij: np.
239 ndarray,
240               eri_as: np.ndarray) -> float:
241     """
242     MP2 correlation energy using full virtual virtual Fock
243     block to handle
244     non canonical orbitals correctly.
245     """
246     occ_abs = np.array(self.active_occ_indices)
247     vir_abs = np.array(self.active_inocc_indices)
248     nocc = len(occ_abs)
249     nvir = len(vir_abs)
250
251     if nocc == 0 or nvir == 0:
252         return 0.0
253
254     a_loc, b_loc = np.triu_indices(nvir, k=1)          # a
255     < b
256     F_virt = fock_spin[np.ix_(vir_abs, vir_abs)]
257     chunk_size = min(500, len(a_loc))
258     n_pairs = len(a_loc)
259
260     # Pre compute all (i,j) with i > j
261     occ_pairs = [(i, j) for i in range(nocc) for j in range
262 (i)]
263
264     energy = 0.0
265     for loc_i, loc_j in occ_pairs:
266         abs_i = occ_abs[loc_i]
267         abs_j = occ_abs[loc_j]
268         eps_ij_sum = fock_spin[abs_i, abs_i] + fock_spin[
269             abs_j, abs_j]
270
271         t_ab = t_abij[a_loc, b_loc, loc_i, loc_j]      #
272             shape (n_pairs,)
273         g_ab = eri_as[a_loc, b_loc, loc_i, loc_j]      #
274             shape (n_pairs,)
275
276         # Diagonal part (2 * t * g)
277         term2 = 2.0 * np.dot(t_ab, g_ab)
278
279         # Off diagonal part
280         term1 = 0.0
281         for r0 in range(0, n_pairs, chunk_size):
282             r1 = min(r0 + chunk_size, n_pairs)
283             t_row = t_ab[r0:r1, None]
284             ar, br = a_loc[r0:r1], b_loc[r0:r1]

```

```

278
279         for c0 in range(0, n_pairs, chunk_size):
280             c1 = min(c0 + chunk_size, n_pairs)
281             t_col = t_ab[None, c0:c1]
282             ac, bc = a_loc[c0:c1], b_loc[c0:c1]
283
284             # Masks for index equalities
285             d_ac = ar[:, None] == ac[None, :]
286             d_bd = br[:, None] == bc[None, :]
287             d_ad = ar[:, None] == bc[None, :]
288             d_bc = br[:, None] == ac[None, :]
289
290             bracket = np.zeros((r1 - r0, c1 - c0))
291             if np.any(d_bd):
292                 bracket += F_virt[ar[:, None], ac[None, :]] * d_bd
293             if np.any(d_ac):
294                 bracket += F_virt[br[:, None], bc[None, :]] * d_ac
295             if np.any(d_bc):
296                 bracket -= F_virt[ar[:, None], bc[None, :]] * d_bc
297             if np.any(d_ad):
298                 bracket -= F_virt[br[:, None], ac[None, :]] * d_ad
299
300             mask_ac_bd = d_ac & d_bd
301             if np.any(mask_ac_bd):
302                 bracket[mask_ac_bd] -= eps_ij_sum
303             mask_ad_bc = d_ad & d_bc
304             if np.any(mask_ad_bc):
305                 bracket[mask_ad_bc] += eps_ij_sum
306
307             term1 += np.sum(t_row * bracket * t_col)
308
309         energy += term1 + term2
310
311     return energy
312
313 def _compute_density(self, t_abij: np.ndarray) -> np.
314 ndarray:
315     """
316     Virtual virtual density matrix  $D_{ab} = \sum_{i>j,c} t_{ac}^{ij} t_{bc}^{ij}$ .
317     """
318     nocc = t_abij.shape[2]
319     i_idx, j_idx = np.tril_indices(nocc, k=-1)
320     t_ij = t_abij[:, :, i_idx, j_idx] # (nvir,
321         nvir, n_pairs)
322     D_ab = np.einsum('acp,bcp->ab', t_ij, t_ij, optimize=

```

```

    True)
    D_ab[np.abs(D_ab) < 1e-12] = 0.0
    return D_ab

#
-----

# Gradient and Hessian
#
-----

def _gradient(self, t_abij: np.ndarray, eri_as: np.ndarray,
               D_ab: np.ndarray, fock_spin: np.ndarray) ->
               np.ndarray:
    """
    Gradient  $G[a,e] = 2 \sum_{i>j,b} t_{ab}^{ij} \langle eb || ij \rangle + 2 \sum_b D_{ab} F_{be}$ 
    """
    vir = np.array(self.active_inocc_indices)
    inactive = np.array(self.inactive_indices)
    nvir = len(vir)
    ninact = len(inactive)

    if ninact == 0:
        return np.zeros((nvir, 0))

    nocc = t_abij.shape[2]
    i_idx, j_idx = np.tril_indices(nocc, k=-1)

    #  $\langle eb || ij \rangle$  for  $e$  in inactive,  $b$  in active,  $i>j$ 
    eri_eb = eri_as[nvir:, :nvir, :, :][:, :, i_idx, j_idx]
    # (ninact, nvir, n_pairs)
    t_ij = t_abij[:, :, i_idx, j_idx]
    # (nvir, nvir, n_pairs)

    G = 2.0 * np.einsum('abp,ebp->ae', t_ij, eri_eb,
                        optimize=True)
    F_be = fock_spin[np.ix_(vir, inactive)]
    # (nvir, ninact)
    G += 2.0 * D_ab @ F_be
    return G

def _hessian(self, t_abij: np.ndarray, eri_as: np.ndarray,
              D_ab: np.ndarray, fock_spin: np.ndarray) -> np
              .ndarray:
    """
    Hessian  $H_{\{ae,bf\}} = 2 \sum_{i>j} t_{ab}^{ij} \langle ef || ij \rangle - [\sum_{c,i>j} (t_{ac}^{ij} \langle bc || ij \rangle + t_{cb}^{ij} \langle ca || ij \rangle)] d_{ef} + D_{ab} (f_{aa} + f_{bb}) d_{ef}$ 
    """

```

```

358         + D_ab f_ef (1 - d_ef)
359     """
360     vir = np.array(self.active_inocc_indices)
361     inactive = np.array(self.inactive_indices)
362     nvir = len(vir)
363     ninact = len(inactive)
364
365     if ninact == 0:
366         return np.zeros((0, 0))
367
368     nocc = t_abij.shape[2]
369     i_idx, j_idx = np.tril_indices(nocc, k=-1)
370
371     fock_vir_diag = np.diag(fock_spin)[vir]
372                     # (nvir,)
373     fock_inact = fock_spin[np.ix_(inactive, inactive)]
374                     # (ninact, ninact)
375     fock_inact_offdiag = fock_inact.copy()
376     np.fill_diagonal(fock_inact_offdiag, 0.0)
377
378     t_ij = t_abij[:, :, i_idx, j_idx]
379                     # (nvir, nvir, n_pairs)
380     eri_ef = eri_as[nvir:, nvir:, :, :][:, :, i_idx, j_idx]
381                     # (ninact, ninact, n_pairs)
382     eri_ab = eri_as[:nvir, :nvir, :, :][:, :, i_idx, j_idx]
383                     # (nvir, nvir, n_pairs)
384
385     # Term 1: 2 t_ab^p <ef||ij>^p -> reshape to (a,e,b,f)
386     term1 = 2.0 * np.einsum('abp,efp->aebf', t_ij, eri_ef,
387                             optimize=True)
388     H = term1.reshape(nvir * ninact, nvir * ninact)
389
390     # Terms 2 & 3 (diagonal in e=f)
391     term2 = -(
392         np.einsum('acp,bcp->ab', t_ij, eri_ab, optimize=
393             True) +
394         np.einsum('cbp,cap->ab', t_ij, eri_ab, optimize=
395             True)
396     )
397     term3 = D_ab * (fock_vir_diag[:, None] + fock_vir_diag[
398         None, :])
399     H += np.kron(term2 + term3, np.eye(ninact)).reshape(
400         nvir * ninact, nvir * ninact)
401
402     # Term 4 (off diagonal e!=f): D_ab f_ef
403     H += np.kron(D_ab, fock_inact_offdiag).reshape(nvir *
404         ninact, nvir * ninact)
405
406     return H

```

```

397 #
398 # -----
399 # Orbital rotation and canonicalization
400 # -----
401
402 def _rotate_orbitals(self, mo_coeffs: List[np.ndarray],
403                      fock_spin: np.ndarray,
404                      R_vec: np.ndarray) -> Tuple[List[np.
405                      ndarray], np.ndarray]:
406
407     """
408     Apply orbital rotation  $U = \exp(R)$  where  $R$  is
409     antisymmetric.
410
411     Rotation only affects VIRTUAL orbitals (active +
412     inactive).
413     Occupied orbitals are never rotated.
414
415     Key: Build  $R$  with correct absolute indexing so occupied
416     block stays as identity.
417     """
418     n_occ = len(self.active_occ_indices)           # Occupied
419           spin-orbitals
420     nvir_act = len(self.active_inocc_indices)      # Active
421           virtual spin-orbitals
422     ninact = len(self.inactive_indices)            # Inactive
423           virtual spin-orbitals
424     n_spin = n_occ + nvir_act + ninact             # Total spin
425           -orbitals
426
427     # Reshape  $R\_vec$  into local matrix form:  $R\_local[a, e]$ 
428     R_local = R_vec.reshape(nvir_act, ninact)
429
430     # ===== BUILD FULL ANTISYMMETRIC R MATRIX IN SPIN-
431           ORBITAL SPACE =====
432     # Occupied block ( $0:n\_occ, 0:n\_occ$ ) stays ZERO (
433           identity under exp)
434     # Virtual block ( $n\_occ:n\_spin, n\_occ:n\_spin$ ) contains
435           the rotation
436     R_full = np.zeros((n_spin, n_spin), dtype=np.float64)
437
438     # Fill active-inactive coupling in the VIRTUAL SUBSPACE
439     for a in range(nvir_act):
440         for e in range(ninact):
441             # Absolute spin-orbital indices
442             abs_a = n_occ + a                      #
443                 Active virtual position
444             abs_e = n_occ + nvir_act + e           #
445                 Inactive virtual position

```

```

430
431     # Antisymmetric placement
432     R_ae = R_local[a, e]
433     R_full[abs_e, abs_a] = R_ae
434     R_full[abs_a, abs_e] = -R_ae
435
436     # Verify antisymmetry
437     assert np.allclose(R_full, -R_full.T, atol=1e-12), \
438         f"R matrix not antisymmetric! Max violation: {np.
439             max(np.abs(R_full + R_full.T))}"
440
441     # ===== GENERATE UNITARY U = exp(R) =====
442     U_spin = scipy.linalg.expm(R_full)
443
444     # Verify unitarity
445     assert np.allclose(U_spin.T @ U_spin, np.eye(n_spin),
446         atol=1e-6), \
447         "U^TU != I: Unitary transformation broken!"
448     assert np.allclose(U_spin @ U_spin.T, np.eye(n_spin),
449         atol=1e-6), \
450         "UU^T != I: Unitary transformation broken!"
451
452     # ===== VERIFY OCCUPIED BLOCK IS IDENTITY =====
453     U_occ = U_spin[:n_occ, :n_occ]
454     assert np.allclose(U_occ, np.eye(n_occ), atol=1e-6), \
455         f"Occupied block not identity! Max deviation: {np.
456             max(np.abs(U_occ - np.eye(n_occ)))}"
457
458     # ===== APPLY UNITARY TO MO COEFFICIENTS =====
459     # For unrestricted formalism, apply U separately to
460     # alpha and beta
461     # U_spin has block structure: [U_alpha_block at even
462     # indices]
463     #                                     [U_beta_block at odd
464     # indices]
465     U_alpha = U_spin[0::2, 0::2]
466     U_beta = U_spin[1::2, 1::2]
467
468     mo_rot = [
469         mo_coeffs[0] @ U_alpha,
470         mo_coeffs[1] @ U_beta
471     ]
472
473     # ===== TRANSFORM FOCK MATRIX =====
474     fock_rot = U_spin.T @ fock_spin @ U_spin
475
476     # Verify Hermiticity
477     assert np.allclose(fock_rot, fock_rot.conj().T, atol=1e
478         -8), \
479         "Rotated Fock not Hermitian!"

```



```

472     # ===== VERIFY OCCUPIED-VIRTUAL DECOUPLING =====
473     # After rotation with identity in occupied block,
474     # occ-virt should remain zero
475     n_occ_spin = n_occ
476     n_virt_spin = n_spin - n_occ_spin
477
478     occ_virt_block = fock_rot[:n_occ_spin, n_occ_spin:]
479     occ_virt_norm = np.linalg.norm(occ_virt_block)
480
481     assert occ_virt_norm < 1e-5, \
482         f"ERROR: Occupied-virtual coupling appeared after
483         rotation! Norm: {occ_virt_norm:.2e}\n" \
484         f"This indicates the unitary was not applied
485         correctly or R had occupied elements."
486
487     return mo_rot, fock_rot, U_spin
488
489 def _canonicalize_active(self, mo_coeffs: List[np.ndarray],
490                          fock_spin: np.ndarray,
491                          U_applied) -> Tuple[List[np.ndarray],
492                                              np.ndarray]:
493     """
494     Canonicalize ONLY the ACTIVE VIRTUAL BLOCK after
495     orbital rotation.
496
497     Does NOT touch inactive orbitals or reorder them, this
498     preserves the active-inactive separation that the
499     gradient/Hessian optimization assumes.
500
501     Inactive orbitals remain in their original order (or
502     untouched).
503     Only the ACTIVE virtuals are diagonalized to restore
504     canonical form
505     within that subspace.
506     """
507     n_occ_a, n_occ_b = self.mol.nelec
508     nact_spatial = self.num_opt_virtual_orbs // 2 # Active
509             virtual count
510
511     # Build full spin transformation
512     U_alpha_full = np.eye(fock_spin.shape[0] // 2)
513     U_beta_full = np.eye(fock_spin.shape[0] // 2)
514
515     # ===== ALPHA: Canonicalize ACTIVE virtual block only
516     =====
517     sl_a = slice(n_occ_a, n_occ_a + nact_spatial)
518     if nact_spatial > 0:
519         # Extract ONLY the active-active block from alpha
520         Fock

```

```

511     fock_alpha_spatial = fock_spin[0::2, 0::2]
512     F_alpha_active = fock_alpha_spatial[sl_a, sl_a]
513
514     # Diagonalize ONLY active-active (leaves active-
515       inactive coupling as-is)
516     evals_alpha_active, evecs_alpha_active = np.linalg.
517       eigh(F_alpha_active)
518     U_alpha_full[sl_a, sl_a] = evecs_alpha_active
519
520     # if self.verbose:
521     #     self._print(f"          [Canon Active] Alpha
522       active virtuals: "
523         f"min={evals_alpha_active[0]:+.6f},
524         max={evals_alpha_active[-1]:+.6f}")
525
526     # ===== BETA: Canonicalize ACTIVE virtual block only
527     =====
528     sl_b = slice(n_occ_b, n_occ_b + nact_spatial)
529     if nact_spatial > 0:
530         # Extract ONLY the active-active block from beta
531         Fock
532         fock_beta_spatial = fock_spin[1::2, 1::2]
533         F_beta_active = fock_beta_spatial[sl_b, sl_b]
534
535         # Diagonalize ONLY active-active (leaves active-
536           inactive coupling as-is)
537         evals_beta_active, evecs_beta_active = np.linalg.
538           eigh(F_beta_active)
539         U_beta_full[sl_b, sl_b] = evecs_beta_active
540
541         # if self.verbose:
542         #     self._print(f"          [Canon Active] Beta
543           active virtuals: "
544             f"min={evals_beta_active[0]:+.6f},
545             max={evals_beta_active[-1]:+.6f}")
546
547     # ===== BUILD SPIN-ORBITAL TRANSFORMATION =====
548     U_canon_spin = np.zeros_like(fock_spin)
549     U_canon_spin[0::2, 0::2] = U_alpha_full
550     U_canon_spin[1::2, 1::2] = U_beta_full
551
552     # ===== APPLY CANONICALIZATION =====
553     mo_canon = [
554         mo_coeffs[0] @ U_alpha_full,
555         mo_coeffs[1] @ U_beta_full
556     ]
557
558     fock_canon = U_canon_spin.T @ fock_spin @ U_canon_spin
559
560     # ===== VERIFICATION =====

```

```

551     # Active blocks should now be diagonal
552     if nact_spatial > 0:
553         F_alpha_active_canon = fock_canon[0::2, 0::2][sl_a,
554             sl_a]
555         F_beta_active_canon = fock_canon[1::2, 1::2][sl_b,
556             sl_b]
557
558         off_diag_alpha = np.linalg.norm(
559             F_alpha_active_canon - np.diag(np.diag(
560                 F_alpha_active_canon)))
561         off_diag_beta = np.linalg.norm(F_beta_active_canon
562             - np.diag(np.diag(F_beta_active_canon)))
563
564         assert off_diag_alpha < 1e-6, \
565             f"Alpha active block not diagonal (norm={
566                 off_diag_alpha:.2e})"
567         assert off_diag_beta < 1e-6, \
568             f"Beta active block not diagonal (norm={
569                 off_diag_beta:.2e})"
570
571     # Eigenvalues must be preserved
572     evals_before = np.sort(np.linalg.eigvalsh(fock_spin))
573     evals_after = np.sort(np.linalg.eigvalsh(fock_canon))
574     assert np.allclose(evals_before, evals_after, atol=1e
575         -8), \
576         "Canonicalization changed eigenvalues!"
577
578     # Extract all orbital energies for diagnostics
579     evals_alpha_all = np.diag(fock_canon)[0::2]
580     evals_beta_all = np.diag(fock_canon)[1::2]
581
582     return mo_canon, fock_canon, [evals_alpha_all,
583         evals_beta_all]
584
585     #
586     -----
587
588     # Initial T1 amplitudes and norm
589     #
590     -----
591
592     def _mp1_t1_amplitudes(self, fock_diag: np.ndarray, eri_as:
593         np.ndarray) -> np.ndarray:
594         """
595         Compute MP1 T1 amplitudes  $t_i^a = -\langle a || i \rangle / (f_{aa} -$ 
596              $f_{ii})$ .
597         T1 should vanish at Brueckner orbitals.
598
599         Parameters
600         -----

```

```

586     fock_diag : np.ndarray
587         Diagonal of spin-orbital Fock matrix
588     eri_as : np.ndarray
589         Antisymmetrized ERIs in spin-orbital basis
590
591     Returns
592     -----
593     t_1 : np.ndarray
594         Shape (nvir_act, nocc_act)
595     """
596     occ_idx = self.active_occ_indices
597     vir_idx = self.active_inocc_indices
598
599     if len(vir_idx) == 0 or len(occ_idx) == 0:
600         return np.zeros((0, 0))
601
602     eps_occ = fock_diag[occ_idx]
603     eps_vir = fock_diag[vir_idx]
604     nvir_act = len(vir_idx)
605     nocc_act = len(occ_idx)
606
607     # Denominator: f_aa - f_ii, shape (nvir_act, nocc_act)
608     denom = eps_vir[:, None] - eps_occ[None, :]
609
610     # Extract active subspace: (nvir_act, nvir_act,
611         nocc_act, nocc_act)
612     eri_active = eri_as[:nvir_act, :nvir_act, :nocc_act, :
613         nocc_act]
614
615     # Extract diagonal elements: a=b, i=j -> shape (
616         nvir_act, nocc_act)
617     t1_numerator = np.einsum('aaic->ai', eri_active)
618
619     # T1 amplitudes: t_i^a = -<a||i> / (f_aa - f_ii)
620     # Add small epsilon to avoid division by zero
621     t1 = -t1_numerator / (denom + 1e-14)
622     t1[np.abs(denom) < 1e-10] = 0.0 # Canonical orbitals
623         have f_aa >> f_ii
624
625     return t1
626
627     def _t1_norm(self, t1: np.ndarray) -> float:
628         """Compute Frobenius norm of T1 amplitudes."""
629         return np.linalg.norm(t1.flatten())
630
631     #
632     -----
633
634     # Block-diagonal RLE approximation and regularization
635     #

```

```

630 def _apply_block_diagonal_rle(self, H: np.ndarray) -> np.
        ndarray:
631     """
632     Apply block-diagonal RLE (Reduced Linear Equation)
        approximation to the Hessian.
633
634     Per Adamowicz & Bartlett (1987): only keep diagonal
        blocks H_{ea,eb}
635     where each block corresponds to rotations of ALL active
        orbitals (a,b)
636     against ONE inactive orbital (e).
637
638     Block size: (len_active, len_active) for each inactive
        orbital.
639
640     Parameters
641     -----
642     H : np.ndarray
643         Full Hessian matrix, shape (nvir_act * ninact,
        nvir_act * ninact)
644
645     Returns
646     -----
647     H_block_diag : np.ndarray
648         Block-diagonal approximation of H, same shape as
        input.
649         Off-diagonal blocks (e != f) are zeroed out.
650     """
651     nvir_act = len(self.active_inocc_indices)
652     ninact = len(self.inactive_indices)
653
654     if ninact == 0 or H.size == 0:
655         return H.copy()
656
657     block_size = nvir_act
658     H_block_diag = np.zeros_like(H)
659
660     # Store diagnostics
661     regularization_applied = []
662     block_eigvals = []
663
664     # Process each diagonal block (one per inactive orbital
        )
665     for e_idx in range(ninact):
666         start = e_idx * block_size
667         end = (e_idx + 1) * block_size
668
669         H_block_orig = H[start:end, start:end].copy()

```

```

670         H_block = H_block_orig.copy()
671
672         # Compute eigenvalue properties for diagnostics
673         eigvals = np.linalg.eigvalsh(H_block_orig)
674         min_eig = np.min(eigvals)
675         max_eig_abs = np.max(np.abs(eigvals))
676         det_val = np.linalg.det(H_block_orig)
677         cond_H = np.linalg.cond(H_block_orig) if
            H_block_orig.size > 0 else np.inf
678
679         block_eigvals.append((e_idx, min_eig, max_eig_abs,
            cond_H))
680
681         H_block_diag[start:end, start:end] = H_block
682
683         # Store diagnostics for optional inspection
684         self._rle_diagnostics = {
685             'block_eigvals': block_eigvals,
686             'regularization_applied': regularization_applied,
687             'num_blocks': ninact
688         }
689
690         return H_block_diag
691
692     #
        -----
693
694     # Newton step solver
695     #
        -----
696
697     def _newton_step(self, G: np.ndarray, H: np.ndarray,
698                     iteration: int, start_counting: bool) -> np.
699                     ndarray:
700
701         """
702         Solve  $H \cdot R = -G$  using block-diagonal RLE approximation
703         with adaptive regularization.
704
705         Strategy:
706         1. Apply block-diagonal RLE to extract only diagonal
707            blocks (cheaper, structured)
708         2. Regularize the full block-diagonal matrix for
709            invertibility
710         3. Solve with pure Newton; fallback to damping if solve
711            fails
712
713         Parameters
714         -----
715         G : np.ndarray
716             Gradient vector, shape (nvir_act * ninact,)

```

```

709     H : np.ndarray
710         Full Hessian matrix, shape (nvir_act * ninact,
711             nvir_act * ninact)
712     iteration : int
713         Current iteration number (for diagnostics)
714     start_counting : bool
715         Whether we're in the convergence-tracking phase (
716             unused here, kept for interface)
717
718     Returns
719     -----
720     R : np.ndarray
721         Orbital rotation step vector, same shape as G
722     """
723     g_vec = G.flatten()
724     dim = len(g_vec)
725     grad_norm = np.linalg.norm(g_vec)
726
727     # ===== APPLY BLOCK-DIAGONAL RLE APPROXIMATION =====
728     # Use block-diagonal approximation for better
729     # conditioning
730     H_block = self._apply_block_diagonal_rle(H)
731
732     cond_H_orig = np.linalg.cond(H) if H.size > 0 else np.
733         inf
734     cond_H_block = np.linalg.cond(H_block) if H_block.size
735         > 0 else np.inf
736     eigvals_orig = np.linalg.eigvalsh(H)
737     eigvals_block = np.linalg.eigvalsh(H_block)
738     n_neg_orig = np.sum(eigvals_orig < -1e-10)
739     n_neg_block = np.sum(eigvals_block < -1e-10)
740
741     if self.verbose:
742         self._print(f"           [RLE] Cond(H): {cond_H_orig
743             :.2e} -> {cond_H_block:.2e}, "
744             f"Neg eigvals: {n_neg_orig} -> {
745                 n_neg_block}")
746
747     H = H_block
748
749     self._grad_norm_history.append(grad_norm)
750     if len(self._grad_norm_history) > 20:
751         self._grad_norm_history.pop(0)
752
753     # Pure Newton
754     R = np.linalg.solve(H, -g_vec)
755
756     return R
757
758 #

```

```

752     # Main driver
753     #
754     def run(self, mo_coeffs: List[np.ndarray],
755            fock_spin: Optional[np.ndarray] = None) -> dict:
756         """
757         Run the OVOS optimization loop.
758
759         Parameters
760         -----
761         mo_coeffs : list of np.ndarray
762             Initial MO coefficients [alpha, beta].
763         fock_spin : np.ndarray or None
764             Initial spin orbital Fock matrix. If None, built
765                 from mo_coeffs.
766
767         Returns
768         -----
769         dict with keys:
770             energy_history : list of float
771             converged_energy_history : list of float (only
772                 converged part)
773             iteration_history : list of int
774             final_mo_coeffs : list of np.ndarray
775             final_fock_spin : np.ndarray
776             stop_reasons : list of str
777         """
778         converged = False
779         start_counting = False
780         iter_count = 0
781         keep_track = 0
782
783         # Storage
784         energy_hist = []
785         iter_hist = []
786         stop_reasons = []
787         mo_hist = []
788         fock_hist = []
789         grad_norm_hist = []
790
791         # Initial Fock if not provided
792         if fock_spin is None:
793             Fmo_a = mo_coeffs[0].T @ self.Fao[0] @ mo_coeffs[0]
794             Fmo_b = mo_coeffs[1].T @ self.Fao[1] @ mo_coeffs[1]
795             fock_spin = self._build_spin_fock(Fmo_a, Fmo_b)
796             eig_a = scipy.linalg.eigh(Fmo_a, eigvals_only=True)
797             eig_b = scipy.linalg.eigh(Fmo_b, eigvals_only=True)

```



```

796         self.eps = self._spatial_to_spin_energies(np.real(
797             eig_a), np.real(eig_b))
798     else:
799         self.eps = np.diag(fock_spin)
800
801     # Get HF ||T1|| for convergence comparison
802     eri_as_initial = self._eri_vovo_antisym(mo_coeffs)
803     t1_initial = self._mp1_t1_amplitudes(self.eps,
804         eri_as_initial)
805     t1_norm_initial = self._t1_norm(t1_initial)
806
807     while iter_count < self.max_iter:
808         iter_count += 1
809
810         prev_mo_coeffs = mo_coeffs
811
812         if self.verbose:
813             self._print()
814             self._print(f"#### OVOS Iteration {iter_count}
815                 ####")
816
817         # MP2 calculation
818         eri_as = self._eri_vovo_antisym(mo_coeffs)
819         t_abij = self._mp1_amplitudes(self.eps, eri_as)
820         assert np.allclose(t_abij, t_abij.transpose(1, 0,
821             3, 2), atol=1e-10), \
822             "t_abij antisymmetry broken"
823         E_corr = self._mp2_energy(fock_spin, t_abij, eri_as
824             )
825
826         # Compute T1 norm amplitudes
827         t1_abij = self._mp1_t1_amplitudes(self.eps, eri_as)
828         t1_norm = self._t1_norm(t1_abij)
829
830         # Track T1 norm history
831         if not hasattr(self, '_t1_norm_history'):
832             self._t1_norm_history = []
833         self._t1_norm_history.append(t1_norm)
834
835         # if self.verbose:
836         #     nact = len(self.active_inocc_indices)
837         #     ntot = nact + len(self.inactive_indices)
838         #     self._print(f"      [{nact}/{ntot}]: MP2 energy
839             = {E_corr:.12f}, T1 norm = {t1_norm:.2e}")
840
841         # Track energy difference, ...
842         if not hasattr(self, '_step_history'):
843             self._step_history = []
844         if iter_count > 1:
845             dE = E_corr - energy_hist[-1]

```

```

840         self._step_history.append(dE)
841         if len(self._step_history) > 10:
842             self._step_history.pop(0)
843
844         energy_hist.append(E_corr)
845         iter_hist.append(iter_count)
846         mo_hist.append(mo_coeffs)
847         fock_hist.append(fock_spin)
848
849         if self.verbose:
850             nocc = len(self.active_occ_indices)
851             nact = len(self.active_inocc_indices)
852             ntot = nact + len(self.inactive_indices)
853             self._print(f"      [{nact}/{ntot}]: MP2 energy =
854                        {E_corr:.12f}")
855
856         if iter_count == 1:
857             stop_reasons.append("Initial")
858             if len(self.virtual_inocc_indices) == 0:
859                 break
860         else:
861             # Check convergence
862             dE = abs(energy_hist[-1] - energy_hist[-2])
863             self.dE = energy_hist[-1] - energy_hist[-2]
864
865             D_ab = self._compute_density(t_abij)
866             G = self._gradient(t_abij, eri_as, D_ab,
867                               fock_spin)
868             grad_norm = np.linalg.norm(G.flatten())
869             grad_norm_hist.append(grad_norm)
870             if len(grad_norm_hist) > 5:
871                 grad_norm_hist.pop(0)
872             dgrad = abs(grad_norm_hist[-1] - grad_norm_hist
873                       [-2]) if len(grad_norm_hist) > 1 else None
874
875             if self.verbose and dgrad is not None:
876                 flag = "(energy increased!)" if self.dE > 0
877                     else ""
878                 self._print(f"      dE = {self.dE:.2e}
879                             ||grad|| = {grad_norm:.2e} {flag}")
880                 self._print(f"      ||T1|| = {t1_norm
881                             :.2e}  OVOS < HF = {t1_norm<
882                             t1_norm_initial}  (||T1||^HF = {
883                             t1_norm_initial:.2e}")
884
885             if (dE < self.conv_energy and grad_norm < self.
886                 conv_grad and t1_norm < self.t1_conv):
887                 stop_reasons.append("Convergence")
888                 start_counting = True
889         else:

```

```

881         stop_reasons.append("Non converged")
882         start_counting = False
883         keep_track = 0
884
885     if start_counting: # (dE < self.conv_energy and
886                       # grad_norm < self.conv_grad) or dE < 1e-12:
887         keep_track += 1
888         if self.verbose:
889             # Make sure self.keep_track_max is not
890             # string
891             if isinstance(self.keep_track_max, str):
892                 :
893                 self._print(f"    Keep track: {
894                             keep_track}/{self.keep_track_max}
895                             ")
896             if isinstance(self.keep_track_max, int):
897                 if keep_track >= self.keep_track_max
898                 and iter_count > 100:
899                     if self.verbose:
900                         self._print(f"OVOS converged
901                                     after {iter_count-keep_track}
902                                     iterations")
903                     # Trim the extra tracked steps
904                     trim = self.keep_track_max - 1
905                     energy_hist = energy_hist[:-trim]
906                     iter_hist = iter_hist[:-trim]
907                     mo_hist = mo_hist[:-trim]
908                     fock_hist = fock_hist[:-trim]
909                     stop_reasons = stop_reasons[:-trim]
910                     converged = True
911                     break
912             elif isinstance(self.keep_track_max, str):
913                 if self.keep_track_max.lower() == "none":
914                     :
915                     # Let it run untill iter_count
916                     # reaches max_iter, but we are in
917                     # the converged state
918                     if keep_track >= 1 and iter_count
919                     == 1000:
920                         if self.verbose:
921                             self._print(f"OVOS
922                                         converged after {
923                                             iter_count-keep_track}
924                                             iterations (
925                                                 keep_track_max=None)")
926                     # Trim the extra tracked steps
927                     trim = keep_track - 1
928                     energy_hist = energy_hist[:-
929                                     trim]
930                     iter_hist = iter_hist[:-trim]

```

```

914         mo_hist = mo_hist[:-trim]
915         fock_hist = fock_hist[:-trim]
916         stop_reasons = stop_reasons[:-trim]
917         converged = True
918         break
919     else:
920         keep_track = 0
921
922     # Compute gradient and Hessian for Newton step
923     D_ab = self._compute_density(t_abij)
924     G = self._gradient(t_abij, eri_as, D_ab, fock_spin)
925     H = self._hessian(t_abij, eri_as, D_ab, fock_spin)
926
927     # Store current energy for use in _newton_step
928     self.current_energy = E_corr
929     self.fock_spin = fock_spin
930     self.mo_coeffs = mo_coeffs
931
932     # Solve Newton step
933     R_vec = self._newton_step(G, H, iter_count,
934                               start_counting)
935
936     # Apply rotation
937     mo_coeffs, fock_spin, U_spin = self.
938         _rotate_orbitals(mo_coeffs, fock_spin, R_vec)
939
940     # Recanonicalize the active virtual block
941     mo_coeffs, fock_spin, evals = self.
942         _canonicalize_active(mo_coeffs, fock_spin, U_spin)
943
944     # Update diagonal elements for next MP1 denominator
945     (approximate, but ok)
946     self.eps = np.diag(fock_spin)
947
948     if iter_count >= self.max_iter:
949         if self.verbose:
950             self._print(f"Reached maximum iterations ({self.max_iter})")
951             break
952
953     # --- Summary ---
954     if self.verbose:
955         self._print()
956         self._print("#### OVOS Summary ####")
957         self._print(f"Initial MP2 energy: {energy_hist[0]:.12f} Ha")
958         self._print(f"Final MP2 energy: {energy_hist[-1]:.12f} Ha")

```

```

955     delta = energy_hist[-1] - energy_hist[0]
956     self._print(f"Total change:          {delta:.12f} Ha")
957     if delta < 0:
958         self._print("OVOS lowered the correlation
959                     energy.")
960     else:
961         self._print("WARNING: OVOS increased the
962                     correlation energy.")
963     # Unrestricted or restricted final orbitals
964     if np.allclose(mo_coeffs[0], mo_coeffs[1], atol=1e
965                   -6):
966         self._print("Final orbitals are effectively
967                     restricted.")
968     else:
969         self._print("Final orbitals are unrestricted.")
970     self._print(f"Total iterations: {iter_count-
971                 keep_track} (converged: {converged})")
972     # Difference in prev_mo_coeffs vs final mo_coeffs
973     mo_diff_a = np.linalg.norm(mo_coeffs[0] -
974                               prev_mo_coeffs[0])
975     mo_diff_b = np.linalg.norm(mo_coeffs[1] -
976                               prev_mo_coeffs[1])
977     self._print(f"Change in MO coefficients (Frobenius
978                 norm): "
979                 f"alpha: {mo_diff_a:.2e}, beta: {mo_diff_b
980                 :.2e}")
981     self._print(f"Stopping reason: {stop_reasons[-1]}")
982
983     # Select best result (lowest energy)
984     best_idx = int(np.argmin(energy_hist))
985     result = [
986         energy_hist[best_idx],
987         energy_hist[:best_idx+1],
988         iter_hist[:best_idx+1],
989         mo_hist[best_idx],
990         fock_hist[best_idx],
991         stop_reasons[best_idx]
992     ]
993
994     # if stop_reasons[best_idx] != "Initial":
995     #     self._print(f"Final ||grad|| = {grad_norm:.2e},
996     #                 ||T1|| = {t1_norm:.2e}")
997
998     return result
999
1000 #
1001
1002 # Example usage
1003 #

```

```

-----
993 if __name__ == "__main__":
994     from pyscf import gto, scf
995
996     # Water molecule, minimal basis
997     mol = gto.Mole()
998     mol.atom = 'O 0.0000 0.0000 0.1173; H 0.0000 0.7572
999               -0.4692; H 0.0000 -0.7572 -0.4692'
1000     mol.basis = '6-31G'
1001     mol.unit = 'Angstrom'
1002     mol.spin = 0
1003     mol.charge = 0
1004     mol.symmetry = False
1005     mol.verbose = 0
1006     mol.build()
1007
1008     # RHF reference
1009     mf = scf.RHF(mol)
1010     mf.verbose = 0
1011     mf.kernel()
1012
1013     # Initial data (RHF orbitals)
1014     Fao = [mf.get_fock(), mf.get_fock()]
1015     mo_coeffs = [mf.mo_coeff, mf.mo_coeff]
1016
1017     # Create OVOS object and run
1018     ovos = OVOS(
1019         mol=mol,
1020         scf=mf,
1021         Fao=Fao,
1022         num_opt_virtual_orbs=2,
1023         mo_coeff=mo_coeffs,
1024         init_orbs="RHF",
1025         verbose=1,
1026         max_iter=1000,
1027         conv_energy=1e-8,
1028         conv_grad=1e-6,
1029         keep_track_max=50
1030     )
1031     E_corr, E_corr_hist, E_corr_iter, E_corr_mo, E_corr_fock,
1032     stop_reason = ovos.run(mo_coeffs, fock_spin=None)
1033
1034     print("\nOptimization finished. Final MP2 energy =", E_corr
1035           , "Hartree (Stopping reason:", stop_reason, ")")

```